

1. 介绍

- 1.1 简单介绍
- 1.2 名词
- 1.3 Linux发行版本
- 1.4 Linux内核

2. Linux目录

- 2.1 结构
- 2.2 路径
- 2.3 命令解释器
 - 2.3.1 简单组成
 - 2.3.2 工作原理
 - 2.3.3 快捷键

3. vim

- 3.1 安装
- 3.2 vim的三种模式
- 3.3 命令模式下的操作(注意以下操作都是在vim的命令模式下完成的)
- 3.4 末行模式
- 3.5 vim配置命令

4. GCC

- 4.1 安装
- 4.2 gcc工作流程
- 4.3 GCC的常用参数
- 4.4 多文件编译
- 4.5 GCC与G++区别

5. 动态库和静态库

- 5.1 静态库（无执行权限）
 - 5.1.1 生成静态链接库
 - 5.1.2 静态库使用
- 5.2 动态库（有执行权限）
 - 5.2.1 生成动态链接库
 - 5.2.2 动态库的使用
 - 5.2.3 解决动态库无法加载问题
 - 5.2.3.1 库的工作原理
 - 5.2.3.2 动态链接器
 - 5.2.3.3 解决方案
 - 5.2.3.4 验证
- 5.3 动态库与静态库优缺点

6. makefile（linux自带的编译工具）

- 6.1 规则
- 6.2 工作原理
 - 6.2.1 规则的执行
 - 6.2.2 文件的时间戳
 - 6.2.3 自动推导
- 6.3 变量
 - 6.3.1 自定义变量
 - 6.3.2 预定义变量
 - 6.3.3 自动变量
- 6.4 模式匹配
- 6.5 函数
 - 6.5.1 wildcard
 - 6.5.2 patsubst
- 6.6 makefile编写

7. GDB调式

7.1 调式准备

7.2 启动和退出gdb

7.2.1 启动gdb

7.2.2 命令行传参

7.2.3 gdb中启动程序

7.2.4 退出gdb

7.3 查看代码

7.3.1 当前文件

7.3.2 切换文件

7.3.3 设置显示的行数

7.4 断点操作

7.4.1 设置断点

7.4.2 查看断点

7.4.3 删除断点

7.4.4 设置断点状态

7.5 调式命令

7.5.1 继续运行gdb

7.5.2 手动打印信息

7.5.2.1 打印变量值

7.5.2.2 打印变量类型

7.5.3 自动打印信息

7.5.3.1 设置变量名自动显示

7.5.3.2 查看自动显示列表

7.5.3.3 取消自动显示

7.5.4 单步调式

7.5.4.1 step

7.5.4.2 finish

7.5.4.3 next

7.5.4.4 until

7.6 设置变量值

8. Linux命令

8.1 `cd`: 目录跳转命令

8.2 `ls`: 查看当前目录的文件

8.2.1 显示所有文件（包括隐藏文件）

8.2.2 显示文件详细信息（长文件显示）

8.2.2.1 文件类型

8.2.2.2 用户类型

8.2.2.3 用户权限

8.2.2.4 硬链接计数（起别名，C++引用，共用内存，第二列）

8.2.2.5 其他属性

8.2.3 单位显示

8.2.4 显示目录后缀

8.3 `pwd`: 查看当前的绝对路径

8.4 `touch` 命令: 创建文件

8.5 `whoami`: 查看当前用户

8.6 `su` 用户名: 切换用户

8.7 `which` 命令: 查看命令路径

8.8 `echo` 命令: 打印到屏幕

8.9 `cp` 命令: 拷贝

8.10 `mv` 命令: 移动

8.11 `tree` 命令: 目录结构显示

- 8.12 创建删除目录
- 8.13 查看文件内容
 - 8.13.1 cat
 - 8.13.2 more
 - 8.13.3 less
 - 8.13.4 head
 - 8.13.5 tail
- 8.14 链接创建
- 8.15 修改文件权限
- 8.16 修改文件所有者
- 8.17 修改文件所属组
- 8.18 重定向命令
- 8.19 添加/删除用户
 - 8.19.1 添加用户
 - 8.19.2 删除用户
- 8.20 添加/删除用户组
- 8.21 修改密码
- 8.22 压缩/解压缩命令
 - 8.22.1 tar
 - 8.22.1.1 压缩(.tar.gz . tar.bz2 .tgz)
 - 8.22.1.2 解压缩 (.tar.gz .tar.bz2 .tgz)
 - 8.22.2 zip
 - 8.22.2.1 压缩 (.zip)
 - 8.22.2.2 解压缩 (.zip)
 - 8.22.3 rar
 - 8.22.3.1 压缩 (.rar)
 - 8.22.3.2 解压缩 (.rar)
 - 8.22.4 xz
 - 8.22.4.1 压缩 (.tar.xz)
 - 8.22.4.2 解压缩(.tar.xz)
- 8.23 搜索命令
 - 8.23.1 find
 - 8.23.1.1 文件名 (-name)
 - 8.23.1.2 文件类型 (-type)
 - 8.23.1.3 文件大小 (-size)
 - 8.23.1.4 目录层级
 - 8.23.1.5 同时执行多个操作
 - 8.23.1.5.1 exec
 - 8.23.1.5.2 ok (具有交互)
 - 8.23.1.5.3 xargs
 - 8.23.2 grep
 - 8.23.3 locate

1. 介绍

1.1 简单介绍

- Linux 是 UNIX 操作系统的一个克隆系统, 但是Linux是开源的内核
- Linux是一个基于文件的操作系统 (一切皆文件)

1.2 名词

- GNU: Gnu's Not Unix. 可以理解成一种口号，最早由Richard Stallman呼吁并倡导的，号召软件自由。
- GPL: General Public License. **GNU通用公共许可证**，GPL 授予程序的接受方下述的权利，即 GPL 所倡导的“自由”：
 - 可以以任何目的运行所购买的程序；
 - 在得到程序代码的前提下，可以以学习为目的，对源程序进行修改；
 - 可以对复制件进行再发行；
 - 对所购买的程序进行改进，并进行公开发布。
- LGPL(GNU Lesser General Public License): LGPL是GPL的一个主要为类库使用设计的开源协议。
 - LGPL允许商业软件通过类库引用(link)方式使用LGPL类库而不需要开源商业软件的代码。
 - 采用LGPL协议的开源代码可以被商业软件作为类库引用并发布和销售。
- BSD开源协议: BSD开源协议是一个给予使用者很大自由的协议。基本上使用者可以“为所欲为”，以BSD协议代码为基础做二次开发自己的产品时，需要满足三个条件：
 - 如果再发布的产品中包含源代码，则在源代码中必须带有原来代码中的BSD协议。
 - 不可以用开源代码的作者/机构名字和原来产品的名字做市场推广。
 - BSD代码鼓励代码共享，但需要尊重代码作者的著作权。
- FSF: 自由软件基本会，给GNU提供资金支付的，毕竟没钱难成事啊。
- 自由软件: GNU项目下的所有软件都基于GPL许可证（非GNU项目也可使用GPL），都是自由软件。
- 开源软件: 是美国Open Source Initiative协会定义，软件开放源代码。
- POSIX: (Portable Operating System Interface for Computing Systems) 是由 IEEE 和 ISO/IEC 开发的一套标准。POSIX 标准是对 UNIX 操作系统的经验和实践的总结，**对操作系统调用的服务接口进行了标准化，保证所编制的应用程序在源代码一级可以在多种操作系统上进行移植。**（所有Linux系统遵循POSIX标准）

1.3 Linux发行版本

- 主流的2个版本: 以Debian为基础再次修改的Linux系统（Ubuntu）；以Red Hat为基础再次修改的Linux系统（CentOS）
- 版本

版本名称	网 址	特 点	软件包管理器
Debian Linux	www.w.debian.org	开放的开发模式，并且易于进行软件包升级	apt
Fedora Core	www.w.re_dhat.com	拥有数量庞大的用户，优秀的社区技术支持，并且有许多创新	up2date (rpm) yum (rpm)

版本名称	网 址	特 点	软件包管理器
CentOS	www.w.ce ntos. org	CentOS 是一种对 RHEL (Red Hat Enterprise Linux) 源代码再编译的产物, 由于 Linux 是 开发源代码的操作系统, 并不排斥基于源代码的再分发, CentOS 就是将商业的 Linux 操作系统 RHEL 进行源代码再编译后分发, 并在 RHEL 的基础上修正了不少已知的漏洞	rpm
SUSE Linux	www.w.su se.co m	专业的操作系统, 易用的 YaST 软件包管理系统	YaST (rpm) , 第三方 apt (rpm) 软件库
Gentoo Linux	www.w.ge ntoo. org	高度的可定制性, 使用手册完整	portage
Ubuntu	www.w.ub unt u.co m	优秀易用的桌面环境, 基于 Debian 构建	apt

Table 1

- 安装稳定版的 ubuntu:
 - 官方每年会发布两个版本, 每个版本的版本号由两部分组成: 主版本号 + 副版本号
 - 主版本号为当年年份, 长期支持版的年份为偶数, 测试版年份为奇数
 - 副版本号为月份, 在4月份发布的为相对稳定版, 在10月份发布的为测试版
 - 因此应当选择**主版本号为偶数, 副版本号为 04**的版本, 进行安装使用。

1.4 Linux内核

- Linux 系统从应用角度来看, 分为内核空间和用户空间两个部分。内核空间是 Linux 操作系统的主要部分, 但是仅有内核的操作系统是不能完成用户任务的。
- Linux 的内核主要由 5 个子系统组成: **进程调度、内存管理、虚拟文件系统、网络接口、进程间通信。**
 - 进程调度 SCHED

进程调度指的是系统对进程的多种状态之间转换的策略。Linux 下的进程调度有 3 种策略: SCHED_OTHER、SCHED_FIFO 和 SCHED_RR。

 - SCHED_OTHER: 分时调度策略 (默认), 是用于针对普通进程的时间片轮转调度策略。
 - SCHED_FIFO: 实时调度策略, 是针对**运行的实时性要求比较高、运行时间短**的进程调度策略。(先进先出)
 - SCHED_RR: 实时调度策略, 是针对**实时性要求比较高、运行时间比较长**的进程调度策略。(时间片轮转)
 - 内存管理 MMU

- 内存管理是多个进程间的内存共享策略。在Linux中，内存管理主要说的是虚拟内存。
- 虚拟内存可以让进程拥有比实际物理内存更大的内存，可以是实际内存的很多倍。
- 每个进程的虚拟内存有不同的地址空间，多个进程的虚拟内存不会冲突。
- 将虚拟内存映射到物理内存
- 虚拟文件系统 VFS
 - 在 Linux 下支持多种文件系统，如 ext、ext2、minix、umsdos、msdos、vfat、ntfs、proc、smb、ncp、iso9660、sysv、hpfs、affs 等。
 - 目前 Linux 下最常用的文件格式是 ext2 和 ext3。
- 网络接口
 - Linux 是在 Internet 飞速发展的时期成长起来的，所以 Linux 支持多种网络接口和协议。**网络接口分为网络协议和驱动程序**，网络协议是一种网络传输的通信标准，而网络驱动则是对硬件设备的驱动程序。Linux 支持的网络设备多种多样，几乎目前所有网络设备都有驱动程序。
- 进程间通信
 - Linux 操作系统支持多进程，进程之间需要进行数据的交流才能完成控制、协同工作等功能，Linux 的进程间通信是从 UNIX 系统继承过来的。Linux 下的进程间的通信方式主要有**管道、信号、消息队列、共享内存和套接字（本地套接字、网络套接字）**等方法。

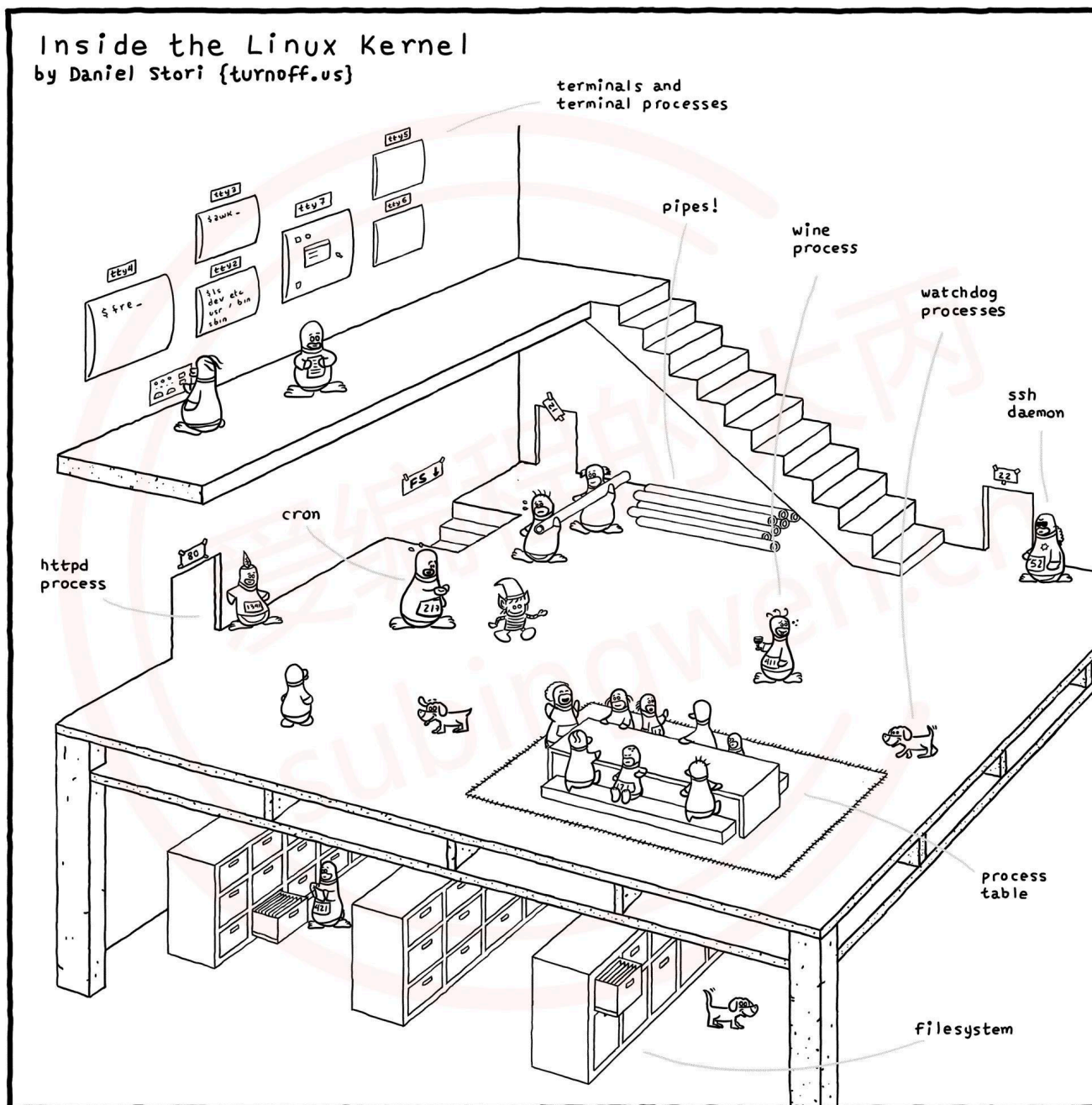


Figure 1

2. Linux目录

- 与 Windows 下的文件组织结构不同，Linux 不使用磁盘分区符号来访问文件系统，而是将整个文件系统表示成树状的结构，Linux 系统每增加一个文件系统都会将其加入到这个树中。
操作系统文件结构的开始，只有一个单独的顶级目录结构，叫做根目录。**所有一切都从“根”开始，用“/”代表**，并且延伸到子目录。Linux 则通过“挂接”的方式把所有分区都放置在“根”下各个目录里。一个 Linux 系统的文件结构如下图所示。

2.1 结构

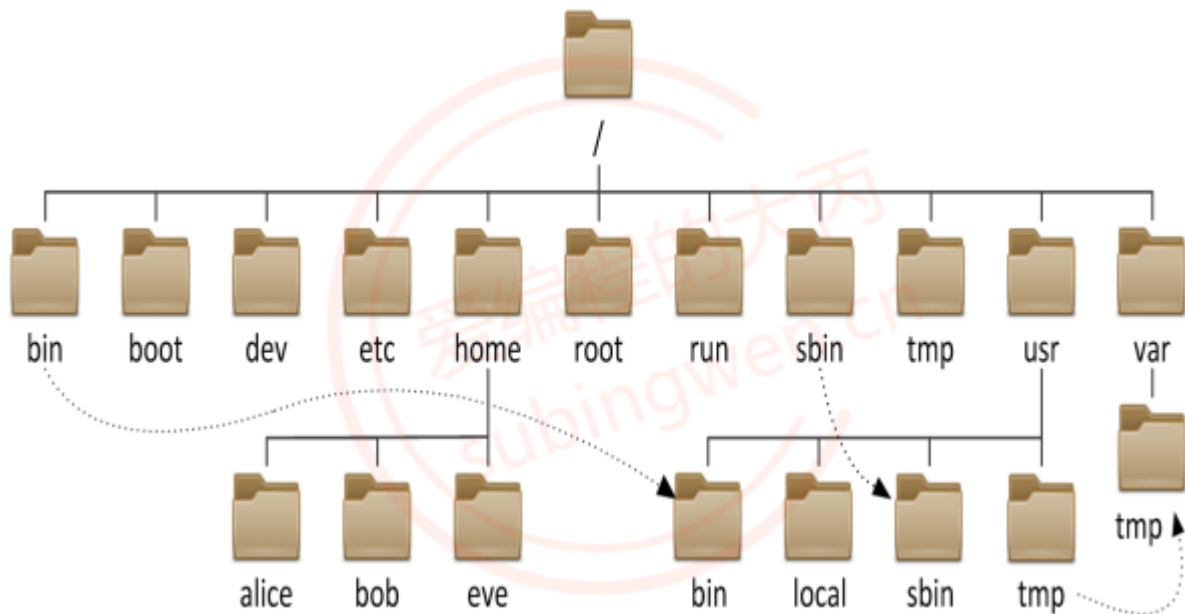


Figure 2

- bin: binary, 二进制文件目录, 存储了可执行程序, 大部分Linux命令都存在在这个目录中
- sbin: super binary, root用户使用的一些二进制可执行程序
- etc: 配置文件目录, 系统的或者用户自己安装的应用程序的配置文件都存储在这个目录中
- lib或者lib64: library, 存储了一些动态库和静态库, 给系统或者安装的软件使用
- media: 挂载目录, 挂载外部设备, 比如: 光驱, 扫描仪
- mnt: 临时挂载目录, 比如我们可以将U盘临时挂载到这个目录下
- proc: 内存使用的一个映射目录, 给操作系统使用的
- run: 给操作系统使用的
- tmp: 临时目录, 存放临时数据, 重启电脑数据就被自动删除了
- boot: 存储了开机相关的设置
- home: 存储了普通用户的家目录, 家目录名和用户名相同
- root: root用户的家目录
- dev: device, 设备目录, Linux中一切皆文件, 所有的硬件会抽象成文件存储起来, 比如: 键盘, 鼠标
- lost+found: 一般时候是空的, 电脑异常关闭/崩溃时用来存储这些无家可归的文件, 用于用户系统恢复
- opt: 第三方软件的安装目录
- var: 存储了系统使用的一些经常会发生变化的文件, 比如: 日志文件
- usr: unix system resource, 系统的资源目录
 - /usr/bin: 可执行的二进制应用程序
 - /usr/games: 游戏目录
 - /usr/include: 包含的标准头文件目录
 - /usr/local: 和opt目录作用相同, 安装第三方软件
 - /usr/src: 源文件

2.2 路径

- 相对路径：相对路径就是相对于当前文件的路径。在Linux中有两个表示路径的特殊符号：
 - `./`：代表目前所在的目录，也可以使用 `.`表示。
 - `../`：代表当前目录的上一层目录，也可以使用 `..`表示
- 绝对路径：从系统磁盘起始节点开始描述的路径。
 - Linux：起始节点为根目录，比如：`/root/luffy/get/onepiece`
 - Windows：起始节点为某个磁盘的盘符，比如：`f:\root\luffy\get\onepiece`

2.3 命令解释器

2.3.1 简单组成

```
1 #ubuntu
2 siguyuan@siguyuan-VMware7-1:/$
3 #CentOS7
4 [root@10 ~]#
```

Fence 1

- root或者siguyuan：当前登录的用户的用户名
- @: at -> 在
- siguyuan-VMware7-1或者10: 主机名, 在安装这个linux操作系统的时候手动指定, 可以修改
- ~: 当前用户的家目录
 - 在linux中有很多用户, 每个用户都用一个属于自己的目录, 这个目录称之为家目录
 - 普通用户家目录/home/用户名, root用户家目录 /root
- #: 代表当前用户是root用户
- \$: 当前用户是普通用户, 也就是说例子中的 robin是一个普通用户

2.3.2 工作原理

- 命令解析器在Linux操作系统中就是一个进程(运行的应用程序), 它的名字叫做**bash**, 通常我们更习惯将其称之为**shell (即: sh)**。他们之间的渊源是这样的, 在Unix操作系统诞生之后一个叫伯恩(Bourne)的人为其编写了命令解析器取名为shell, Linux操作系统诞生之后伯恩再次改写了shell (sh), 将其称之为bash (Bourne Again SHell), **bash 就是 sh 的增强版本**。
- 在Linux操作系统中默认使用的命令解析器是 bash, 当然也同样支持使用sh。当用户打开一个终端窗口, 并输入相关指令, 按回车键, 这时候命令解析器就开始工作了, 具体步骤如下：
 - 在Linux中有一个叫做**PATH的环境变量**, 里边存储了一些系统目录 (windows也有, 叫 Path)

```
1 # 通过 echo 命令可以查看环境变量 PATH 中的值, 在shell中变量名前加 $ 就是取值 :做间隔
2 siguyuan@siguyuan-VMware7-1:~$ echo $PATH
3 /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

Fence 2

- 命令解析器需要依次搜索 PATH中的各个目录, 检查这些目录中是否有用户输入的指令
 - 如果找到了, 执行该目录下的可执行程序, 用户输入的命令就被执行完毕了
 - 如果没有找到, 继续搜索其他目录, 最后还是没有找到, 会提示命令找不到, 因此无法被执行

```

1  #CentOS7
2  [root@10 siguyuan]# ll
3  bash: ll: 未找到命令...
4  相似命令是: 'ls'
5  #ubuntu
6  siguyuan@siguyuan-vmware7-1:~$ ll
7  找不到命令 “ll”，但有 16 个相似命令。

```

Fence 3

2.3.3 快捷键

快捷键	功能	备注
Tab	命令自动补齐	在终端中输入 某个命令的前一个或若干个字符, 再按Tab键 由于很定shell命令的开头字母是相同的, 在这种情况下, 按一次Tab是不会自动补齐的, 我们可以连续按两次Tab键, 在当前终端中就可以显示出所有匹配成功的shell命令 为了能够快速补全shell指令, 我们可以多输入一些前缀字符之后, 再按Tab键
Ctrl+p	显示输入的上一个历史命令	从输入的最后一个命令往前倒, 也可以使用 ↑键
Ctrl+n	显示输入的下一个历史命令	也可以使用 ↓键
Ctrl+a	光标移动命令行首	也可以使用 Home键
Ctrl+e	光标移动命令行尾	也可以使用 End键
Ctrl+u	删除光标前的部分字符串	无
Ctrl+k	删除光标后的部分字符串	无

快捷键	功能	备注
→	光标向右移动一个字符	无
←	光标向左移动一个字符	无
Backspace/Delete	删除光标前/后的一个字符	无

Table 2

3. vim

3.1 安装

- Vim是Linux操作系统中一款功能强大的文本编辑器, 支持安装各种插件。但是vim和windows中的文件编辑器所不同的是它没有UI界面, 所有的操作都是通过键盘快捷键操作完成的, 因此要想熟练使用vim在Linux中进行文本编辑是有成本的, 需要花费一定的时间去练习。

如果我们拿到了一个纯净版的Linux, 里边是没有vim的, 但是有一个类似的文本编辑器叫做Vi。vi编辑器的功能不是很强, 可以这样理解**vim就是vi的增强版**。

首先介绍一下如何在线安装vim, 软件安装需要管理员权限:

- Ubuntu: `sudo apt install vim`
- CentOS: `sudo yum install vim`
- 注意: 上面的在线安装安装的vim不一定是最新版本, 通过 `vim --version` 查看vim的版本号
- 通过 `vimtutor` 查看帮助文档

3.2 vim的三种模式

- 在vim中一共有三种模式, 分别是 **命令模式**, **末行模式**, **编辑模式**, 当我们打开vim之后默认进入的是命令模式。
 - 命令模式: 在该模式下我们可以进行查看文件内容, 简单修改文件, 关键的搜索等操作。
 - 编辑模式: 在该模式下主要对文件内容进行修改和内容添加。
 - 末行模式: 在该模式下可以进行 执行Linux命令, 保存文件, 进行行的跳转, 窗口分屏等操作。
- 三种模式之间是可以相互切换的:
 - 命令模式 -> 编辑模式 -> 命令模式
 - 命令模式 -> 末行模式 -> 命令模式
 - 编辑模式和末行模式之间是不能相互直接切换的
 - 图中shell指的是返回终端

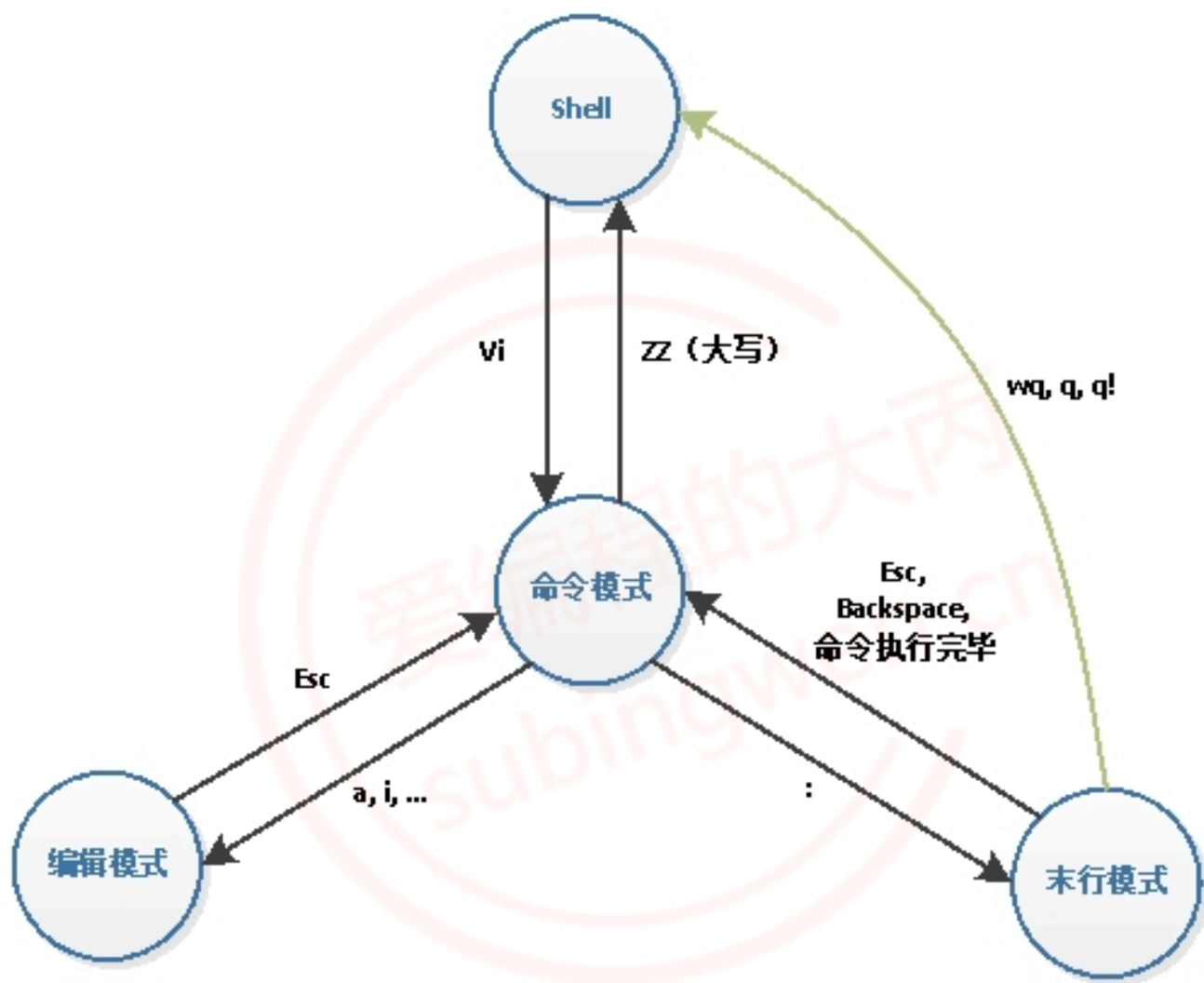


Figure 3

3.3 命令模式下的操作(注意以下操作都是在vim的命令模式下完成的)

- 通过vim打开一个文件, 如果文件不存在, 在退出的时候进行了保存, 文件就会被创建出来

```
1 | # 打开一个文件
2 | $ vim 文件名
```

Fence 4

- 保存退出

直接在键盘上操作, 通过键盘按两个连续的大写的Z (此处是大写的Z, 需要先按住 Shift 再操作哦)

```
1 | # 先按住 shift 键, 然后连续按两次 z
2 | ZZ
```

Fence 5

- 代码格式化

在编码过程中, 为了便于阅读和代码维护, 代码都需要按照样式对其, 如果代码格式凌乱, 可以在命令模式下快速进行代码的格式化, 让其看起来更加美观, 这个操作需要在键盘上连续输入多个字符。

```
1 # 假设写的C/C++代码没有对齐, 通过该命令可以对齐代码
2 # 一定要注意最后一个字符是 大写的 G, 因此需要先按 shift
3 gg=G
```

Fence 6

• 光标移动

在vim中可以使用键盘上的方向键(↑, ↓, ←, →)来移动光标, 这种操作相对比较麻烦, 有一种更加简便的操作方式, 就是使用键盘上的 h, j, k, l。

```
1 # 标准的移动光标的方法: 使用 h, j, k, l
2
3                                     光标上移
4                                     ↑
5                                     |
6 光标左移 <-- h      j      k      l --> 光标右移
7                                     |
8                                     ↓
9                                     光标下移
```

Fence 7

除此之外我们还可以使用一些快捷键实现光标的快速跳转, 常用的有:

快捷键	功能	备注
0	光标移动到行首	无
\$	光标移动到行尾部	通过按两个键: shift + 4完成\$的输入
gg	光标移动到文件头	第一行的开始
G	光标移动到文件尾部	最后一行的开始
nG	行跳转	n 代表要跳转到哪一行
n+回车	相对跳转 n 行	从光标所在当前行往下跳 n 行, n 对应的是一个整数

Table 3

• 删除命令

在vim中是没有删除操作的, 其实**所谓的删除就是剪切**, 被删除的数据都可被粘贴到文档的任意位置, 即便如此我们还是习惯性的将剪切操作称之为删除, 常用的删除操作如下表所示:

快捷键	功能	备注
x (小写)	删除光标后边的字符	vim中的光标比较宽会盖住后边的字符

快捷键	功能	备注
X (大写)	删除光标前边的字符	无
dw	删除单词	要先把光标移动到单词的第一个字母上再删除, 否则单词只能被删除一部分
d0	删除光标前的字符串	从字符串开头到光标当前位置的字符串被删除了
d\$ (D)	删除光标后的字符串	从光标当前位置到字符串尾部的字符串被删除了, 使用 D也行
dd	删除光标所在行	无
ndd	删除n行	从光标所在行开始删除n行, n对应的是一个整数

Table 4

- 撤销和反撤销

撤销和反撤销对应windows中的 ctrl+z和ctrl+y, 但是在vim中使用这两个快捷键是不行的。

快捷键	功能	备注
u	撤销	等价于 windows 中的 ctrl+z
ctrl+r	反撤销	等价于 windows 中的 ctrl+y

Table 5

- 复制和粘贴

前边已经介绍了, 在vim中做删除操作就相当于剪切, 剪切或者复制之后的数据都可以用来做粘贴操作, 在vim中对应的快捷键如下

快捷键	功能	备注
p	粘贴到光标所在行的下边	小写的 p
P	粘贴到光标所在行的上边	大写的 P
yy	复制光标所在行	无
nyy	从光标所在行向下复制 n 行	n是要复制的行数, 代表一个整数

Table 6

- 可视模式

在编辑文件的过程中, 有时候需要删除或者需要复制的数据不整行的, 而是一行中的某一部分, 这时候可以使用可视模式进行文本的选择, 之后再通过相关的快捷键对所选中的数据块进行复制或者删除操作。

有三种方式可以切换到可视模式:

- v: 进入的字符可视化模式 (Characterwise visual mode), 文本选择是以**字符为单位的**。
- V: 进入的行可视化模式 (Linewise visual mode), 文本选择是以**行为单位的**。
- ctrl-v: 进入的块可视化模式 (Blockwise visual mode) , 可以选择一个**矩形内的文本**。

进入到可视模式之后, 就可以进行文本块的选择和复制以及删除了

快捷键	功能	备注
h	光标向左移动	移动光标用于可视模式下的数据块选择
j	光标向下移动	移动光标用于可视模式下的数据块选择
k	光标向上移动	移动光标用于可视模式下的数据块选择
l	光标向右移动	移动光标用于可视模式下的数据块选择
d	删除(剪切)	删除可视模式下选中的数据块
y	复制	复制可视模式下选中的数据块
p (小写)	数据粘贴到光标的后边	粘贴在可视模式下复制或者剪切的数据块
P (大写)	数据粘贴到光标的前边	粘贴在可视模式下复制或者剪切的数据块

Table 7

◦ 字符可视模式

控制光标方向用来选择文件中的不规则数据块, 可以对选中的文本信息进行复制和删除

```
1 # 进入到字符可视模式, 直接在键盘上按 v 即可:
2 v
```

Fence 8

通过 v 切换到字符可视模式之后, 在窗口的最下方会看到 -- VISUAL-- 字样。

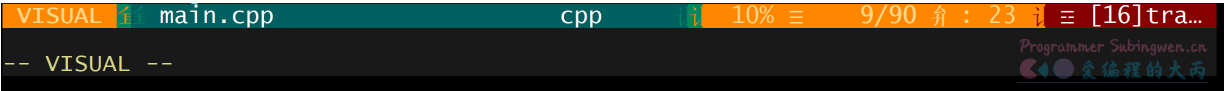


Figure 4

◦ 行可视模式

向下移动光标可以选择一整行, 向上移动光标可以取消整行选择

```
1 # 进入行可视模式, 键盘上按 shift+v
2 v
```

Fence 9

通过 V 切换到行可视模式之后, 在窗口的最下方会看到 -- VISUAL LINE -- 字样。



Figure 5

◦ 块可视化模式

通过向上，下移动光标控制矩形文本块的高度，通过向左，右移动光标控制矩形文本块的宽度。

```
1 | # 进入块可视模式，选择一个矩形文本块
2 | ctrl+v
```

Fence 10

通过 ctrl+v 切换到块可视模式之后，在窗口的最下方会看到 -- VISUAL BLOCK -- 字样。



Figure 6

• 代码注释

代码块注释可以使用块可视模式，具体操作步骤如下：

- 1. 通过 ctrl+v 进入块可视模式
- 2. 移动光标上移 (k) 或者下移 (j)，选中多个代码行的开头，如下图所示：

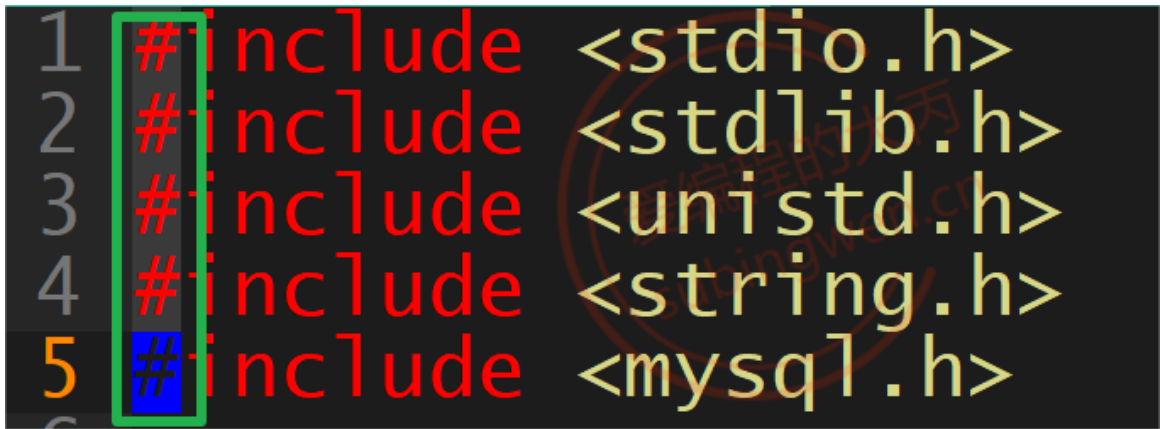


Figure 7

- 3. 选择完毕后，按大写的的I键，此时下方会提示进入“insert”模式，输入你要插入的注释符，例如：//
- 4. 最后按ESC键，你就会发现选中的多行代码已经被注释了

删除多行注释的方法，同样 Ctrl+v 进入列选择模式，移动光标把要删除的注释符选中，按下 d，注释就被删除了。

• 替换

命令模式下替换功能并不强，常用于某个单字符的替换。

快捷键	功能	备注
r	替换光标后的单个字符	无
R	替换光标后的多个字符	按 esc 结束替换

Table 8

• 查找

在vim的命令模式下一共有三种查找方式, 首先需要在键盘上输入对应的字符, 然后按回车键vim会进行关键字匹配, 之后就可以通过 n或者N进行关键字之间的切换了。

快捷键+搜索的词

快捷键	关键字遍历	描述	备注
/	n	从当前位置向下	直接按键盘上的 /即可
	N	从当前位置向上	
?	n	从当前位置向下	直接按键盘上的 ?即可, 需要使用组合键
	N	从当前位置向上	
#	n	从当前位置向下	光标需要先放在被搜索的关键字上, 键盘上按 #
	N	从当前位置向上	

Table 9

关于 ? 和 # 都需要使用组合键, 这点要注意一下。

下面总结一下这三种搜索方式:

- 使用 / 或者 ? 搜索效果一样, 只是遍历关键字的时候的顺序是相反的
- 使用 # 必须先从被搜索的文件中找到要搜索的关键字, 好处就是搜索的内容不需要通过键盘输入
- 以上两种搜索方式各有优劣, 请根据实际情况选择使用。

• 查看man文档

man 文档, 是Linux中默认自带的帮助文档, 作为程序猿可以通过这个文档来查询shell命令或者标准API函数或者系统自带的配置文件格式的说明等信息。

man文档一共有9个章节, 具体如下:

章节	说明
section 1	Linux提供的所有shell命令
section 2	系统函数 (由内核提供的)
section 3	库调函数(程序库中的函数 (C 相关的函数)
section 4	特殊文件(通常在/dev目录中可以找到)
section 5	系统配置文件格式和约定, 比如: /etc/passwd
section 6	游戏 (如果有的话)
section 7	杂项(包括宏包和约定)
section 8	系统管理命令(通常仅针对root用户)
section 9	内核例程[非标准]

Table 10

```

1  # 打开 man 文档首页
2  $ man man
3  # 退出 man 文档，直接按键盘上的 q 即可
4  q
5
6
7  # 那么，我们如何通过 man 文档查询相关的shell命令或者函数等信息呢？
8  # 下边举几个例子：
9
10 # 查询第一章的shell命令
11 $ man 1 cp
12
13 # 查询第二章的系统函数（如：read, write, open 等）
14 $ man 2 read
15
16 # 查询第三章的标准的库函数（如：fread, fwrite, fopen 等）
17 $ man 3 fread
18
19 # 查询第五章的特殊的配置文件说明，比如：/etc/passwd 或者 /etc/group
20 $ man 5 passwd
21
22

```

Fence 11

#查询的时候章节号是可以省略的，只是查到的结果不精确。如果不写章节号，从第一章开始搜索查询的关键词，如果查询到了，直接就结束了。也就是说如果查询的是函数，但是这个函数和某个命令的名字是相同的，查询到第一章搜索就结束了。

如果当前是在vim的命令模式下，我们可以直接跳转到man 文档：

- 找到要查看的函数，然后将光标放到该函数上
- 在键盘上依次输入: **章节号(可选) + K(shift+k)(大写的k)**，就会自动调整到 man 文档中了

• 切换文本编辑模式

如果要编辑文件，需要从命令模式切换到文件编辑模式，切换模式的快捷键有很多，不同的快捷键对应的效果有所不同，效果如下表所示：

快捷键	功能
i	从光标前边开始输入
a	从光标的后边开始输入
o	在光标下边创建新行，在新行中输入
s	删除光标覆盖的字符，从删除的字符位置开始输入
I(大写的i)	从当前行行首开始输入
A	从当前行行尾开始输入
O	在光标上边创建新行，在新行中输入

快捷键	功能
S	删除当前行, 在当前行开始输入

Table 11

文件编辑完成之后, 从编辑模式回到命令模式只需要按键盘上的Esc即可。

3.4 末行模式

- 命令模式到末行模式

从命令模式切换到末行模式只需要在键盘上输入一个:，同时这个符号也会出现在窗口的最下端，这时候我们就可以在最后一行输入我们执行的命令了。

```
1  # 命令模式切换到末行模式
2  在命令模式下键盘输入一个 冒号 -> :
3
4  # 从未行模式 -> 命令模式
5  1. 按两次esc
6  2. 在末行模式下执行一个完整指令，执行完毕，自动回到命令模式
```

Fence 12

- 保存退出

使用vim对文件编辑完成之后, 需要保存或者退出vim一般都是在末行模式下完成的, 不管是进行那种操作都有对应的操作命令, 如下表:

末行模式下输入的命令	功能
q	退出, 如果退出的时候文件没有保存, vim会提示是否要保存
q!	直接退出, 不保存 (强制退出)
w	保存, 不退出 (相当在windows中于按了ctrl+s)
wq	保存退出
x	保存退出

Table 12

- 替换

末行模式下的替换比命令模式下的替换功能要强大的多, 在末行模式下可以指定将什么样的内容替换为什么样的内容, 并且可以指定替换某一行或者某几行或者是全文替换。

替换对应的命令是 s并且可以给其指定参数，默认情况下只替换相关行的第一个满足条件的关键字，**如果需要整行替换需要加参数/g。**


```
1  # 在vim打开文件的时候指定打开多个文件和分屏方式
2  # 水平分屏
3  $ vim -o 文件1, 文件2, 文件3 ...
4  # 垂直分屏
5  $ vim -O 文件1, 文件2, 文件3 ...
```

Fence 13

- 行跳转

在vim中不仅可以在命令模式下进行行的跳转,也可以在末行模式下进行行跳转,末行模式下指定哪一行光标就可以跳转到哪一行。

```
1  :行号 # 输入完行号之后敲回车
```

Fence 14

- 执行shell命令

在使用vim编辑文件的过程中,还可以在末行模式下执行需要的shell命令,在执行shell命令之前需要在前边加上一个叹号!。

```
1  # 语法:
2  :!shell命令
3
4  # 举例
5  :!ls # 回车即可
```

Fence 15

3.5 vim配置命令

- vim 是一个文本编辑器工具,这个工具也是有配置文件的,文件的名字叫做vimrc,在里边可以设置样式,功能,快捷键等属性。**对应的配置文件分为两种 用户级别和系统级别。**
 - 用户级别的配置文件 (~/.vimrc) 只对当前用户有效

```
1  - 系统级别的配置文件 (/etc/vim/vimrc) 对所有Linux用户都有效
2
3  - 如果两个配置文件都设置了, 用户级别的配置文件起作用 (用户级别优先级高)。
```

Fence 16

- vim插件(vimplus)
 - vimplus项目的github地址: <https://github.com/chxuan/vimplus>
 - Linux 64-bit安装
 - 支持以下发行版



Figure 9

■ 安装vimplus

```
1  git clone https://github.com/chxuan/vimplus.git ~/.vimplus
2  cd ~/.vimplus
3  ./install.sh //不加sudo
4
5  # 后续出现Y/N 选择Y
6  # 出现选择python2或者python3，随意
```

Fence 17

■ 设置Nerd Font

为防止vimplus显示乱码，需设置linux终端字体为Droid Sans Mono Nerd Font。

■ 多用户支持

将vimplus在某个用户下安装好后，若需要在其他用户也能够使用vimplus，则执行

```
sudo ./install_to_user.sh username1 username2 //替换为真实用户名
```

■ 更新vimplus

```
./update.sh
```

○ 自定义

- ~/.vimrc为vimplus的默认配置，一般不做修改
- ~/.vimrc.custom.plugins为用户自定义插件列表，用户增加、卸载插件请修改该文件
- ~/.vimrc.custom.config为用户自定义配置文件，一般性配置请放入该文件，可覆盖~/.vimrc里的配置

○ 插件列表

■ 插件	说明
cpp-mode	提供生成函数实现、函数声明/实现跳转、.h .cpp切换等功能(I'm author 😊)
vim-edit	方便的文本编辑插件(I'm author 😊)
change-colorscheme	随心所欲切换主题(I'm author 😊)
prepare-code	新建文件时，生成预定义代码片段(I'm author 😊)
vim-buffer	vim缓存操作(I'm author 😊)
vimplus-startify	vimplus开始页面(修改自 mhinz/vim-startify)
tagbar	使用 preservim/tagbar 的最新版， taglist 的替代品，显示类/方法/变量

插件	说明
vim-plug	比Vundle下载更快的插件管理软件
YouCompleteMe	史上最强大的基于语义的自动补全插件，支持C/C++、C#、Python、PHP等语言
NerdTree	代码资源管理器
vim-nerdtree-syntax-highlight	NerdTree文件类型高亮
nerdtree-git-plugin	NerdTree显示git状态
vim-devicons	显示文件类型图标
Airline	可以取代powerline的状态栏美化插件
auto-pairs	自动补全引号、圆括号、花括号等
LeaderF	比ctrlp更强大的文件的模糊搜索工具
ack	强大的文本搜索工具
vim-surround	自动增加、替换配对符的插件
vim-commentary	快速注释代码插件
vim-repeat	重复上一次操作
vim-endwise	if/end/endif/endfunction补全
tabular	代码、注释、表格对齐
vim-easymotion	强大的光标快速移动工具，强大到颠覆你的插件观
incsearch.vim	模糊字符搜索插件
vim-fugitive	集成Git
gv	显示git提交记录
vim-slash	优化搜索，移动光标后清除高亮
echodoc	补全函数时在命令栏显示函数签名
vim-smooth-scroll	让翻页更顺畅
clever-f.vim	强化F和f键

Table 15

◦ 快捷键

- 以下是部分快捷键，可通过vimplus的 ,h 命令查看[vimplus帮助文档](#)。

快捷键	说明
	Leader Key

快捷键	说明
<code><leader>n</code>	打开/关闭代码资源管理器
<code><leader>t</code>	打开/关闭函数列表
<code><leader>a</code>	.h .cpp 文件切换
<code><leader>u</code>	转到函数声明
<code><leader>U</code>	转到函数实现
<code><leader>u</code>	转到变量声明
<code><leader>o</code>	打开include文件
<code><leader>y</code>	拷贝函数声明
<code><leader>p</code>	生成函数实现
<code><leader>w</code>	单词跳转
<code><leader>f</code>	搜索~目录下的文件
<code><leader>F</code>	搜索当前目录下的文本
<code><leader>g</code>	显示git仓库提交记录
<code><leader>G</code>	显示当前文件提交记录
<code><leader>gg</code>	显示当前文件在某个commit下的完整内容
<code><leader>ff</code>	语法错误自动修复(FixIt)
<code><c-p></code>	切换到上一个buffer
<code><c-n></code>	切换到下一个buffer
<code><leader>d</code>	删除当前buffer
<code><leader>D</code>	删除当前buffer外的所有buffer
<code>vim</code>	运行vim编辑器时,默认启动开始页面
<code><F5></code>	显示语法错误提示窗口
<code><F9></code>	显示上一主题
<code><F10></code>	显示下一主题
<code><leader>l</code>	按竖线对齐
<code><leader>=</code>	按等号对齐
<code>Ya</code>	复制行文本到字母a

快捷键	说明
<code>Da</code>	剪切行文本到字母a
<code>Ca</code>	改写行文本到字母a
<code>rr</code>	替换文本
<code><leader>r</code>	全局替换，目前只支持单个文件
<code>rev</code>	翻转当前光标下的单词或使用V模式选择的文本
<code>gcc</code>	注释代码
<code>gcap</code>	注释段落
<code>vif</code>	选中函数内容
<code>dif</code>	删除函数内容
<code>cif</code>	改写函数内容
<code>vaf</code>	选中函数内容（包括函数名 花括号）
<code>daf</code>	删除函数内容（包括函数名 花括号）
<code>caf</code>	改写函数内容（包括函数名 花括号）
<code>fa</code>	查找字母a，然后再按f键查找下一个
<code><leader>e</code>	快速编辑~/.vimrc文件
<code><leader>s</code>	重新加载~/.vimrc文件
<code><leader>vp</code>	快速编辑~/.vimrc.custom.plugins文件
<code><leader>vc</code>	快速编辑~/.vimrc.custom.config文件
<code><leader>h</code>	打开vimplus帮助文档
<code><leader>H</code>	打开当前光标所在单词的vim帮助文档
<code><leader><leader>t</code>	生成try-catch代码块
<code><leader><leader>y</code>	复制当前选中到系统剪切板
<code><leader><leader>i</code>	安装插件
<code><leader><leader>u</code>	更新插件
<code><leader><leader>c</code>	删除插件

Table 16

◦ FAQ

■ vimplus怎么安装新插件？

编辑[~/.vimrc.custom.plugins](#)，添加自定义插件。

- **vimplus怎么添加自定义配置？**

编辑`~/.vimrc.custom.config`，添加自定义配置。

- **vimplus安装脚本会在自己电脑上安装哪些软件？**

网络良好情况下，vimplus只需30分钟左右即可将vim cpp环境配置好，vimplus真正的做到了一键配置，不让用户操心。vimplus会安装一些必备软件，比如说python、cmake、gcc、fontconfig等，vimplus也考虑到了有些系统的vim不支持python，它会自动去下载vim源码将python支持编译进去，vimplus也会安装nerd-font不让vim显示出现乱码，最最重要的是vimplus实现了ycm自动编译安装，给折腾了几天ycm都没有安装好的用户带来了新的希望，而且vimplus也支持macos和linux众多发行版，让linux发烧友频繁切换发行版而不用操心vim环境配置。最后说了这么多，不如看[vimplus安装脚本](#)来的直接 😊。

- **启动vim报错：RequestsDependencyWarning: old version of cryptography ([1, 2, 3]) may cause slowdown.**

可以尝试将cryptography删掉，具体见[issues #208](#)。

- **vimplus不支持目前用户正在使用的系统怎么办？**

可以给作者提[issues](#)，或者自己fork vimplus来修改，并提交pr，贡献自己的一份力量。

- **安装vimplus后Airline等插件有乱码，怎么解决？**

linux和mac系统需设置终端字体为 `Droid Sans Mono Nerd Font`。

- **xshell连接远程主机不能使用vim-devicons或乱码。**

windows系统安装[Nerd Font](#)字体后并更改xshell字体即可。

- **ubuntu18.04安装了nerd font但通过终端属性并没有看到该字体。**

可以试试dconf-editor软件来设置，可以参考[这里](#)。

- **使用第三方库时怎么让ycm补全第三方库API？**

vimplus安装完毕之后，`~`目录下将会生成两个隐藏文件分别是.vimrc和.ycm_extra_conf.py，其中.vimrc是vim的配置文件，.ycm_extra_conf.py是ycm插件的配置文件，当你需要创建一个project时，需要将.ycm_extra_conf.py拷贝到project的顶层目录，通过修改该配置文件里面的 `flags` 变量来添加你的第三方库路径。

- **使用vi命令报错：E492: Not an editor command:**

vimplus安装完成后，linux下可能会同时存在vi和vim命令，执行vi时，vi加载`~/.vimrc`文件可能会报错，但不影响使用，如果要消除错误可以设置软链接 `ln -s /usr/bin/vim /usr/bin/vi`

- **怎么自定义文件头，比如说添加作者、创建时间？**

你可以修改[chxuan/prepare-code](#)插件来达到目的，可以参考[这里](#)。

- **安装vimplus在“[95%] Building CXX object**

`ycm/CMakeFiles/ycm_core.dir/ycm_core.cpp.o”`等进度时出现编译报错

编译ycm需要消耗较大内存，建议内存大于1G，实在不行也可以开启linux swap分区。

- **以上没有我遇到的问题怎么办？**

您可以通过上网找解决方法，或提[issues](#)，也可以通过加QQ `787280310`、发邮件方式 `787280310@qq.com` 一起讨论解决方法。

- 无法翻墙的这种解决方案

- https://www.bilibili.com/video/BV1rA41157qK/?vd_source=5f8610a33e469a605635adbbdcc8c98e

4. GCC

- GCC 是 Linux 下的编译工具集，是 GNU Compiler Collection 的缩写，包含 gcc、g++ 等编译器。这个工具集不仅包含编译器，还包含其他工具集，例如 ar、nm 等。

GCC 工具集不仅能编译 C/C++ 语言，其他例如 Objective-C、Pascal、Fortran、Java、Ada 等语言均能进行编译。GCC 在可以根据不同的硬件平台进行编译，即能进行交叉编译，在 A 平台上编译 B 平台的程序，支持常见的 X86、ARM、PowerPC、mips 等，以及 Linux、Windows 等软件平台。

4.1 安装

```
1  # 安装软件必须要有管理员权限
2  # ubuntu
3  $ sudo apt update          # 更新本地的软件下载列表，得到最新的下载地址
4  $ sudo apt install gcc g++ # 通过下载列表中提供的地址下载安装包，并安装
5
6  # centos
7  $ sudo yum update          # 更新本地的软件下载列表，得到最新的下载地址
8  $ sudo yum install gcc g++ # 通过下载列表中提供的地址下载安装包，并安装
9
10
11 # 查看 gcc 版本
12 $ gcc -v
13 $ gcc --version
14
15 # 查看 g++ 版本
16 $ g++ -v
17 $ g++ --version
18
19 # 4.8.5及以上才有C++11的特性
```

Fence 18

4.2 gcc工作流程

- GCC 编译器对程序的编译如下图所示，分为 4 个阶段：**预处理（预编译）、编译和优化、汇编和链接**。GCC 的编译器可以将这 4 个步骤合并成一个。先介绍一个每个步骤都分别做了写什么事儿：
 - 预处理: 在这个阶段主要做了三件事: **展开头文件、宏替换、去掉注释行**
 - 这个阶段需要GCC调用预处理器来完成, 最终得到的还是源文件, 文本格式
 - 编译: 这个阶段需要GCC调用编译器对文件进行编译, 最终得到一个汇编文件
 - 汇编: 这个阶段需要GCC调用汇编器对文件进行汇编, 最终得到一个二进制文件
 - 链接: 这个阶段需要GCC调用链接器对程序需要调用的库进行链接, 最终得到一个可执行的二进制文件

文件名后缀	说明	gcc 参数
.c	源文件	无
.i	预处理后的 C 文件	-E
.s	编译之后得到的汇编语言的源文件	-S
.o	汇编后得到的二进制文件	-c

Table 17

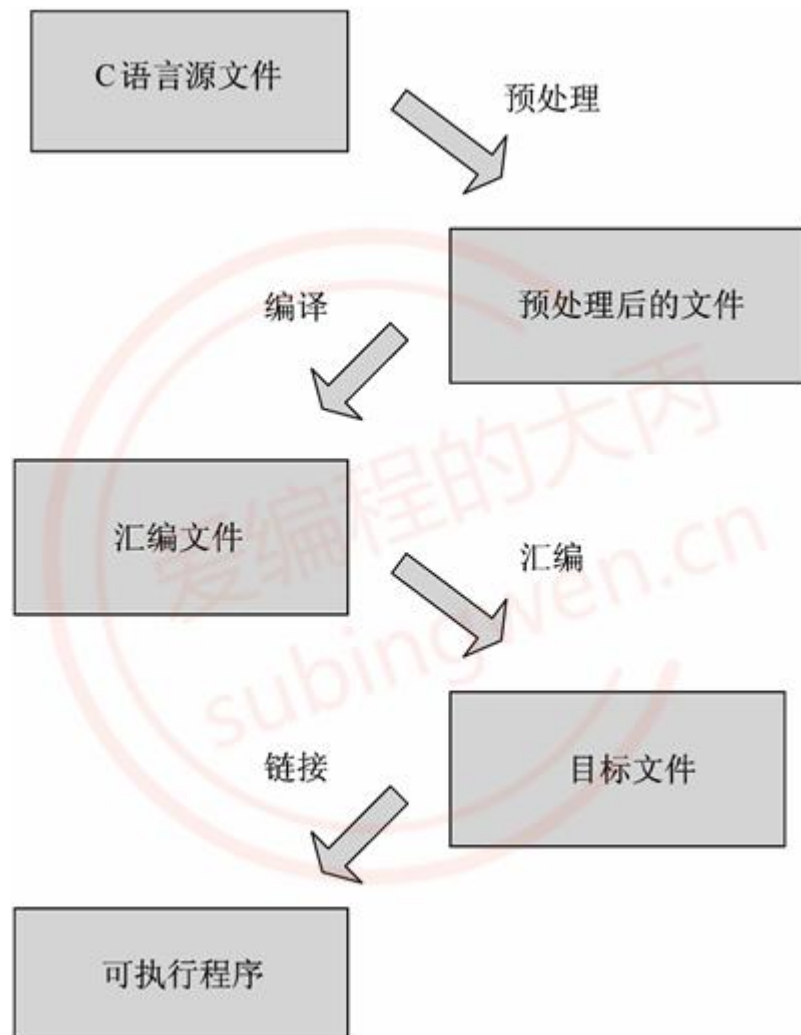


Figure 10

- 在 Linux 下使用 GCC 编译器编译单个文件十分简单，直接使用 gcc 命令后面加上要编译的 C 语言的源文件，GCC 会自动生成文件名为 a.out 的可执行文件（也可以通过参数 -o 指定生成的文件名），也就是通过一个简单的命令上边提到的4个步骤就全部执行完毕了。但是如果想要单步执行也是没问题的，下边基于这段示例程序给大家演示一下。（.c用GCC编译，.cpp用G++编译）

```

1 // 假设程序对应的源文件名为 test.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <string.h>

```

```

6
7   int main()
8   {
9       int array[5] = {1,2,3,4,5};
10      for(int i=0; i<5; ++i)
11      {
12          printf("array[%d] = %d\n", i, array[i]);
13      }
14      return 0;
15  }
16
17
18  # 1. 预处理, -o 指定生成的文件名
19  $ gcc -E test.c -o test.i
20
21
22  # 2. 编译, 得到汇编文件
23  $ gcc -S test.i -o test.s
24
25
26  # 3. 汇编
27  $ gcc -c test.s -o test.o
28
29
30  # 4. 链接
31  $ gcc test.o -o test
32
33
34  # 自动执行前面的步骤
35  # 参数 -c 是进行文件的汇编, 汇编之前的两步会自动执行
36  $ gcc test.c -c -o app.o
37
38  # 该命令是直接进行链接生成可执行程序, 链接之前的三步会自动执行
39  $ gcc test.c -o app
40

```

Fence 19

4.3 GCC的常用参数

- 下面的表格中列出了常用的一些gcc参数, 这些参数在 gcc命令中没有位置要求, 只需要编译程序的时候将需要的参数指定出来即可。

gcc编译选项	选项的意义
-E	预处理指定的源文件, 不进行编译
-S	编译指定的源文件, 但是不进行汇编
-c	编译、汇编指定的源文件, 但是不进行链接

gcc编译选项	选项的意义
-o [file1] [file2] / [file2] -o [file1]	将文件 file2 编译成文件 file1
-I directory (大写的i)	指定 include 包含文件的搜索目录
-g	在编译的时候，生成调试信息，该程序可以被调试器调试
-D	在程序编译的时候，指定一个宏
-w	不生成任何警告信息, 不建议使用, 有些时候警告就是错误
-Wall	生成所有警告信息
-On	n的取值范围：0~3。编译器的优化选项的4个级别，-O0表示没有优化，-O1为缺省值，-O3优化级别最高
-l	在程序编译的时候，指定使用的库
-L	指定编译的时候，搜索的库的路径。
-fPIC / fpic	生成与位置无关的代码
-shared	生成共享目标文件。通常用在建立共享库时
-std	指定C方言，如:-std=c99，gcc默认的方言是GNU C，Linux下位-std=gnu99

Table 18

指定生成的文件名 (-o)

```

1  # 该参数用于指定原文件通过 gcc 处理之后生成的新文件的名字，有两种写法，原文件可以写在参数 -o
   # 前边后缀写在后边。
2  # 参数 -o的用法，原材料 test.c 最终生成的文件名为 app
3  # test.c 写在 -o 之前
4  $ gcc test.c -o app
5
6  # test.c 写在 -o 之后
7  $ gcc -o app test.c

```

Fence 20

搜索头文件 (-I)

```

1  # 如果在程序中包含了一些头文件，但是包含的一些头文件在程序预处理的时候因为找不到无法被展开，
   # 导致程序编译失败，这时候我们可以在gcc命令中添加 -I参数重新指定要引用的头文件路径，保证编译
   # 顺利完成。
2  # -I，指定头文件目录
3  $ tree
4  .
5  ├── add.c
6  ├── div.c

```

```

7 |— include
8 |   └─ head.h
9 |— main.c
10|— mult.c
11|— sub.c
12
13 # 编译当前目录中的所有源文件，得到可执行程序
14 $ gcc *.c -o calc
15 main.c:2:18: fatal error: head.h: No such file or directory
16 compilation terminated.
17 sub.c:2:18: fatal error: head.h: No such file or directory
18 compilation terminated.
19
20 # 通过编译得到的错误信息可以知道，源文件中包含的头文件无法被找到。通过提供的目录结构可以得知
    头文件 head.h 在 include 目录中，因此可以在编译的时候重新指定头文件位置，具体操作如下：
21
22
23 # 可以在编译的时候重新指定头文件位置 -I 头文件目录
24 $ gcc *.c -o calc -I ./include
25

```

Fence 21

◦ 指定一个宏 (-D)

```

1 # 在程序中我们可以使用宏定义一个常量，也可以通过宏控制某段代码是否能够被执行。在下面这段程序
    中第8行判断是否定义了一个叫做 DEBUG的宏，如果没有定义第9行代码就不会被执行了，通过阅读代码
    能够知道这个宏是在程序中被定义的。
2
3 // test.c
4 #include <stdio.h>
5 #define NUMBER 3
6
7 int main()
8 {
9     int a = 10;
10 #ifdef DEBUG
11     printf("我是一个程序猿，我不会爬树...\n");
12 #endif
13     for(int i=0; i<NUMBER; ++i)
14     {
15         printf("hello, GCC!!!\n");
16     }
17     return 0;
18 }
19
20
21 # 如果不想在程序中定义这个宏，但是又想让它存在，通过gcc的参数 -D就可以实现了，编译器会认为
    参数后边指定的宏在程序中是存在的。
22
23 # 在编译命令中定义这个 DEBUG 宏，
24 $ gcc test.c -o app -D DEBUG
25

```

```

26 # 执行生成的程序， 可以看到程序第9行的输出
27 $ ./app
28 我是一个程序猿， 我不会爬树...
29 hello, GCC!!!
30 hello, GCC!!!
31 hello, GCC!!!
32

```

Fence 22

-D 参数的应用场景:

在发布程序的时候, 一般都会要求将程序中所有的log输出去掉, 如果不去掉会影响程序的执行效率, 很显然删除这些打印log的源代码是一件很麻烦的事情, 解决方案是这样的:

- 将所有的打印log的代码都写到一个宏判定中, 可以模仿上边的例子
 - 在编译程序的时候指定 -D 就会有log输出
 - 在编译程序的时候不指定 -D, log就不会输出

4.4 多文件编译

- GCC 可以自动编译链接多个文件, 不管是目标文件还是源文件, 都可以使用同一个命令编译到一个可执行文件中。
- 首先将程序编译之前需要的代码准备出来, 例如一个项目包含3个文件, 文件 string.h, string.c 中有一个函数 strlen 用于计算字符串的长度, 而在 main.c 中调用这个函数将计算的结果显示出来。

```

1 // 头文件
2 #ifndef _STRING_H_
3 #define _STRING_H_
4 int strlen(char *string);
5 #endif // _STRING_H_
6
7
8 //源文件 string.c
9 #include "string.h"
10
11 int strlen(char *string)
12 {
13     int len = 0;
14     while(*string++ != '\0')    // 当*string 的值为'\0'时, 停止计算
15     {
16         len++;
17     }
18     return len;    // 返回字符串长度
19 }
20
21
22 // main.c
23 #include <stdio.h>
24 #include "string.h"
25
26 int main(void)

```



```

27 {
28     char *src = "Hello, I'am Monkey·D·Luffy!!!";
29     printf("string length is: %d\n", strlen(src));
30     return 0;
31 }

```

Fence 23

• 编译运行

因为头文件是包含在源文件中的, 因此在使用gcc编译程序的时候不需要指定头文件的名字 **(在头文件无法被找到的时候需要使用参数 -I 指定其具体路径而不是名字)**。我们可以通过一个 gcc 命令将多个源文件编译并生成可执行程序, 也可以分多步完成这个操作。

```

1  # 直接链接生成可执行程序
2  # 直接生成可执行程序 test
3  $ gcc -o test string.c main.c
4
5  # 运行可执行程序
6  $ ./test
7
8
9  # 先将源文件编成目标文件, 然后进行链接得到可执行程序
10 # 汇编生成二进制目标文件, 指定了 -c 参数之后, 源文件会自动生成 string.o 和 main.o
11 $ gcc -c string.c main.c
12
13 # 链接目标文件, 生成可执行程序 test
14 $ gcc -o test string.o main.o
15
16 # 运行可执行程序
17 $ ./test
18
19
20 # 总结 多文件就是将.c文件放在gcc后面
21 gcc *.c -o xxx

```

Fence 24

4.5 GCC与G++区别

- 关于对gcc和g++很多人的理解都是比较片面的或者是对二者的理解有一些误区, 下边从三个方面介绍一下二者的区别:
 - 在代码编译阶段 (第二个阶段) :
 - 后缀为 .c 的, gcc 把它当作是C程序, 而 g++ 当作是 C++ 程序
 - 后缀为.cpp的, 两者都会认为是 C++ 程序, C++ 的语法规则更加严谨一些
 - g++会调用gcc, 对于C++代码, 两者是等价的, 也就是说 gcc 和 g++ 都可以编译 C/C++代码
 - 在链接阶段 (最后一个阶段) :
 - gcc 和 g++ 都可以自动链接到标准C库

- g++ 可以自动链接到标准C++库, gcc如果要链接到标准C++库需要加参数 -lstdc++
- 关于 __cplusplus宏的定义
 - g++ 会自动定义__cplusplus宏, 但是这个不影响它去编译C程序
 - gcc 需要根据文件后缀判断是否需要定义 __cplusplus 宏 (规则参考第一条)
- 综上所述:
 - 不管是 gcc 还是 g++ 都可以编译 C 程序, 编译程序的规则和参数都相同
 - g++可以直接编译C++程序, **gcc 编译 C++程序需要添加额外参数 -lstdc++**
 - 不管是 gcc 还是 g++ 都可以定义 __cplusplus宏

```
1  # 编译 c 程序
2  $ gcc test.c -o test      # 使用gcc
3  $ g++ test.c -o test      # 使用g++
4
5  # 编译 c++ 程序
6  $ g++ test.cpp -o test      # 使用g++
7  $ gcc test.cpp -lstdc++ -o test  # 使用gcc
```

Fence 25

5. 动态库和静态库

- 在项目中使用库一般有两个目的, 一个是为了使程序更加简洁不需要在项目中维护太多的源文件, 另一方面是为了源代码保密, 毕竟不是所有人都想把自己编写的程序开源出来。
- 拿到了库文件(动态库、静态库)之后要想使用还必须有这些库中提供的API函数的声明, 也就是头文件, 把这些都添加到项目中

5.1 静态库 (无执行权限)

- 在Linux中静态库由程序 ar 生成, 现在静态库已经不像之前那么普遍了, 这主要是由于程序都在使用动态库。关于静态库的命名规则如下:
 - 在Linux中静态库以lib作为前缀, 以.a作为后缀, 中间是库的名字自己指定即可, 即: libxxx.a
 - 在Windows中静态库一般以lib作为前缀, 以lib作为后缀, 中间是库的名字需要自己指定, 即: libxxx.lib

5.1.1 生成静态链接库

- 生成静态库, 需要先对源文件进行汇编操作 (使用参数 -c) 得到二进制格式的目标文件 (.o 格式), 然后在通过 ar 工具将目标文件打包就可以得到静态库文件了 (libxxx.a)。

使用ar工具创建静态库的时候需要三个参数:

- 参数c: 创建一个库, 不管库是否存在, 都将创建。
- 参数s: 创建目标文件索引, 这在创建较大的库时能加快时间。
- 参数r: 在库中插入模块(替换)。默认新的成员添加在库的结尾处, 如果模块名已经在库中存在, 则替换同名的模块。

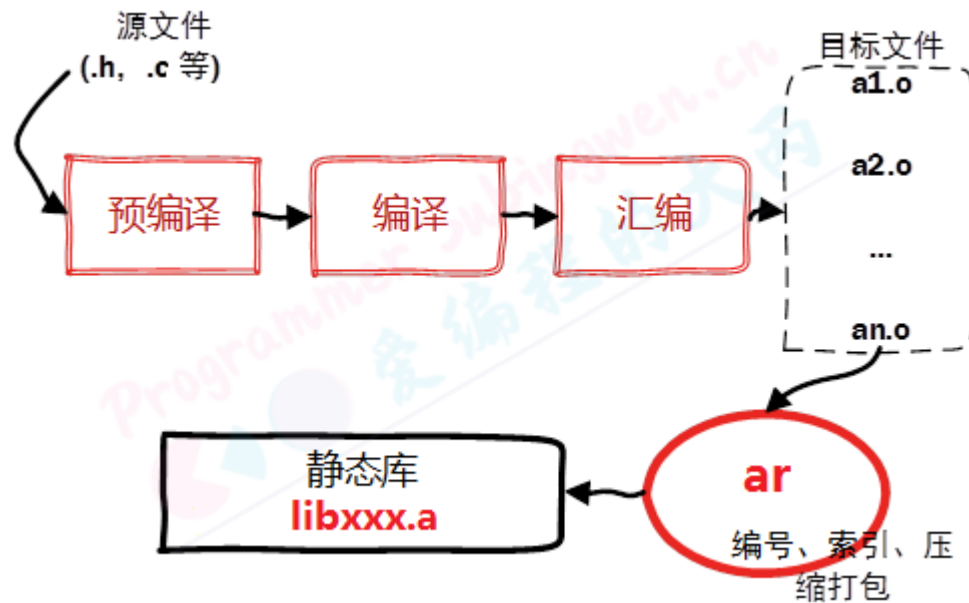


Figure 11

```

1  # 需要将源文件进行汇编，得到 .o 文件，需要使用参数 -c
2  # 执行如下操作，默认生成二进制的 .o 文件
3  # -c 参数位置没有要求
4  $ gcc 源文件(*.c) -c
5
6
7  # 将得到的 .o 进行打包，得到静态库
8  $ ar rcs 静态库的名字(libxxx.a) 原材料(*.o)
9
10
11 # 发布静态库
12 # 发布静态库
13     1. 提供头文件 *.h
14     2. 提供制作出来的静态库 libxxx.a
15

```

Fence 26

```

1  # 测试
2  # 在某个目录中有如下的源文件，用来实现一个简单的计算器：
3  # 目录结构 add.c div.c mult.c sub.c -> 算法的源文件，函数声明在头文件 head.h
4  # main.c中是对接口的测试程序，制作库的时候不需要将 main.c 算进去
5  .
6  |— add.c
7  |— div.c
8  |— include
9  |   |— head.h
10 |— main.c
11 |— mult.c
12 |— sub.c
13
14
15 # 加法计算源文件 add.c:
16 #include <stdio.h>

```

```
17  #include "head.h"
18
19  int add(int a, int b)
20  {
21      return a+b;
22  }
23
24
25  # 减法计算源文件 sub.c:
26  #include <stdio.h>
27  #include "head.h"
28
29  int subtract(int a, int b)
30  {
31      return a-b;
32  }
33
34
35  # 乘法计算源文件 mult.c:
36  #include <stdio.h>
37  #include "head.h"
38
39  int multiply(int a, int b)
40  {
41      return a*b;
42  }
43
44
45  # 减法计算的源文件 div.c
46  #include <stdio.h>
47  #include "head.h"
48
49  double divide(int a, int b)
50  {
51      return (double)a/b;
52  }
53
54
55  # 头文件 head.h
56  #ifndef _HEAD_H
57  #define _HEAD_H
58  // 加法
59  int add(int a, int b);
60  // 减法
61  int subtract(int a, int b);
62  // 乘法
63  int multiply(int a, int b);
64  // 除法
65  double divide(int a, int b);
66  #endif
67
68
69  # 测试文件main.c
```

```

70 #include <stdio.h>
71 #include "head.h"
72
73 int main()
74 {
75     int a = 20;
76     int b = 12;
77     printf("a = %d, b = %d\n", a, b);
78     printf("a + b = %d\n", add(a, b));
79     printf("a - b = %d\n", subtract(a, b));
80     printf("a * b = %d\n", multiply(a, b));
81     printf("a / b = %f\n", divide(a, b));
82     return 0;
83 }
84
85
86 # 生成静态库 第一步：将源文件add.c, div.c, mult.c, sub.c 进行汇编，得到二进制目标文件 add.o,
div.o, mult.o, sub.o
87 # 1. 生成.o
88 $ gcc add.c div.c mult.c sub.c -c
89 sub.c:2:18: fatal error: head.h: No such file or directory
90 compilation terminated.
91
92 # 提示头文件找不到，添加参数 -I 重新头文件路径即可
93 $ gcc add.c div.c mult.c sub.c -c -I ./include/
94
95 # 查看目标文件是否已经生成
96 $ tree
97 .
98 └─ add.c
99 └─ add.o          # 目标文件
100 └─ div.c
101 └─ div.o          # 目标文件
102 └─ include
103 │   └─ head.h
104 └─ main.c
105 └─ mult.c
106 └─ mult.o         # 目标文件
107 └─ sub.c
108 └─ sub.o         # 目标文件
109
110
111 # 第二步：将生成的目标文件通过 ar工具打包生成静态库
112 # 2. 将生成的目标文件 .o 打包成静态库
113 $ ar rcs libcalc.a a.o b.o c.o    # a.o b.o c.o在同一个目录中可以写成 *.o
114
115 # 查看目录中的文件
116 $ tree
117 .
118 └─ add.c
119 └─ add.o
120 └─ div.c
121 └─ div.o

```

```

122 | include
123 |   └─ `head.h` ==> 和静态库一并发布
124 |   └─ `libcalc.a` ==> 生成的静态库
125 | main.c
126 | mult.c
127 | mult.o
128 | sub.c
129 |   └─ sub.o
130
131
132 # 第三步：将生成的静态库 libcalc.a和库对应的头文件head.h一并发布给使用者就可以了。
133 # 3. 发布静态库
134     1. head.h    => 函数声明
135     2. libcalc.a => 函数定义(二进制格式)

```

Fence 27

5.1.2 静态库使用

- 当我们得到了一个可用的静态库之后, 需要将其放到一个目录中, 然后根据得到的头文件编写测试代码, 对静态库中的函数进行调用。

```

1 # 1. 首先拿到了发布的静态库
2   `head.h` 和 `libcalc.a`
3
4 # 2. 将静态库, 头文件, 测试程序放到一个目录中准备进行测试
5 .
6 |─ head.h          # 函数声明
7 |─ libcalc.a       # 函数定义 (二进制格式)
8 |─ main.c          # 函数测试

```

Fence 28

- 编译测试程序, 得到可执行文件。

```

1 # 3. 编译测试程序 main.c
2 $ gcc main.c -o app
3 /tmp/ccR7Fk49.o: In function `main':
4 main.c:(.text+0x38): undefined reference to `add'
5 main.c:(.text+0x58): undefined reference to `subtract'
6 main.c:(.text+0x78): undefined reference to `multiply'
7 main.c:(.text+0x98): undefined reference to `divide'
8 collect2: error: ld returned 1 exit status

```

Fence 29

- 上述错误分析:
 - 编译的源文件中包含了头文件 head.h, 这个头文件中声明的函数对应的定义 (也就是函数体实现) 在静态库中, 程序在编译的时候没有找到函数实现, 因此提示 undefined reference to xxxx。
 - 解决方案: 在编译的时将静态库的路径和名字都指定出来

- -L: 指定库所在的目录(相对或者绝对路径)
- -l: 指定库的名字, 需要掐头(lib)去尾(.a) 剩下的才是需要的静态库的名字

```

1  # 4. 编译的时候指定库信息
2      -L: 指定库所在的目录(相对或者绝对路径)
3      -l: 指定库的名字, 掐头(lib)去尾(.a) ==> calc
4  # -L -l, 参数和参数值之间可以有空格, 也可以没有 -L./ -lcalc
5  $ gcc main.c -o app -L ./ -l calc
6
7  # 查看目录信息, 发现可执行程序已经生成了
8  $ tree
9  .
10 |— app          # 生成的可执行程序
11 |— head.h
12 |— libcalc.a
13 |— main.c

```

Fence 30

5.2 动态库 (有执行权限)

- 动态链接库是程序运行时加载的库, 当动态链接库正确部署之后, 运行的多个程序可以使用同一个加载到内存中的动态库, 因此在Linux中动态链接库也可称之为共享库。

动态链接库是目标文件的集合, 目标文件在动态链接库中的组织方式是按照特殊方式形成的。库中函数和变量的地址使用的是相对地址 (静态库中使用的是绝对地址), 其真实地址是在应用程序加载动态库时形成的。

关于动态库的命名规则如下:

- 在Linux中动态库以lib作为前缀, 以.so作为后缀, 中间是库的名字自己指定即可, 即: **libxxx.so**
- 在Windows中动态库一般以lib作为前缀, 以dll作为后缀, 中间是库的名字需要自己指定, 即: **libxxx.dll**
 - 制作工具不一样, 生成的也不一样, 如: 使用vs制作, 库文件有两个.lib与.dll, 两个一起使用才是完整动态库

5.2.1 生成动态链接库

- 生成动态链接库是**直接使用gcc命令并且需要添加-fPIC (-fpic) 以及-shared 参数**。
 - -fPIC 或 -fpic 参数的作用是使得 gcc 生成的代码是与位置无关的, 也就是使用相对位置。
 - -shared参数的作用是告诉编译器生成一个动态链接库。

生成的.o为是 与位置无关的代码 (进程使用时, 将代码放入动态库加载区, 使用的相对地址)。静态库生成的.o是放在代码区的, 且使用的是绝对地址

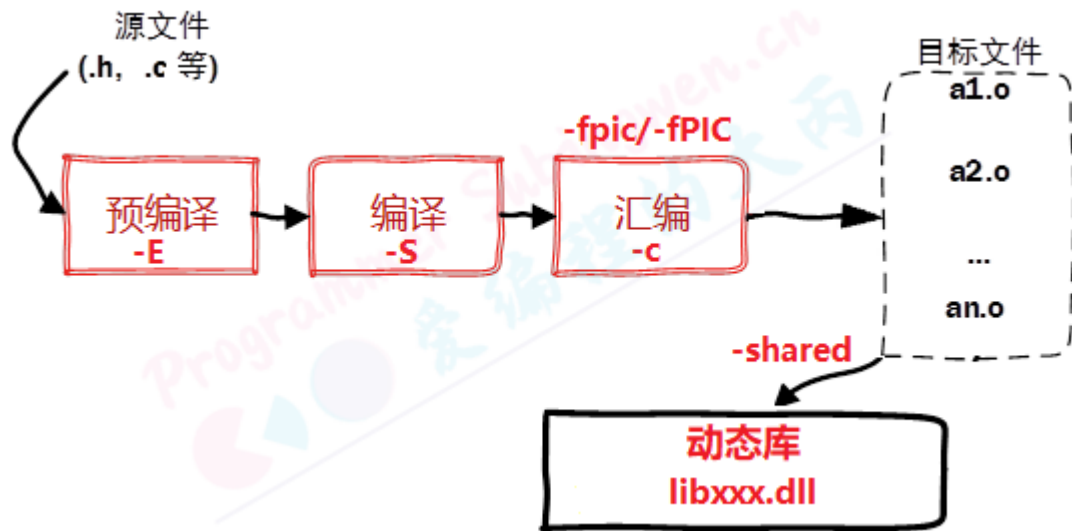


Figure 12

- 生成动态链接库的具体步骤如下:

- 将源文件进行汇编操作, 需要使用参数 `-c`, 还需要添加额外参数 `-fpic / -fPIC`

```

1 | # 得到若干个 .o 文件
2 | $ gcc 源文件(*.c) -c -fpic

```

Fence 31

- 将得到的.o文件打包成动态库, 还是使用gcc, 使用参数 `-shared` 指定生成动态库(位置没有要求)

```

1 | $ gcc -shared 与位置无关的目标文件(*.o) -o 动态库(libxxx.so)

```

Fence 32

- 发布动态库和头文件

```

1 | # 发布
2 |     1. 提供头文件: xxx.h
3 |     2. 提供动态库: libxxx.so

```

Fence 33

```

1 | # 例子
2 | # 在此还是以上面制作静态库使用的实例代码为例来制作动态库
3 | # 举例, 示例目录如下:
4 | # 目录结构 add.c div.c mult.c sub.c -> 算法的源文件, 函数声明在头文件 head.h
5 | # main.c中是对接口的测试程序, 制作库的时候不需要将 main.c 算进去
6 | .
7 | ├── add.c
8 | ├── div.c
9 | ├── include
10 | |   └── head.h
11 | ├── main.c
12 | ├── mult.c
13 | └── sub.c

```



```

14
15
16 # 第一步：使用gcc将源文件进行汇编(参数-c)，生成与位置无关的目标文件，需要使用参数 -fpic或者-fPIC
17 # 1. 将.c汇编得到.o，需要额外的参数 -fpic/-fPIC
18 $ gcc add.c div.c mult.c sub.c -c -fpic -I ./include/
19
20 # 查看目录文件信息，检查是否生成了目标文件
21 $ tree
22 .
23 └─ add.c
24 └─ add.o # 生成的目标文件
25 └─ div.c
26 └─ div.o # 生成的目标文件
27 └─ include
28 │   └─ head.h
29 └─ main.c
30 └─ mult.c
31 └─ mult.o # 生成的目标文件
32 └─ sub.c
33 └─ sub.o # 生成的目标文件
34
35
36 # 第二步：使用gcc将得到的目标文件打包生成动态库，需要使用参数 -shared
37 # 2. 将得到 .o 打包成动态库，使用gcc，参数 -shared
38 $ gcc -shared add.o div.o mult.o sub.o -o libcalc.so
39
40 # 检查目录中是否生成了动态库
41 $ tree
42 .
43 └─ add.c
44 └─ add.o
45 └─ div.c
46 └─ div.o
47 └─ include
48 │   └─ `head.h ==> 和动态库一起发布
49 └─ `libcalc.so ==> 生成的动态库
50 └─ main.c
51 └─ mult.c
52 └─ mult.o
53 └─ sub.c
54 └─ sub.o
55
56
57
58 # 第三步：发布生成的动态库和相关的头文件
59 # 3. 发布库文件和头文件
60     1. head.h
61     2. libcalc.so

```

5.2.2 动态库的使用

- 当我们得到了一个可用的动态库之后, 需要将其放到一个目录中, 然后根据得到的头文件编写测试代码, 对动态库中的函数进行调用。

```
1  # 1. 拿到发布的动态库
2      `head.h`  libcalc.so
3  # 2. 基于头文件编写测试程序, 测试动态库中提供的接口是否可用
4      `main.c`
5  # 示例目录:
6  .
7  └─ head.h          ==> 函数声明
8  └─ libcalc.so      ==> 函数定义
9  └─ main.c          ==> 函数测试
```

Fence 35

- 编译测试程序

```
1  # 3. 编译测试程序
2  $ gcc main.c -o app
3  /tmp/ccw1upvy.o: In function `main':
4  main.c:(.text+0x38): undefined reference to `add'
5  main.c:(.text+0x58): undefined reference to `subtract'
6  main.c:(.text+0x78): undefined reference to `multiply'
7  main.c:(.text+0x98): undefined reference to `divide'
8  collect2: error: ld returned 1 exit status
```

Fence 36

- 错误原因:

和使用静态库一样, 在编译的时候需要指定库相关的信息: 库的路径 -L和 库的名字 -l

- 添加库信息相关参数, 重新编译测试代码:

```
1  # 在编译的时候指定动态库相关的信息: 库的路径 -L, 库的名字 -l
2  $ gcc main.c -o app -L./ -lcalc
3
4  # 查看是否生成了可执行程序
5  $ tree
6  .
7  └─ app          # 生成的可执行程序
8  └─ head.h
9  └─ libcalc.so
10 └─ main.c
11
12 # 执行生成的可执行程序, 错误提示 ==> 可执行程序执行的时候找不到动态库
13 $ ./app
14 ./app: error while loading shared libraries: libcalc.so: cannot open shared
    object file: No such file or directory
```

Fence 37

- 关于整个操作过程的报告：

gcc通过指定的动态库信息生成了可执行程序, 但是可执行程序运行却提示无法加载到动态库。

5.2.3 解决动态库无法加载问题

5.2.3.1 库的工作原理

- 静态库如何被加载
 - 在程序编译的最后一个阶段也就是链接阶段, 提供的静态库会被打包到可执行程序中。当可执行程序被执行, 静态库中的代码也会一并被加载到内存中, 因此不会出现静态库找不到无法被加载的问题。
- 动态库如何被加载
 - 在程序编译的最后一个阶段也就是链接阶段:
 - 在gcc命令中虽然指定了库路径(使用参数 -L), 但是这个路径并没有记录到可执行程序中, 只是检查了这个路径下的库文件是否存在。
 - 同样对应的**动态库文件也没有被打包到可执行程序中, 只是在可执行程序中记录了库的名字。**
 - 可执行程序被执行起来之后:
 - 程序执行的时候会先检测需要的动态库是否可以被加载, 加载不到就会提示上边的错误信息
 - 当动态库中的函数在程序中被调用了, 这个时候动态库才加载到内存, 如果不被调用就不加载 (随用随加载)
 - 动态库的检测和内存加载操作都是由动态连接器来完成的

5.2.3.2 动态连接器

- 动态连接器是一个独立于应用程序的进程, 属于操作系统, 当用户的程序需要加载动态库的时候动态连接器就开始工作了, 很显然动态连接器根本就不知道用户通过 gcc 编译程序的时候通过参数 -L指定的路径。

那么动态连接器是如何搜索某一个动态库的呢, 在它内部有一个默认的搜索顺序, 按照优先级从高到低的顺序分别是:

- 可执行文件内部的 DT_RPATH 段
- 系统的环境变量 LD_LIBRARY_PATH (默认为空)
- 系统动态库的缓存文件 /etc/ld.so.cache
- 存储动态库/静态库的系统目录 /lib/, /usr/lib等
- 按照以上四个顺序, 依次搜索, 找到之后结束遍历, 最终还是没找到, 动态连接器就会提示动态库找不到的错误信息。

5.2.3.3 解决方案

- 可执行程序生成之后, 根据动态链接器的搜索路径, 我们可以提供三种解决方案, 我们只需要将动态库的路径放到对应的环境变量或者系统配置文件中, 同样也可以将动态库拷贝到系统库目录 (或者将动态库的软链接文件放到这些系统库目录中)。

1. 方案1: 将库路径添加到环境变量 LD_LIBRARY_PATH 中

找到相关的配置文件

- 用户级别: ~/.bashrc → 设置对当前用户有效
- 系统级别: /etc/profile → 设置对所有用户有效

使用 vim 打开配置文件, 在文件最后添加这样一句话:

- 1 # 自己把路径写进去就行了
- 2 export LD_LIBRARY_PATH =LD_LIBRARY_PATH :动态库的绝对路径

Fence 38

让修改的配置文件生效

- 修改了用户级别的配置文件, 关闭当前终端, 打开一个新的终端配置就生效了
- 修改了系统级别的配置文件, 注销或关闭系统, 再开机配置就生效了
- 不想执行上边的操作, 可以执行一个命令让配置重新被加载

```
1 # 修改的是哪一个就执行对应的那个命令
2 # source 可以简写为一个 . , 作用是让文件内容被重新加载
3 $ source ~/.bashrc          (. ~/.bashrc)
4 $ source /etc/profile       (. /etc/profile)
```

2. 方案2: 更新 /etc/ld.so.cache 文件

Fence 39

找到动态库所在的绝对路径 (不包括库的名字) 比如: /home/robin/Library/

使用vim 修改 /etc/ld.so.conf 这个文件, 将上边的路径添加到文件中(独自占一行)

```
1 # 1. 打开文件
2 $ sudo vim /etc/ld.so.conf
3
4 # 2. 添加动态库路径, 并保存退出
5 # 如: /aaa/aa/a
```

Fence 40

更新 /etc/ld.so.conf中的数据到 /etc/ld.so.cache 中

```
1 # 必须使用管理员权限执行这个命令
2 $ sudo ldconfig
```

Fence 41

3. 方案3: 拷贝动态库文件到系统库目录 /lib/ 或者 /usr/lib 中 (或者将库的软链接文件放进去)

```
1 # 第一种 库拷贝
2 sudo cp /xxx/xxx/libxxx.so /usr/lib
3
4 # 第二种 创建软连接 (对于后续的更新维护, 更方便)
5 sudo ln -s /xxx/xxx/libxxx.so /usr/lib/libxxx.so
```

Fence 42

5.2.3.4 验证

- 在启动可执行程序之前, 或者在设置了动态库路径之后, 我们可以通过一个命令检测程序能不能通过动态链接器加载到对应的动态库, 这个命令叫做 ldd

```

1  # 语法:
2  $ ldd 可执行程序名
3
4  # 举例:
5  $ ldd app
6      linux-vdso.so.1 => (0x00007ffe8fbd6000)
7      libc.so.6 => /home/robin/Linux/3Day/calc/test/libc.so.6 (0x00007f5d85dd4000)
8      /lib/x86_64-linux-gnu/libc.so.6 (0x00007f5d85a0a000)
9      /lib64/ld-linux-x86-64.so.2 (0x00007f5d85fd6000) ==> 动态链接器, 操作系统提供
10 # 如果每个都能加载到, 后面都有一个地址; 如果不能加载, 一般为not found

```

Fence 43

5.3 动态库与静态库优缺点

- 静态库
 - 优点:
 - 静态库被打包到应用程序中加载速度快
 - 发布程序无需提供静态库, 移植方便
 - 缺点:
 - 相同的库文件数据可能在内存中被加载多份, 消耗系统资源, 浪费内存
 - 库文件更新需要重新编译项目文件, 生成新的可执行程序, 浪费时间。

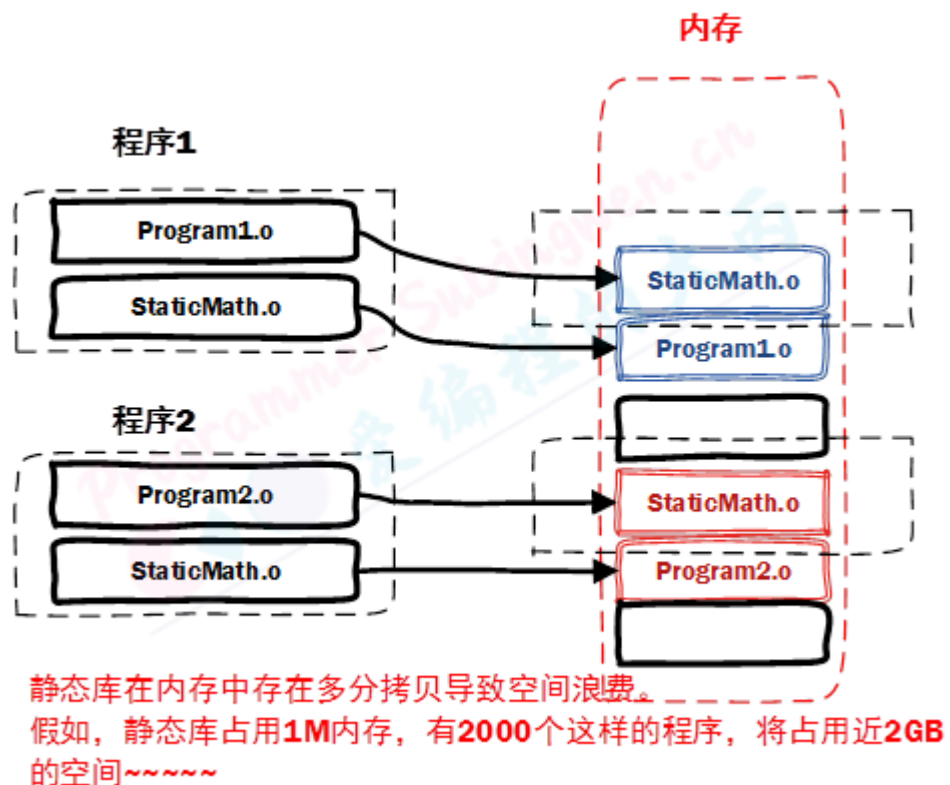


Figure 13

- 动态库

- 优点：
 - 可实现不同进程间的资源共享
 - 动态库升级简单, 只需要替换库文件, 无需重新编译应用程序
 - 程序猿可以控制何时加载动态库, 不调用库函数动态库不会被加载
- 缺点：
 - 加载速度比静态库慢, 以现在计算机的性能可以忽略
 - 发布程序需要提供依赖的动态库

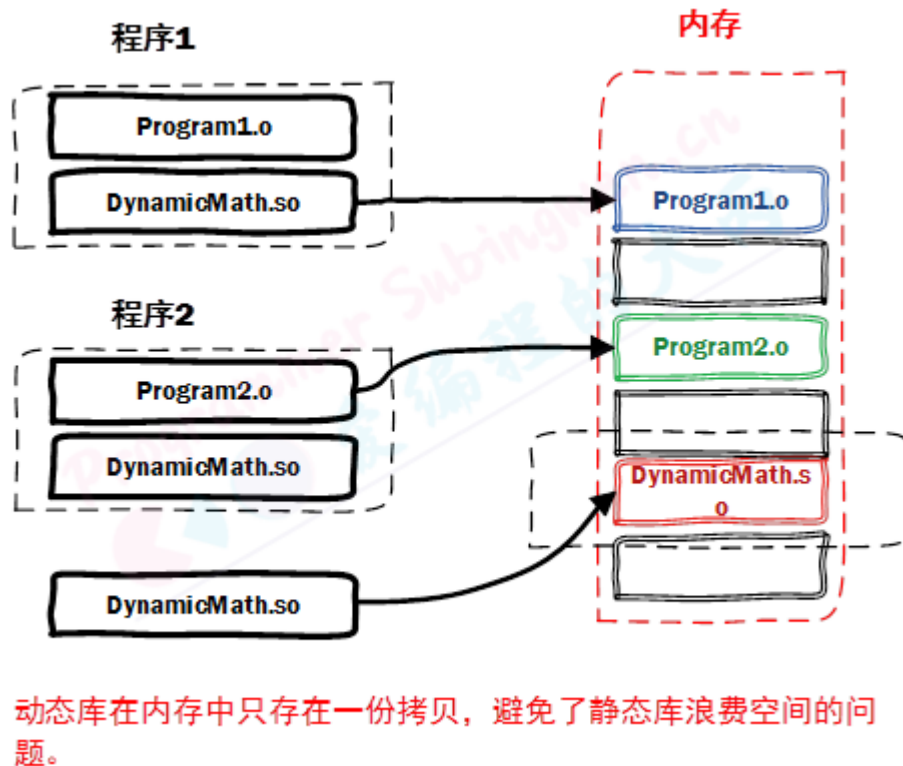


Figure 14

6. makefile (linux自带的编译工具)

- 使用 GCC 的命令行进行程序编译在单个文件下是比较方便的，当工程中的文件逐渐增多，甚至变得十分庞大的时候，使用 GCC 命令编译就会变得力不从心。这种情况下我们需要借助项目构造工具 make 帮助我们完成这个艰巨的任务。**make是一个命令工具，是一个解释makefile中指令的命令工具**，一般来说，大多数的IDE都有这个命令，比如：Visual C++的nmake，QtCreator的qmake等。
- make工具在构造项目的时候需要加载一个叫做makefile的文件，makefile关系到了整个工程的编译规则。一个工程中的源文件不计数，其按类型、功能、模块分别放在若干个目录中，makefile定义了一系列的规则来指定哪些文件需要先编译，哪些文件需要后编译，哪些文件需要重新编译，甚至于进行更复杂的功能操作，因为makefile就像一个Shell脚本一样，其中也可以执行操作系统的命令。（make为厨师，makefile为食谱）
- **makefile带来的好处就是——“自动化编译”，一旦写好，只需要一个make命令，整个工程完全自动编译，极大的提高了软件开发的效率。**
- makefile文件有两种命名方式 makefile 和 Makefile，构建项目的时候在哪个目录下执行构建命令 make这个目录下的 makefile 文件就会加载，因此在一个项目中可以有多个 makefile 文件，分别位于不同的项目目录中。

6.1 规则

- Makefile的框架是由规则构成的。make命令执行时先在Makefile文件中查找各种规则，对各种规则进行解析后运行规则。规则的基本格式为：

```
1  # 每条规则的语法格式：
2  target1,target2...: depend1, depend2, ...
3      command
4      .....
5      .....
```

Fence 44

- 每条规则由三个部分组成分别是目标(target), 依赖(depend)和命令(command)。（一条规则，可以有多个目标、依赖、命令）
 - 命令(command): 当前这条规则的动作，一般情况下这个动作就是一个 shell 命令
 - 例如：通过某个命令编译文件、生成库文件、进入目录等。
 - 动作可以是多个，**每个命令前必须有一个Tab缩进并且独占占一行。**
 - 依赖(depend): 规则所必需的依赖条件，在规则的命令中可以使用这些依赖。
 - 例如：生成可执行文件的**目标文件 (*.o)** 可以作为依赖使用
 - 如果规则的命令中不需要任何依赖，那么规则的依赖可以为空
 - 当前规则中的依赖可以是其他规则中的某个目标，这样就形成了规则之间的嵌套
 - 依赖可以根据要执行的命令的实际需求, 指定很多个
 - 目标(target): 规则中的目标，这个目标和规则中的命令是对应的
 - 通过执行规则中的命令，可以生成一个和目标同名的文件
 - 规则中可以有多个命令, 因此可以通过这多条命令来生成多个目标, 所有目标也可以有很多个
 - 通过执行规则中的命令，可以只执行一个动作，不生成任何文件，这样的目标被称为**伪目标**

```
1  # 举例：有源文件 a.c b.c c.c head.h，需要生成可执行程序 app
2  ##### 例1 #####
3  app:a.c b.c c.c
4      gcc a.c b.c c.c -o app
5
6  ##### 例2 #####
7  # 有多个目标，多个依赖，多个命令
8  app,app1:a.c b.c c.c d.c
9      gcc a.c b.c -o app
10     gcc c.c d.c -o app1
11
12 ##### 例3 #####
13 # 规则之间的嵌套
14 # a.o b.o c.o默认不存在，通过下面的规则实现
15 app:a.o b.o c.o
16     gcc a.o b.o c.o -o app
17 # a.o 是第一条规则中的依赖
18 a.o:a.c
19     gcc -c a.c
20 # b.o 是第一条规则中的依赖
```

```

21 b.o:b.c
22     gcc -c b.c
23 # c.o 是第一条规则中的依赖
24 c.o:c.c
25     gcc -c c.c
26 # 一般来说，后面的规则为第一条规则服务

```

Fence 45

6.2 工作原理

6.2.1 规则的执行

- 在调用 make 命令编译程序的时候，make 会首先找到 Makefile 文件中的第 1 个规则，分析并执行相关的动作。但是需要注意的是，好多时候要执行的动作（命令）中使用的依赖是不存在的，如果使用的依赖不存在，这个动作也就不会被执行。
- 对应的解决方案是先将需要的依赖生成出来，我们就可以在makefile中添加新的规则，将不存在的依赖作为这个新的规则中的目标，当这条新的规则对应的命令执行完毕，对应的目标就被生成了，同时另一条规则中需要的依赖也就存在了。
- 这样，makefile中的某一条规则在需要的时候，就会被其他的规则调用，直到makefile中的第一条规则中的所有依赖全部被生成，第一条规则中的命令就可以基于这些依赖生成对应的目标，make 的任务也就完成了。

```

1 # makefile
2 # 规则之间的嵌套
3 # 规则1
4 app:a.o b.o c.o
5     gcc a.o b.o c.o -o app
6 # 规则2
7 a.o:a.c
8     gcc -c a.c
9 # 规则3
10 b.o:b.c
11     gcc -c b.c
12 # 规则4
13 c.o:c.c
14     gcc -c c.c

```

Fence 46

在这个例子中，如果执行 make 命令就会根据这个 makefile 中的4条规则编译这三个源文件。在解析第一条规则的时候发现里边的三个依赖都是不存在的，因此规则对应的命令也就不能被执行。

当依赖不存在的时候，make就是查找其他的规则，看哪一条规则是用来生成需要的这个依赖的，找到之后就会执行这条规则中的命令。因此规则2，规则3，规则4里的命令会相继被执行，当规则1中依赖全部被生成之后对应的命令也就被执行了，因此规则1的目标被生成，make工作结束。

- 如果想要执行 makefile 中**非第一条规则对应的命令**，那么就不能直接 make，需要**将那条规则的目标也写到 make 的后边**，比如只需要执行规则3中的命令，就需要: make b.o。

6.2.2 文件的时间戳

- make 命令执行的时候会根据文件的时间戳判定是否执行makefile文件中相关规则中的命令。
 - 执行完成：
 - 目标是通过依赖生成的，因此**正常情况下：目标时间戳 > 所有依赖的时间戳**，如果执行 make 命令的时候检测到规则中的目标和依赖满足这个条件，那么规则中的命令就不会被执行。
 - 更新：
 - 当依赖文件被更新了，文件时间戳也会随之被更新，这时候 **目标时间戳 < 某些依赖的时间戳**，在这种情况下目标文件会通过规则中的命令被重新生成。
 - 如果规则中的**目标对应的文件根本就不存在**，那么规则中的命令肯定会被执行。

```
1  # makefile
2  # 规则之间的嵌套
3  # 规则1
4  app:a.o b.o c.o
5      gcc a.o b.o c.o -o app
6  # 规则2
7  a.o:a.c
8      gcc -c a.c
9  # 规则3
10 b.o:b.c
11     gcc -c b.c
12 # 规则4
13 c.o:c.c
14     gcc -c c.c
15
16 # 根据上文的描述，先执行 make 命令，基于这个 makefile 编译这几个源文件生成对应的目标文件。然后再修改例子中的 a.c，再次通过make编译这几个源文件，那么这个时候先执行规则2更新目标文件a.o，然后再执行规则1更新目标文件app，其余的规则是不会被执行的。
```

Fence 47

6.2.3 自动推导

- make 是一个功能强大的构建工具，虽然make需要根据 makefile 中指定的规则来完成源文件的编译。作为小白的我们编写makefile的时候难免写的不是那么严谨从而漏写一些构建规则，但是我们会发现程序还是会被编译成功。这是因为 make 有自动推导的能力，不会完全依赖 makefile。
- 比如: 使用命令 make 编译扩展名为.c 的 C 语言文件的时候，源文件的编译规则不用明确给出。这是因为 make 进行编译的时候会使用一个默认的编译规则，按照默认规则完成对.c文件的编译，生成对应的.o 文件。它使用命令 `cc -c` 来编译.c 源文件。在 Makefile 中只要给出需要构建的目标文件名（一个.o 文件），make 会自动为这个.o 文件寻找合适的依赖文件（对应的.c 文件），并且使用默认的命令来构建这个目标文件。

```
1  # 假设本地项目目录中有以下几个源文件：
2  $ tree
3  .
4  ├── add.c
5  ├── div.c
6  └── head.h
```

```

7 | └─ main.c
8 | └─ makefile
9 | └─ mult.c
10| └─ sub.c
11|
12| # 目录中 makefile 文件内容如下
13| # 这是一个完整的 makefile 文件
14| calc:add.o div.o main.o mult.o sub.o
15|      gcc add.o div.o main.o mult.o sub.o -o calc
16|
17|
18| # 通过make构建项目：
19| $ make
20| cc -c -o add.o add.c
21| cc -c -o div.o div.c
22| cc -c -o main.o main.c
23| cc -c -o mult.o mult.c
24| cc -c -o sub.o sub.c
25| gcc add.o div.o main.o mult.o sub.o -o calc
26|
27|
28| # 我们可以发现上边的 makefile 文件中只有一条规则，依赖中所有的 .o文件在本地项目目录中是不存在的，并且也没有其他的规则用来生成这些依赖文件，这时候 make 会使用内部默认的构造规则先将这些依赖文件生成出来，然后在执行规则中的命令，最后生成目标文件 calc。
29|
30|

```

Fence 48

- makefile文件的命名：文件名全小写（makefile） / 文件名首字母大写（Makefile）

6.3 变量

- 使用 Makefile 进行规则定义的时候，为了写起来更加灵活，我们可以在里边使用变量。makefile中的变量分为三种：**自定义变量**，**预定义变量**和**自动变量**。

6.3.1 自定义变量

- 用 Makefile 进行规则定义的时候，用户可以定义自己的变量，称为用户自定义变量。**makefile 中的变量是没有类型的**，直接创建变量然后给其赋值就可以了。

```

1 | # 错误，只创建了变量名，没有赋值
2 | 变量名
3 | # 正确，创建一个变量名并且给其赋值
4 | 变量名=变量值

```

Fence 49

- 在给makefile中的变量赋值之后, 如何在需要的时候将变量值取出来呢?

```

1  # 如果将变量的值取出?
2  $(变量的名字)
3
4  # 举例 add.o div.o main.o mult.o sub.o
5  # 定义变量并赋值
6  obj=add.o div.o main.o mult.o sub.o
7  # 取变量的值
8  $(obj)

```

Fence 50

- 自定义变量使用举例：

```

1  # 这是一个规则，普通写法
2  calc:add.o div.o main.o mult.o sub.o
3      gcc add.o div.o main.o mult.o sub.o -o calc
4
5  # 这是一个规则，里边使用了自定义变量
6  obj=add.o div.o main.o mult.o sub.o
7  target=calc
8  $(target):$(obj)
9      gcc $(obj) -o $(target)

```

Fence 51

6.3.2 预定义变量

- 在 Makefile 中有一些已经定义的变量，用户可以直接使用这些变量，不用进行定义。在进行编译的时候，某些条件下 Makefile 会使用这些预定义变量的值进行编译。这些预定义变量的名字一般都是大写的，经常采用的预定义变量如下表所示：

变量名	含义	默认值
AR	生成静态库文件的程序名称	ar
AS	汇编编译器的名称	as
CC	C 语言编译器的名称	cc
CPP	C 语言预编译器的名称	\$(CC) -E
CXX	C++语言编译器的名称	g++
FC	FORTTRAN 语言编译器的名称	f77
RM	删除文件程序的名称	rm -f
ARFLAGS	生成静态库文件程序的选项	无默认值
ASFLAGS	汇编语言编译器的编译选项	无默认值
CFLAGS	C 语言编译器的编译选项	无默认值
CPPFLAGS	C 语言预编译的编译选项	无默认值
CXXFLAGS	C++语言编译器的编译选项	无默认值

变量名	含义	默认值
FFLAGS	FORTRAN 语言编译器的编译选项	无默认值

Table 19

```
1  # 这是一个规则，普通写法
2  calc:add.o div.o main.o mult.o sub.o
3      gcc add.o div.o main.o mult.o sub.o -o calc
4
5  # 这是一个规则，里边使用了自定义变量和预定义变量
6  obj=add.o div.o main.o mult.o sub.o
7  target=calc
8  CFLAGS=-O3 # 代码优化
9  $(target):$(obj)
10     $(CC) $(obj) -o $(target) $(CFLAGS)
```

Fence 52

6.3.3 自动变量

- Makefile 中的变量除了用户自定义变量和预定义变量外，还有一类自动变量。Makefile 中的规则语句中经常会出现目标文件和依赖文件，**自动变量用来代表这些规则中的目标文件和依赖文件，并且它们只能在规则的命令中使用**。（从第二行以下的命令才能使用自动变量）

下表中是一些常见的自动变量。

变量	含义
\$\$	表示目标文件的名称，不包含目标文件的扩展名（忽略后缀名）
\$\$+	表示所有的依赖文件，这些依赖文件之间以空格分开，按照出现的先后为顺序，其中可能包含重复的依赖文件
\$\$<	表示依赖项中第一个依赖文件的名称（某个规则的依赖项的第一个依赖文件）
\$\$?	依赖项中，所有比目标文件时间戳晚的依赖文件，依赖文件之间以空格分开
\$\$@	表示目标文件的名称，包含文件扩展名
\$\$^	依赖项中，所有不重复的依赖文件，这些文件之间以空格分开

Table 20

```
1  # 这是一个规则，普通写法
2  calc:add.o div.o main.o mult.o sub.o
3      gcc add.o div.o main.o mult.o sub.o -o calc
4
5  # 这是一个规则，里边使用了自定义变量
6  # 使用自动变量，替换相关的内容
7  calc:add.o div.o main.o mult.o sub.o
8      gcc $$^ -o $$@ # 自动变量只能在规则的命令中使用
9
```

6.4 模式匹配

```

1  calc:add.o div.o main.o mult.o sub.o
2      gcc add.o div.o main.o mult.o sub.o -o calc
3  # 语法格式重复的规则，将 .c -> .o，使用的命令都是一样的 gcc *.c -c
4  add.o:add.c
5      gcc add.c -c
6
7  div.o:div.c
8      gcc div.c -c
9
10 main.o:main.c
11     gcc main.c -c
12
13 sub.o:sub.c
14     gcc sub.c -c
15
16 mult.o:mult.c
17     gcc mult.c -c

```

Fence 54

- 在阅读过程中能够发现从第二个规则开始到第六个规则做的是相同的事情,但是由于文件名不同不得不在文件中写出多个规则,这就让 makefile 文件看起来非常的冗余,我们可以将这一系列的相同操作整理成一个模板,所有类似的操作都通过模板去匹配 makefile 会因此而精简不少,只是可读性会有所下降。
- 这个规则模板可以写成下边的样子,这种操作就称之为模式匹配。

```

1  # 模式匹配 -> 通过一个公式，代表若干个满足条件的规则
2  # 依赖有一个，后缀为.c，生成的目标是一个 .o 的文件，% 是一个通配符，匹配的是文件名
3  %.o:%.c
4      gcc $< -c

```

Fence 55

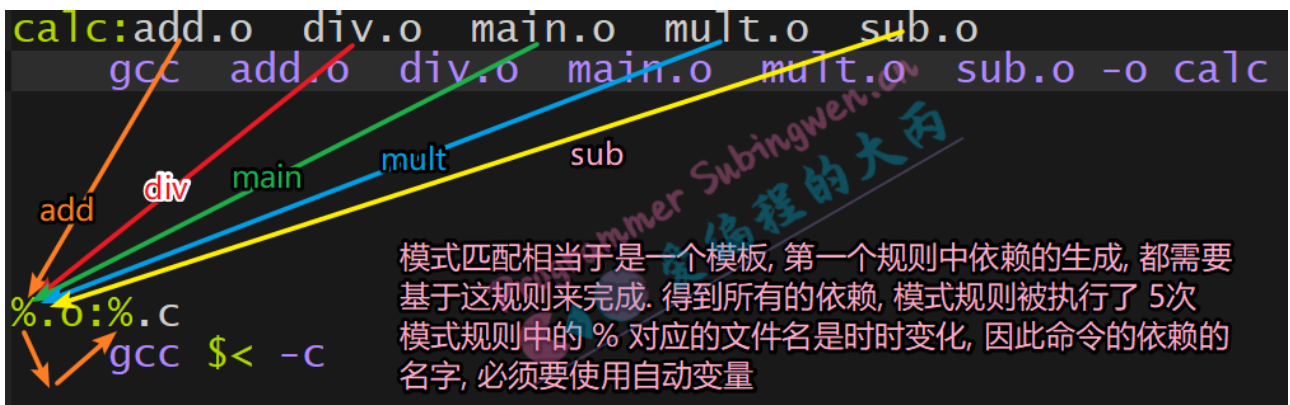


Figure 15

6.5 函数

- makefile中有很多函数并且**所有的函数都是有返回值的**。makefile中函数的格式和C/C++中函数也不同，其写法是这样的：**`$(函数名 参数1, 参数2, 参数3, ...)`**，主要目的是让我们能够快速方便的得到函数的返回值。

这里为大家介绍两个 makefile 中使用频率比较高的函数：wildcard和patsubst。

6.5.1 wildcard

- 这个函数的主要作用是获取指定目录下指定类型的文件名，其返回值是以空格分割的、指定目录下的所有符合条件的文件名列表。函数原型如下：

```
1  # 该函数的参数只有一个，但是这个参数可以分成若干个部分，通过空格间隔
2  $(wildcard PATTERN...)
3  参数：指定某个目录，搜索这个路径下指定类型的文件，比如： *.c
```

Fence 56

◦ 参数功能:

- PATTERN 指的是某个或多个目录下的对应的某种类型的文件, 比如当前目录下的.c文件可以写成 *.c
- 可以指定多个目录，每个路径之间使用空格间隔

◦ 返回值:

- 得到的若干个文件的文件列表，文件名之间使用空格间隔
- 示例：`$(wildcard .c ./sub/.c)`
 - 返回值格式: a.c b.c c.c d.c e.c f.c ./sub/aa.c ./sub/bb.c

```
1  # 使用举例：分别搜索三个不同目录下的 .c 格式的源文件
2  src = $(wildcard /home/robin/a/*.c /home/robin/b/*.c *.c) # *.c == ./*.c
3  # 返回值：得到一个大的字符串，里边有若干个满足条件的文件名，文件名之间使用空格间隔
4  /home/robin/a/a.c /home/robin/a/b.c /home/robin/b/c.c /home/robin/b/d.c e.c f.c
5
```

Fence 57

6.5.2 patsubst

- 这个函数的功能是按照指定的模式替换指定的文件名的后缀, 函数原型如下:

```
1  # 有三个参数，参数之间使用 逗号间隔
2  $(patsubst <pattern>,<replacement>,<text>)
```

Fence 58

◦ 参数功能:

- pattern: 这是一个模式字符串, 需要指定出要被替换的文件名中的后缀是什么
 - 文件名和路径不需要关心, 因此使用 % 表示即可 [通配符是 %]
 - 在通配符后边指定出要被替换的后缀, 比如: %.c, 意味着 .c的后缀要被替换掉
- replacement: 这是一个模式字符串, 指定参数pattern中的后缀最终要被替换为什么
 - 还是使用 % 来表示参数pattern 中文件的路径和名字
 - 在通配符 % 后边指定出新的后缀名, 比如: %.o 这表示原来的后缀被替换为 .o

- text: 该参数中存储这要被替换的原始数据
- 返回值:
 - 函数返回被替换过后的字符串。

```
1  src = a.cpp b.cpp c.cpp e.cpp
2  # 把变量 src 中的所有文件名的后缀从 .cpp 替换为 .o
3  obj = $(patsubst %.cpp, %.o, $(src))
4  # obj 的值为: a.o b.o c.o e.o
```

Fence 59

6.6 makefile编写

- 下面基于一个简单的项目, 为大家演示一下编写一个makefile从不标准到标准的进化过程。

```
1  # 项目目录结构
2  .
3  ├── add.c
4  ├── div.c
5  ├── head.h
6  ├── main.c
7  ├── mult.c
8  └── sub.c
9  # 需要编写makefile对该项目进行自动化编译
```

Fence 60

```
1  # 版本1
2
3  calc:add.c div.c main.c mult.c sub.c
4      gcc add.c div.c main.c mult.c sub.c -o calc
5
6  # 这个版本的优点: 书写简单
7  # 这版本的缺点: 只要依赖中的某一个源文件被修改, 所有的源文件都需要被重新编译, 太耗时、效率低
8  # 改进方式: 提高效率, 修改哪一个源文件, 哪个源文件被重新编译, 不修改就不重新编译
9
10
11
12 # 版本2
13
14 # 默认所有的依赖都不存在, 需要使用其他规则生成这些依赖
15 # 因为 add.o 被更新, 需要使用最新的依赖, 生成最新的目标
16 calc:add.o div.o main.o mult.o sub.o
17     gcc add.o div.o main.o mult.o sub.o -o calc
18
19 # 如果修改了add.c, add.o 被重新生成
20 add.o:add.c
21     gcc add.c -c
22
23 div.o:div.c
24     gcc div.c -c
25
```

```

26 main.o:main.c
27     gcc main.c -c
28
29 sub.o:sub.c
30     gcc sub.c -c
31
32 mult.o:mult.c
33     gcc mult.c -c
34
35 # 这个版本的优点：相较于版本1效率提升了
36 # 这个版本的缺点：规则比较冗余，需要精简
37 # 改进方式：在 makefile 中使用变量 和 模式匹配
38
39
40
41 # 版本3
42
43 # 添加自定义变量 -> makefile中注释前 使用 #
44 obj=add.o div.o main.o mult.o sub.o
45 target=calc
46
47 $(target):$(obj)
48     gcc $(obj) -o $(target)
49
50 %.o:%.c
51     gcc $< -c
52
53 # 这个版本的优点：文件精简不少，变得简洁了
54 # 这个版本的缺点：变量 obj 的值需要手动的写出来，如果需要编译的项目文件很多，都用手写出来不现实
55 # 改进方式：在makefile中使用函数
56
57
58
59 # 版本4
60
61 # 添加自定义变量 -> makefile中注释前 使用 #
62 # 使用函数搜索当前目录下的源文件 .c
63 src=$(wildcard *.c)
64 # 将源文件的后缀替换为 .o
65 # % 匹配的内容是不能被替换的，需要替换的是第一个参数中的后缀，替换为第二个参数中指定的后缀
66 # obj=$(patsubst %.cpp, %.o, $(src)) 将src中的关键字 .cpp 替换为 .o
67 obj=$(patsubst %.c, %.o, $(src))
68 target=calc
69
70 $(target):$(obj)
71     gcc $(obj) -o $(target)
72
73 %.o:%.c
74     gcc $< -c
75
76 # 这个版本的优点：解决了自动加载项目文件的问题，解放了双手
77 # 这个版本的缺点：没有文件删除的功能，不能删除项目编译过程中生成的目标文件 (*.o) 和可执行程序
78 # 改进方式：在makefile文件中添加新的规则用于删除生成的目标文件 (*.o) 和可执行程序

```



```

79
80
81
82 # 版本5
83
84 # 添加自定义变量 -> makefile中注释前 使用 #
85 # 使用函数搜索当前目录下的源文件 .c
86 src=$(wildcard *.c)
87 # 将源文件的后缀替换为 .o
88 obj=$(patsubst %.c, %.o, $(src))
89 target=calc
90 # obj 的值 xxx.o xxx.o xxx.o xx.o
91 $(target):$(obj)
92     gcc $(obj) -o $(target)
93
94 %.o:%.c
95     gcc $< -c
96
97 # 添加规则，删除生成文件 *.o 可执行程序
98 # 这个规则比较特殊，clean根本不会生成，这是一个伪目标
99 clean:
100     rm $(obj) $(target)
101
102 # 这个版本的优点：添加了新的规则（16行）用于文件的删除，直接 make clean 就可以执行规则中的删除命令了
103 # 这个版本的缺点：在下面有具体的问题演示和分析
104 # 改进方式：在makefile文件中声明 clean是一个伪目标，让 make 放弃对它的时间戳检测。
105
106 # 正常情况下这个版本的makefile是可以正常工作的，但是我们如果在这个项目目录中添加一个叫做clean的文件
    （和规则中的目标名称相同），再进行 make clean发现这个规则就不能正常工作了。
107
108 # 在项目目录中添加一个叫 clean的文件，然后在 make clean 这个规则中的命令就不工作了
109 $ ls
110 add.c  calc  div.c  head.h  main.o  mult.c  sub.c
111 add.o  div.o  main.c  makefile  mult.o  sub.o  clean ---> 新添加的
112
113 # 使用 makefile 中的规则删除生成的目标文件和可执行程序
114 $ make clean
115 make: 'clean' is up to date.
116
117 # 查看目录，发现相关文件并没有被删除，make clean 失败了
118 $ ls
119 add.c  calc  div.c  head.h  main.o  mult.c  sub.c
120 add.o  clean  div.o  main.c  makefile  mult.o  sub.o
121
122 # 这个问题的关键点在于 clean是一个伪目标，不对应任何实体文件，在前边讲关于文件时间戳更新问题的时候
    说过，如果目标不存在规则的命令肯定被执行，如果目标文件存在了就需要比较规则中目标文件和依赖文件的时间
    戳，满足条件才执行规则的命令，否则不执行。
123 # 解决这个问题需要在 makefile 中声明 clean是一个伪目标，这样 make 就不会对文件的时间戳进行检测，
    规则中的命令也就每次都会被执行了。
124 # 在 makefile 中声明一个伪目标需要使用 .PHONY 关键字，声明方式为：.PHONY:伪文件名称
125
126

```

```

127
128
129 # 最终版
130
131 # 添加自定义变量 -> makefile中注释前 使用 #
132 # 使用函数搜索当前目录下的源文件 .c
133 src=$(wildcard *.c)
134 # 将源文件的后缀替换为 .o
135 obj=$(patsubst %.c, %.o, $(src))
136 target=calc
137
138 $(target):$(obj)
139     gcc $(obj) -o $(target)
140
141 %.o:%.c
142     gcc $< -c
143
144 # 添加规则，删除生成文件 *.o 可执行程序
145 # 声明clean为伪文件
146 .PHONY:clean
147 clean:
148     # shell命令前的 - 表示强制这个指令执行，如果执行失败也不会终止
149     -rm $(obj) $(target)
150     echo "hello, 我是测试字符串"
151

```

Fence 61

- 在 makefile 中声明一个伪目标需要使用 .PHONY 关键字, 声明方式为: **.PHONY:伪文件名称**

```

1 # 目录结构
2 .
3 |— include
4 |   |— head.h ==> 头文件，声明了加减乘除四个函数
5 |— main.c ==> 测试程序，调用了head.h中的函数
6 |— src
7     |— add.c ==> 加法运算
8     |— div.c ==> 除法运算
9     |— mult.c ==> 乘法运算
10    |— sub.c ==> 减法运算
11
12 # makefile文件
13 # 最终的目标名 app
14 target = app
15 # 搜索当前项目目录下的源文件
16 src=$(wildcard *.c ./src/*.c)
17 # 将文件的后缀替换掉 .c -> .o
18 obj=$(patsubst %.c, %.o, $(src))
19 # 头文件目录
20 include=./include
21
22 # 第一条规则
23 # 依赖中都是 xx.o yy.o zz.o
24 # gcc命令执行的是链接操作

```

```

25 $(target):$(obj)
26     gcc $^ -o $@
27
28 # 模式匹配规则
29 # 执行汇编操作，前两步：预处理，编译是自动完成
30 %.o:%.c
31     gcc $< -c -I $(include) -o $@
32
33 # 添加一个清除文件的规则
34 .PHONY:clean
35
36 clean:
37     -rm $(obj) $(target) -f

```

Fence 62

7. GDB调试

- gdb 是由 GNU 软件系统社区提供的调试器，同 gcc 配套组成了一套完整的开发环境，可移植性很好，支持非常多的体系结构并被移植到各种系统中（包括各种类 Unix 系统与 Windows 系统里的 MinGW 和 Cygwin）。此外，除了 C 语言之外，gcc/gdb 还支持包括 C++、Objective-C、Ada 和 Pascal 等各种语言后端的编译和调试。gcc/gdb 是 Linux 和许多类 Unix 系统中的标准开发环境，Linux 内核也是专门针对 gcc 进行编码的。
- GDB 是一套字符界面的程序集，可以使用命令 gdb 加载要调试的程序。

7.1 调式准备

- 项目程序如果是为了进行调试而编译时，必须要打开调试选项(-g)。另外还有一些可选项，比如：在尽量不影响程序行为的情况下关掉编译器的优化选项(-O0)，-Wall选项打开所有 warning，也可以发现许多问题，避免一些不必要的 bug。
- -g选项的作用是在可执行文件中加入源代码的信息，比如可执行文件中第几条机器指令对应源代码的第几行，但并不是把整个源文件嵌入到可执行文件中，所以在调试时必须保证gdb能找到源文件。
- 习惯上如果是c程序就使用gcc编译, 如果是 c++程序就使用g++编译, 编译命令中添加上边提到的参数即可。
 - 假设有一个文件 args.c, 要对其进行gdb调试，编译的时候必须要添加参数 -g，加入了源代码信息的可执行文件比不加之前要大一些。

```

1 # -g 将调试信息写入到可执行程序中
2 $ gcc -g args.c -o app
3
4 # 编译不添加 -g 参数
5 $ gcc args.c -o app1
6
7 # 查看生成的两个可执行程序的大小
8 $ ll
9
10 -rwxrwxr-x 1 robin robin 9816 Apr 19 09:25 app*      # 可以用于gdb调试
11 -rwxrwxr-x 1 robin robin 8608 Apr 19 09:25 app1*    # 不能用于gdb调试

```

Fence 63

7.2 启动和退出gdb

7.2.1 启动gdb

- gdb是一个用于应用程序调试的进程, 需要先将其打开, **一定要注意 gdb进程启动之后, 需要的被调试的应用程序是没有执行的**。打开Linux终端, 切换到要调试的可执行程序所在路径, 执行如下命令就可以启动 gdb了。

```
1  # 在终端中执行如下命令
2  # gdb程序启动了, 但是可执行程序并没有执行
3  $ gdb 可执行程序的名字
4
5  # 使用举例:
6  $ gdb app
7  (gdb)      # gdb等待输入调试的相关命令
```

Fence 64

7.2.2 命令行传参

- 有些程序在启动的时候需要传递命令行参数, 如果要调试这类程序, 这些命令行参数必须要在应用程序启动之前通过调试程序的gdb进程传递进去。下面是一段带命令行参数的程序:

```
1  // args.c
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5  #include <string.h>
6
7  #define NUM 10
8
9  // argc, argv 是命令行参数
10 // 启动应用程序的时候
11 int main(int argc, char* argv[])
12 {
13     printf("参数个数: %d\n", argc);
14     for(int i=0; i<argc; ++i)
15     {
16         printf("%d\n", NUM);
17         printf("参数 %d: %s\n", i, argv[i]);
18     }
19     return 0;
20 }
21
```

Fence 65

```

1  # 第一步：编译出带条信息的可执行程序
2  $ gcc args.c -o app -g
3  # 第二步：启动gdb进程，指定需要gdb调试的应用程序名称
4  $ gdb app
5  (gdb)
6  # 第三步：在启动应用程序 app之前设置命令行参数。gdb中设置参数的命令叫做set args ...，查看设置的
   # 命令行参数命令是 show args。语法格式如下：
7  # 设置的时机：启动gdb之后，在应用程序启动之前
8  (gdb) set args 参数1 参数2 ....
9  # 查看设置的命令行参数
10 (gdb) show args
11

```

Fence 66

```

1  # 例子
2  # 非gdb调试命令行传参
3  # argc 参数总个数，argv[0] == ./app, argv[1] == "11" argv[2] == "22" ... argv[5]
   == "55"
4  $ ./app 11 22 33 44 55      # 这是数据传递给main函数
5
6  # 使用 gdb 调试
7  $ gdb app
8  GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.5) 7.11.1
9  Copyright (C) 2016 Free Software Foundation, Inc.
10 # 通过gdb给应用程序设置命令行参数
11 (gdb) set args 11 22 33 44 55
12 # 查看设置的命令行参数
13 (gdb) show args
14 Argument list to give program being debugged when it is started is "11 22 33 44
   55"

```

Fence 67

7.2.3 gdb中启动程序

- 在gdb中启动要调试的应用程序有两种方式, 一种是使用run命令, 另一种是使用start命令启动。在整个 gdb 调试过程中, 启动应用程序的命令只能使用一次。
 - run: 可以缩写为 r, 如果程序中设置了断点会停在第一个断点的位置, 如果没有设置断点, 程序就执行完了
 - start: 启动程序, 最终会阻塞在main函数的第一行, 等待输入后续其它 gdb 指令

```

1  # 两种方式
2  # 方式1: run == r
3  (gdb) run
4
5  # 方式2: start
6  (gdb) start

```

Fence 68

- 如果想让程序start之后继续运行, 或者在断点处继续运行, 可以使用 continue命令, 可以简写为 c

```
1 # continue == c
2 (gdb) continue
```

Fence 69

7.2.4 退出gdb

- 退出gdb调试, 就是终止 gdb 进程, 需要使用 quit命令, 可以缩写为 q

```
1 # quit == q
2 (gdb) quit
```

Fence 70

7.3 查看代码

- 因为gdb调试没有IDE那样的完善的可视化窗口界面, 给调试的程序打断点又是调试之前必须做的一项工作。因此gdb提供了查看代码的命令, 这样就可以轻松定位要调试的代码行的位置了。

查看代码的命令叫做**list**可以缩写为 **l**, 通过这个命令我们可以查看项目中任意一个文件中的内容, 并且还可以通过文件行号, 函数名等方式查看。

7.3.1 当前文件

- 一个项目中一般是有很多源文件的, **默认情况下通过list查看到代码信息位于程序入口函数main对应的的那个文件中**。因此如果不进行文件切换main函数所在的文件就是当前文件, 如果进行了文件切换, 切换到哪个文件哪个文件就是当前文件。查看文件内容的方式如下:

```
1 # 使用 list 和使用 l 都可以
2 # 从第一行开始显示
3 (gdb) list
4
5 # 列值这行号对应的上下文代码, 默认情况下只显示10行内容
6 (gdb) list 行号
7
8 # 显示这个函数的上下文内容, 默认显示10行
9 (gdb) list 函数名
10
```

Fence 71

- 通过list去查看文件代码, 默认只显示10行, 如果还想继续查看后边的内容, 可以继续执行list命令, 也可以**直接回车 (再次执行上一次执行的那个gdb命令)**。

7.3.2 切换文件

- 在查看文件内容的时候, 很多情况下需要进行文件切换, 我们只需要在list命令后边将要查看的文件名指定出来就可以了, 切换命令执行完毕之后, 这个文件就变成了当前文件。文件切换方式如下:

```
1 # 切换到指定的文件, 并列出这行号对应的上下文代码, 默认情况下只显示10行内容
2 (gdb) l 文件名:行号
3
4 # 切换到指定的文件, 并显示这个函数的上下文内容, 默认显示10行
5 (gdb) l 文件名:函数名
```

- 默认通过list只能一次查看10行代码, 如果想显示更多, 可以通过set listsize设置, 同样如果想查看当前显示的行数可以通过 show listsize查看, 这里的listsize可以简写为 list。具体语法格式如下:

```
1  # 以下两个命令中的 listsize 都可以写成 list
2  (gdb) set listsize 行数
3
4  # 查看当前list一次显示的行数
5  (gdb) show listsize
```

7.4 断点操作

- 想要通过gdb调试某一行或者得到某个变量在运行状态下的实际值, 就需要在在这一行设置断点, 程序指定到断点的位置就会阻塞, 我们就可以通过gdb的调试命令得到我们想要的信息了。

设置断点的命令叫做break可以缩写为b。

7.4.1 设置断点

- 断点的设置有两种方式一种是**常规断点**, 程序只要运行到这个位置就会被阻塞, 还有一种叫**条件断点**, 只有指定的条件被满足了程序才会在断点处阻塞。

调试程序的断点可以设置到某个具体的行, 也可以设置到某个函数上, 具体的设置方式如下:

- 设置普通断点到当前文件

```
1  # 在当前文件的某一行上设置断点
2  # break == b
3  (gdb) b 行号
4  (gdb) b 函数名      # 停止在函数的第一行
```

- 设置普通断点到某个非当前文件上

```
1  # 在非当前文件的某一行上设置断点
2  (gdb) b 文件名:行号
3  (gdb) b 文件名:函数名      # 停止在函数的第一行
```

- 设置条件断点

```
1  # 必须要满足某个条件, 程序才会停在这个断点的位置上
2  # 通常情况下, 在循环中条件断点用的比较多
3  (gdb) b 行数 if 变量名==某个值
```

7.4.2 查看断点

- 断点设置完毕之后, 可以通过 **info break**命令查看设置的断点信息, 其中**info**可以缩写为**i**

```
1 # info == i
2 # 查看设置的断点信息
3 (gdb) i b #info break
4
5 # 举例
6 (gdb) i b
7 Num      Type      Disp Enb Address      What
8 1        breakpoint keep y 0x0000000000400cb5 in main() at test.cpp:12
9 2        breakpoint keep y 0x0000000000400cbd in main() at test.cpp:13
10 3        breakpoint keep y 0x0000000000400cec in main() at test.cpp:18
11 4        breakpoint keep y 0x00000000004009a5 in insertionSort(int*, int)
12                                     at insert.cpp:8
13 5        breakpoint keep y 0x0000000000400cdd in main() at test.cpp:16
14 6        breakpoint keep y 0x00000000004009e5 in insertionSort(int*, int)
```

Fence 77

- 在显示的断点信息中有一些属性需要在其他操作中被使用, 下面介绍一下:

- Num: 断点的编号, 删除断点或者设置断点状态的时候都需要使用
- Enb: 当前断点的状态, y表示断点可用, n表示断点不可用
- What: 描述断点被设置在了哪个文件的哪一行或者哪个函数上

7.4.3 删除断点

- 如果确定设置的某个断点不再被使用了, 可用将其删除, 删除命令是 **delete 断点编号**, 这个**delete**可以简写为**del**也可以再简写为**d**。

删除断点的方式有两种: 删除(一个或者多个)指定断点或者删除一个连续的断点区间, 具体操作如下:

```
1 # delete == del == d
2 # 需要 info b 查看断点的信息, 第一列就是编号
3 (gdb) d 断点的编号1 [断点编号2 ...]
4 # 举例:
5 (gdb) d 1 # 删除第1个断点
6 (gdb) d 2 4 6 # 删除第2,4,6个断点
7
8 # 删除一个范围, 断点编号 num1 - numN 是一个连续区间
9 (gdb) d num1-numN
10 # 举例, 删除第1到第5个断点
11 (gdb) d 1-5
```

Fence 78

7.4.4 设置断点状态

- 如果某个断点只是临时不需要了，我们可以将其**设置为不可用状态**，设置命令为**disable 断点编号**，当需要的时候再将其**设置回可用状态**，设置命令为 **enable 断点编号**。
- 设置断点无效

```
1  # 让断点失效之后，gdb调试过程中程序是不会停在这个位置的
2  # disable == dis
3  # 设置某一个或者某几个断点无效
4  (gdb) dis 断点1的编号 [断点2的编号 ...]
5
6  # 设置某个区间断点无效
7  (gdb) dis 断点1编号-断点n编号
8
9
10 # 例子
11 # 查看断点信息
12 (gdb) i b
13
14 Num      Type      Disp Enb Address          What
15 2        breakpoint keep y  0x0000000000400cce in main() at test.cpp:14
16 4        breakpoint keep y  0x0000000000400cdd in main() at test.cpp:16
17 5        breakpoint keep y  0x0000000000400d46 in main() at test.cpp:23
18 6        breakpoint keep y  0x0000000000400d4e in main() at test.cpp:25
19 7        breakpoint keep y  0x0000000000400d6e in main() at test.cpp:28
20 8        breakpoint keep y  0x0000000000400d7d in main() at test.cpp:30
21
22 # 设置第2，第4 个断点无效
23 (gdb) dis 2 4
24
25 # 查看断点信息
26 (gdb) i b
27
28 Num      Type      Disp Enb Address          What
29 2        breakpoint keep n  0x0000000000400cce in main() at test.cpp:14
30 4        breakpoint keep n  0x0000000000400cdd in main() at test.cpp:16
31 5        breakpoint keep y  0x0000000000400d46 in main() at test.cpp:23
32 6        breakpoint keep y  0x0000000000400d4e in main() at test.cpp:25
33 7        breakpoint keep y  0x0000000000400d6e in main() at test.cpp:28
34 8        breakpoint keep y  0x0000000000400d7d in main() at test.cpp:30
35
36 # 设置 第5,6,7,8个 断点无效
37 (gdb) dis 5-8
38
39 # 查看断点信息
40 (gdb) i b
41
42 Num      Type      Disp Enb Address          What
43 2        breakpoint keep n  0x0000000000400cce in main() at test.cpp:14
44 4        breakpoint keep n  0x0000000000400cdd in main() at test.cpp:16
45 5        breakpoint keep n  0x0000000000400d46 in main() at test.cpp:23
46 6        breakpoint keep n  0x0000000000400d4e in main() at test.cpp:25
47 7        breakpoint keep n  0x0000000000400d6e in main() at test.cpp:28
48 8        breakpoint keep n  0x0000000000400d7d in main() at test.cpp:30
```

- 让无效的断点生效

```

1  # enable == ena
2  # 设置某一个或者某几个断点有效
3  (gdb) ena 断点1的编号 [断点2的编号 ...]
4
5  # 设置某个区间断点有效
6  (gdb) ena 断点1编号-断点n编号
7
8
9  # 例子
10 # 查看断点信息
11 (gdb) i b
12
13 Num      Type      Disp Enb Address      What
14 2        breakpoint keep n  0x0000000000400cce in main() at test.cpp:14
15 4        breakpoint keep n  0x0000000000400cdd in main() at test.cpp:16
16 5        breakpoint keep n  0x0000000000400d46 in main() at test.cpp:23
17 6        breakpoint keep n  0x0000000000400d4e in main() at test.cpp:25
18 7        breakpoint keep n  0x0000000000400d6e in main() at test.cpp:28
19 8        breakpoint keep n  0x0000000000400d7d in main() at test.cpp:30
20
21 # 设置第2, 第4个断点有效
22 (gdb) ena 2 4
23
24 # 查看断点信息
25 (gdb) i b
26
27 Num      Type      Disp Enb Address      What
28 2        breakpoint keep y  0x0000000000400cce in main() at test.cpp:14
29 4        breakpoint keep y  0x0000000000400cdd in main() at test.cpp:16
30 5        breakpoint keep n  0x0000000000400d46 in main() at test.cpp:23
31 6        breakpoint keep n  0x0000000000400d4e in main() at test.cpp:25
32 7        breakpoint keep n  0x0000000000400d6e in main() at test.cpp:28
33 8        breakpoint keep n  0x0000000000400d7d in main() at test.cpp:30
34
35 # 设置第5,6,7个断点有效
36 (gdb) ena 5-7
37
38 # 查看断点信息
39 (gdb) i b
40
41 Num      Type      Disp Enb Address      What
42 2        breakpoint keep y  0x0000000000400cce in main() at test.cpp:14
43 4        breakpoint keep y  0x0000000000400cdd in main() at test.cpp:16
44 5        breakpoint keep y  0x0000000000400d46 in main() at test.cpp:23
45 6        breakpoint keep y  0x0000000000400d4e in main() at test.cpp:25
46 7        breakpoint keep y  0x0000000000400d6e in main() at test.cpp:28
47 8        breakpoint keep n  0x0000000000400d7d in main() at test.cpp:30

```

7.5 调式命令

7.5.1 继续运行gdb

- 如果调试的程序被断点阻塞了又想让程序继续执行，这时候就可以使用continue命令。程序会继续运行, 直到遇到下一个有效的断点。continue可以缩写为 c。

```
1  # continue == c
2  (gdb) continue
```

Fence 81

7.5.2 手动打印信息

- 当程序被某个断点阻塞之后, 可以通过一些命令打印变量的名字或者变量的类型，并且还可以跟踪打印某个变量的值。

7.5.2.1 打印变量值

- 在gdb调试的时候如果需要打印变量的值，使用的命令是 **print**, 可缩写为 **p**。如果打印的变量是整数还可以指定输出的整数的格式, 格式化输出的整数对应的字符表如下：

格式化字符(/fmt)	说明
/x	以十六进制的形式打印出整数。
/d	以有符号、十进制的形式打印出整数。
/u	以无符号、十进制的形式打印出整数。
/o	以八进制的形式打印出整数。
/t	以二进制的形式打印出整数。
/f	以浮点数的形式打印变量或表达式的值。
/c	以字符形式打印变量或表达式的值。

Table 21

- print命令的语法格式如下:

```
1  # print == p
2  (gdb) p 变量名
3
4  # 如果变量是一个整形，默认对应的值是以10进制格式输出，其他格式请参考上表
5  (gdb) p/fmt 变量名
```

Fence 82

- 举例:

```

1  # 举例
2  (gdb) p i      # 10进制
3  $5 = 3
4  (gdb) p/x i    # 16进制
5  $6 = 0x3
6  (gdb) p/o i    # 8进制
7  $7 = 03

```

Fence 83

7.5.2.2 打印变量类型

- 如果在调试过程中需要查看某个变量的类型, 可以使用命令ptype, 语法格式如下:

```

1  # 语法格式
2  (gdb) ptype 变量名

```

Fence 84

- 例子

```

1  # 打印变量类型
2  (gdb) ptype i
3  type = int
4  (gdb) ptype array[i]
5  type = int
6  (gdb) ptype array
7  type = int [12]

```

Fence 85

7.5.3 自动打印信息

7.5.3.1 设置变量名自动显示

- 和 print 命令一样, **display** 命令也用于调试阶段查看某个变量或表达式的值, 它们的区别是, **使用 display 命令查看变量或表达式的值, 每当程序暂停执行 (例如单步执行) 时, GDB 调试器都会自动帮我们打印出来, 而 print 命令则不会。**因此, 当我们想频繁查看某个变量或表达式的值从而观察它的变化情况时, 使用 display 命令可以一劳永逸。display 命令没有缩写形式, 常用的语法格式如下 2 种:

```

1  # 在变量的有效取值范围内, 自动打印变量的值(设置一次, 以后就会自动显示)
2  (gdb) display 变量名
3
4  # 以指定的整形格式打印变量的值, 关于 fmt 的取值, 请参考 print 命令的格式化字符表
5  (gdb) display/fmt 变量名

```

Fence 86

7.5.3.2 查看自动显示列表

- 对于使用 `display` 命令查看的目标变量或表达式，都会被记录在一张列表（称为自动显示列表）中。通过执行 `info display` 命令，可以打印出这张表：

```
1  # info == i
2  (gdb) info display
3  Auto-display expressions now in effect:
4  Num Enb Expression
5  1:   y   i
6  2:   y  array[i]
7  3:   y  /x array[i]
```

Fence 87

- 在展示出的信息中, 每个列的含义如下:
 - Num: 变量或表达式的编号, GDB 调试器为每个变量或表达式都分配有唯一的编号
 - Enb: 表示当前变量（表达式）是处于激活状态还是禁用状态, 如果处于激活状态（用 y 表示）, 则每次程序停止执行, 该变量的值都会被打印出来; 反之, 如果处于禁用状态（用 n 表示）, 则该变量（表达式）的值不会被打印。
 - Expression: 被自动打印值的变量或表达式的名字。

7.5.3.3 取消自动显示

- 对于不需要再打印值的变量或表达式, 可以将其删除或者禁用。
- 删除自动显示列表中的变量或表达式

```
1  # 命令中的 num 是通过 info display 得到的编号, 编号可以是一个或者多个
2  (gdb) undisplay num [num1 ...]
3  # num1 - numN 表示一个范围
4  (gdb) undisplay num1-numN
5
6  (gdb) delete display num [num1 ...]
7  (gdb) delete display num1-numN
8
9
10 # 例子
11 # 查看显示列表
12 (gdb) info display
13 Auto-display expressions now in effect:
14 Num Enb Expression
15 1:   y   i
16 2:   y  array[i]
17 3:   y  /x array[i]
18
19 # 删除变量显示, 需要使用 info display 得到的变量/表达式编号
20 (gdb) undisplay 1 2
21
22 # 查看显示列表, 只剩下一个了
23 (gdb) i display
24 Auto-display expressions now in effect:
25 Num Enb Expression
```

```
26 3: y /x array[i]
27
```

Fence 88

- 如果不想删除自动显示的变量, 也可以**禁用**自动显示列表中处于激活状态下的变量或表达式

```
1 # 命令中的 num 是通过 info display 得到的编号, 编号可以是一个或者多个
2 (gdb) disable display num [num1 ...]
3 # num1 - numN 表示一个范围
4 (gdb) disable display num1-numN
```

Fence 89

- 当需要**启用**自动显示列表中被禁用的变量或表达式时, 可以使用下边的命令

```
1 # 命令中的 num 是通过 info display 得到的编号, 编号可以是一个或者多个
2 (gdb) enable display num [num1 ...]
3 # num1 - numN 表示一个范围
4 (gdb) enable display num1-numN
```

Fence 90

7.5.4 单步调式

- 当程序阻塞到某个断点上之后, 可以通过以下命令对程序进行单步调试:

7.5.4.1 step

- step命令可以缩写为s, 命令被执行一次代码被向下执行一行, **如果这一行是一个函数调用, 那么程序会进入到函数体内部。**

```
1 # 从当前代码行位置, 一次调试当前行下的每一行代码
2 # step == s
3 # 如果这一行是函数调用, 执行这个命令, 就可以进入到函数体的内部
4 (gdb) step
```

Fence 91

7.5.4.2 finish

- 如果通过 s 单步调试**进入到函数内部, 想要跳出这个函数体**, 可以执行finish命令。**如果想要跳出函数体必须要保证函数体内不能有有效断点, 否则无法跳出。**

```
1 # 如果通过 s 单步调试进入到函数内部, 想要跳出这个函数体
2 (gdb) finish
```

Fence 92

7.5.4.3 next

- next命令和step命令功能是相似的, **只是在使用next调试程序的时候不会进入到函数体内部**, next可以缩写为 n

```
1 # next == n
2 # 如果这一行是函数调用, 执行这个命令, 不会进入到函数体的内部
3 (gdb) next
```

- 通过 `until` 命令**可以直接跳出某个循环体**，这样就能提高调试效率了。如果想直接从循环体中跳出, 必须要满足以下的条件，否则命令不会生效：
 - 要跳出的循环体内部不能有有效的断点
 - 必须要在循环体的开始/结束行执行该命令

```
1 | (gdb) until
```

7.6 设置变量值

- 在调试程序的时候, 我们需要在某个变量等于某个特殊值的时候查看程序的运行状态, 但是通过程序运行让变量等于这个值又非常困难, 这种情况下就可以在 `gdb` 中直接对这个变量进行值的设置, 或者是在单步调试的时候通过设置循环因子的值直接跳出某个循环, 值设置的命令格式为: **`set var 变量名=值`**

```
1 | # 可以在循环中使用，直接设置循环因子的值
2 | # 假设某个变量的值在程序中==90的概率是5%，这时候可以直接通过命令将这个变量值设置为90
3 | (gdb) set var 变量名=值
```

8. Linux命令

查找命令：微信小程序 [Linux命令速查表](#)

8.1 `cd`: 目录跳转命令

- 进入指定目录

```
1 | cd 目录名
2 | #目录名：使用相对路径/绝对路径都可以，该路径必须是一个有效路径
3 | #特殊的目录：
4 | # ..：表示当前目录的上一级目录，使用cd .. 或者 cd ../都可以
5 | # .：表示当前目录，使用 .或者./都可以，cd .不会切换目录
```

- 进入家目录
 - 每个用户(普通用户和root)都有一个属于自己的目录, 比如:
 - siguyuan是用户, 家目录: `/home/siguyuan`
 - root是管理员用户, 家目录: `/root`

- 按照上边的格式来表示家目录书写起来比较麻烦, 有一种相对简单的写法就是**使用 ~ 表示**。如果是使用cd命令切换到家目录, 后边可以什么路径都不加, 这样也可以进入到当前用户的家目录。综上所述也就是通过cd进入到当前用户的家目录一共有三种不同的方式。

```
1 cd          #方式1
2 cd ~        #方式2
3 cd /home/用户名 #方式3
4
5 #前两种方式只能切换到当前目录
```

Fence 97

- 在临近的两个目录之间切换
 - 如果我们要频繁的在两个路径之间切换, 也有相关的快捷操作, 尤其是对于比较长的路径, 可以是这简直是一个福利:

```
1 # 通过cd进入到目录1: /usr/include/c++/7.5.0/ext/pb_ds/detail/list_update_map_/
2 # 通过cd进入到目录2: /home/luffy/get/onepiece/itis/a/goldfish
3 # 使用下面命令, 可以频繁在两个目录之间切换
4 cd -
```

Fence 98

8.2 ls: 查看当前目录的文件

```
1 ls [args]      # 查看当前目录
2 ls [args] 目录名 # 查看指定目录
3 ls [args] 文件名 # 查看某个文件的信息
```

Fence 99

8.2.1 显示所有文件（包括隐藏文件）

- 给ls添加**-a**参数（就是all的意思）就可以显示指定目录中是所有文件了, 因为默认情况下具有隐藏属性的文件是不会显示出来的。那么在Linux中什么样是文件才能被隐藏呢? 其实很简单只需要在**文件名前边加一个点(.)文件就具有隐藏属性了**, 例如: .demo, .a.txt

```
1 # 查看siguyuan家目录下所有非隐藏文件
2 [siguyuan@10 ~]$ ls
3 公共 模板 视频 图片 文档 下载 音乐 桌面
4
5 # 查看siguyuan家目录下所有文件（隐藏 + 非隐藏）
6 [siguyuan@10 ~]$ ls -a
7 . .bash_history .bash_profile .cache .dbus .ICEauthority .mozilla 公共
  视频 文档 音乐
8 .. .bash_logout .bashrc .config .esd_auth .local .Xauthority 模板
  图片 下载 桌面
```

Fence 100

8.2.2 显示文件详细信息（长文件显示）

- 给 ls 添加 -l 参数（就是 list 的意思）我们就可以看到文件的详细信息了，里边的信息量还是非常大的，其中包括：文件类型，文件所有者对文件的操作权限，文件所属组用户对文件的操作权限，其他人对文件的操作权限，硬链接计数，文件所有者，文件所属组，文件大小，文件的修改日期，文件名

```
1 # 显示文件的详细信息
2 [siguyuan@10 ~]$ ls -l
3 总用量 0
4 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 公共
5 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 模板
6 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 视频
7 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 图片
8 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 文档
9 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 下载
10 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 音乐
11 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 桌面
12
13
14 # 文件详细信息介绍
15 d      rwx      rwx      r-x      5      robin      robin      4096      Jan 13 17:35      udp
16 |      |      |      |      |      |      |      |      |      |
17 文件    文件所    文件所    其他人  硬链接    文件      文件      文件大小  文件修改时间  文件
18 名
19 类型    有者权限  属组权限  权限    计数     所有者    所属组
```

Fence 101

- 在查看文件详细信息的时候，还有一种简单的写法，可以使用 ll 命令：
 - 有些版本的Linux中 ll 等价于 ls -l；
 - 有些版本的Linux中 ll 等价于 ls -laF；

8.2.2.1 文件类型

- 在Linux操作系统中，一共有7中文件类型，这7中类型是根据文件属性进行划分的，而不是根据文件后缀划分的。
 - : 普通的文件，在Linux终端中没有执行权限的为白色，压缩包为红色，可执行程序为绿色字体
 - d: 目录(directory)，在Linux终端中为蓝色字体，如果目录的所有权限都是开放的，有绿色的背景色
 - l: 软链接文件(link)，相当于windows中的快捷方式，在Linux终端中为淡蓝色(青色)字体
 - c: 字符设备(char)，在Linux终端中为黄色字体
 - b: 块设备(block)，在Linux终端中为黄色字体
 - p: 管道文件(pipe)，在Linux终端中为棕黄色字体
 - s: 本地套接字文件(socket)，在Linux终端中为粉色字体

```
[root@VM-8-14-centos ~/file]# ls -l
drwxr-xr-x 2 root root 4096 Jan 25 14:29 dir
lrwxrwxrwx 1 root root 15 Jan 25 14:30 link -> /root/file/test
prw-r--r-- 1 root root 0 Jan 25 14:24 pipe
-rw-r--r-- 1 root root 0 Jan 25 14:23 test
crw-rw-rw- 1 root root 3 Jan 24 17:24 null
crw-r----- 1 root kmem 1 Jan 24 17:24 mem
brw-rw---- 1 root disk 2253 Jan 24 17:24 vda1
srwxrwxrwx 1 root root 0 Jan 24 17:25 yd.socket.server
-rw-r--r-- 1 root root 0 Jan 25 14:49 haha.tar.gz
-rwxrwxrwx 1 root root 0 Jan 25 14:49 hello
prw-r--r-- 1 root root 0 Jan 25 14:24 pipe-2
drwxrwxrwx 2 root root 4096 Jan 25 15:20 subdir
```

Figure 16

8.2.2.2 用户类型

- 在Linux中有三大类用户: **文件所有者**, **文件所属组用户**, **其他人**, 我们可以对同一个文件给这三种人设置不同的操作权限, 用于限制用户对文件的访问。
 - 文件所有者
 - Linux中的所有文件都有一个所有者, 就是文件的主人
 - 文件所属组 (一般创建所有者时, 会创建一个同名的组)
 - 文件的主人属于哪个组, 这个文件默认也就属于哪个组
 - 用户组中可以有多个用户, 这些组中的其他用户和所有者的权限可以是不一样的
 - 其他人
 - 这个用户既不是文件所有者也不是文件所属组中的用户, 就称之为其他人
 - 其他用户对文件也可以拥有某些权限

8.2.2.3 用户权限

- Linux中不同的用户可以对文件拥有不同的操作权限, 权限一共有四种: 读权限, 写权限, 执行权限, 无权限。
 - 读权限: 使用 r 表示, 即: read
 - 写权限: 使用 w 表示, 即: write
 - 执行权限: 使用 x 表示, 即: excute
 - 没有任何权限: 使用 - 表示

1	#crw-rw-----	1	root	video	10, 175	4月	21	21:16	agpgart
2	c	rw-	rw-	---	.	1	root	video	10, 175 4月 21 21:16 agpgart
3									
4	文件类型	文件所有者权限	文件所属组权限	其他人权限	安全设置				
5					相关				

- 从上边的例子中可以看出:
 - 文件所有者对文件有读、写权限、没有执行权限

- 文件所属组用户对文件有 读、写权限、没有执行权限
- 其他人对文件有没有任何权限

8.2.2.4 硬链接计数（起别名，C++引用，共用内存，第二列）

- 硬链接计数是一个整数，如果这个数为 $N(N \geq 1)$ ，就说明在一个或者多个目录下一共有 N 个文件，但是这 N 个文件并不占用多块磁盘空间，他们使用的是同一块，如果通过其中一个文件修改了磁盘数据，那么其他文件中的内容也就变了。每当我们给磁盘文件创建一个硬链接（使用 `ln`），磁盘上就会出现一个新的文件名，硬链接计数加1，但是这新文件并不占用任何的磁盘空间，文件名还是映射到原来的磁盘地址上。

下图为大家展示了给文件创建硬链接，和直接进行文件拷贝的区别，**创建硬链接只是多了一个新的文件名，拷贝文件不仅多了新的文件名在磁盘上数据也进行了拷贝**

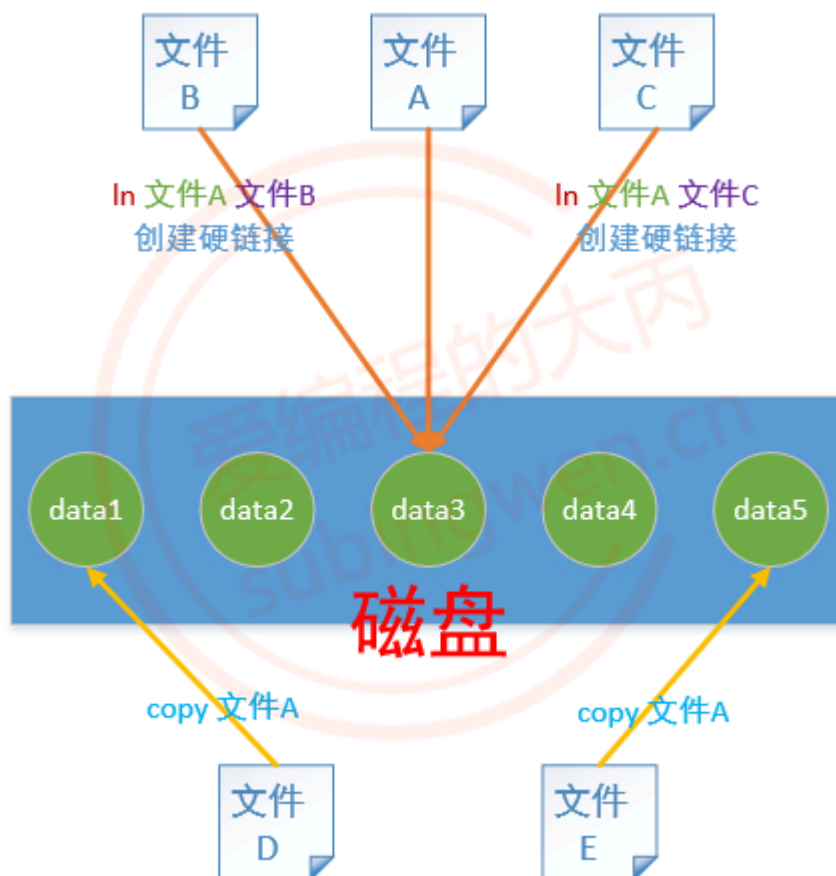


Figure 17

8.2.2.5 其他属性

1	lrwxrwxrwx.	1	root	root	13	4月 21 21:16	fd -> /proc/self/fd
2							
3	硬链接计数		所有者	所属组	文件大小	文件日期	文件名

Fence 103

- 文件大小 —> 单位是字节
 - 如果文件是目录显示为4096, 这是目录自身大小, 不包括目录中的文件大小
- 文件日期: 显示的是文件的修改日期, 只要文件被更新, 日期也会随之变化
- 文件名: 文件自己的名字

- 如果文件类型是软连接会这样显示: link -> /root/file/test, 后边的路径表示快捷方式链接的是哪个磁盘文件

8.2.3 单位显示

- 在查看文件大小的时候, 如果文件比较大对应的数组自然也就很大, 我们还需要基于字节进行相关的换算, 不能直观得到我们想要的结果, 如果数学不好, 我们可以使用一个参数 **-h** (human)(就是命令说人话)。

```
1 # 直接查看文件大小
2 [siguyuan@10 ~]$ ls -laF
3 总用量 36
4 drwx-----. 15 siguyuan siguyuan 4096 4月 25 09:57 ./
5 drwxr-xr-x. 3 root root 22 4月 21 01:37 ../
6 -rw-----. 1 siguyuan siguyuan 53 4月 21 15:46 .bash_history
7 -rw-r--r--. 1 siguyuan siguyuan 18 11月 25 2021 .bash_logout
8 -rw-r--r--. 1 siguyuan siguyuan 193 11月 25 2021 .bash_profile
9 -rw-r--r--. 1 siguyuan siguyuan 231 11月 25 2021 .bashrc
10 drwx-----. 17 siguyuan siguyuan 4096 4月 21 21:19 .cache/
11 drwxr-xr-x. 14 siguyuan siguyuan 261 4月 21 02:17 .config/
12 drwx-----. 3 siguyuan siguyuan 25 4月 21 02:16 .dbus/
13 -rw-----. 1 siguyuan siguyuan 16 4月 21 02:16 .esd_auth
14 -rw-----. 1 siguyuan siguyuan 620 4月 21 21:16 .ICEauthority
15 drwx-----. 3 siguyuan siguyuan 19 4月 21 02:16 .local/
16 drwxr-xr-x. 6 siguyuan siguyuan 81 4月 21 21:19 .mozilla/
17 -rw-----. 1 siguyuan siguyuan 56 4月 25 09:57 .xauthority
18 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 公共/
19 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 模板/
20 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 视频/
21 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 图片/
22 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 文档/
23 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 下载/
24 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 音乐/
25 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 桌面/
26
27 # 添加 -h 参数查看文件大小 (主要是第5列的区别)
28 [siguyuan@10 ~]$ ls -laFh
29 总用量 36K
30 drwx-----. 15 siguyuan siguyuan 4.0K 4月 25 09:57 ./
31 drwxr-xr-x. 3 root root 22 4月 21 01:37 ../
32 -rw-----. 1 siguyuan siguyuan 53 4月 21 15:46 .bash_history
33 -rw-r--r--. 1 siguyuan siguyuan 18 11月 25 2021 .bash_logout
34 -rw-r--r--. 1 siguyuan siguyuan 193 11月 25 2021 .bash_profile
35 -rw-r--r--. 1 siguyuan siguyuan 231 11月 25 2021 .bashrc
36 drwx-----. 17 siguyuan siguyuan 4.0K 4月 21 21:19 .cache/
37 drwxr-xr-x. 14 siguyuan siguyuan 261 4月 21 02:17 .config/
38 drwx-----. 3 siguyuan siguyuan 25 4月 21 02:16 .dbus/
39 -rw-----. 1 siguyuan siguyuan 16 4月 21 02:16 .esd_auth
40 -rw-----. 1 siguyuan siguyuan 620 4月 21 21:16 .ICEauthority
41 drwx-----. 3 siguyuan siguyuan 19 4月 21 02:16 .local/
42 drwxr-xr-x. 6 siguyuan siguyuan 81 4月 21 21:19 .mozilla/
43 -rw-----. 1 siguyuan siguyuan 56 4月 25 09:57 .xauthority
```

```

44 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 公共/
45 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 模板/
46 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 视频/
47 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 图片/
48 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 文档/
49 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 下载/
50 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 音乐/
51 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 桌面/
52

```

Fence 104

8.2.4 显示目录后缀

- 在查看文件信息的时候, 处理通过文件类型区分该文件是不是一个目录之外, 还可以通过一个参数 `-F` 在目录名后边显示一个 `/`, 这样就可以直接区分出来了。

```

1  # 直接查看文件信息
2  [siguyuan@10 ~]$ ls -l
3  总用量 0
4  drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 公共
5  drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 模板
6  drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 视频
7  drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 图片
8  drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 文档
9  drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 下载
10 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 音乐
11 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 桌面
12
13 # 添加了 -F 参数查看文件信息
14 [siguyuan@10 ~]$ ls -lF
15 总用量 0
16 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 公共/
17 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 模板/
18 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 视频/
19 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 图片/
20 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 文档/
21 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 下载/
22 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 音乐/
23 drwxr-xr-x. 2 siguyuan siguyuan 6 4月 21 02:16 桌面/
24

```

Fence 105

8.3 pwd: 查看当前的绝对路径

```

1  # 查看当前用户在哪个目录中, 所在的目录一般称之为工作目录
2  [root@VM-8-14-centos ~/luffy/get/onepiece]# pwd
3  /root/luffy/get/onepiece          # 当前工作目录

```

Fence 106

8.4 touch 命令：创建文件

- 使用touch命令可以创建一个新的空文件, 如果指定的文件是已存在的, 只会更新文件的修改日期, 对内容没有任何影响。推荐使用vim命令

```
1  # 语法: touch 文件名
2
3  # 查看目录信息
4  [root@VM-8-14-centos ~/luffy]# ll
5  total 8
6  drwxr-xr-x 3 root root 4096 Jan 25 17:38 get
7  -rw-r--r-- 2 root root 37 Jan 25 17:26 onepiece.txt
8
9  # 创建一个新的文件 robin.txt
10 [root@VM-8-14-centos ~/luffy]# touch robin.txt
11
12 # 再次查看目录中的文件信息, 发现 robin.txt是空的, 大小为 0
13 [root@VM-8-14-centos ~/luffy]# ll
14 total 8
15 drwxr-xr-x 3 root root 4096 Jan 25 17:38 get
16 -rw-r--r-- 2 root root 37 Jan 25 17:26 onepiece.txt
17 -rw-r--r-- 1 root root 0 Jan 25 17:54 robin.txt
18
19 # touch 后的参数指定一个已经存在的文件名
20 [root@VM-8-14-centos ~/luffy]# touch onepiece.txt
21
22 # 继续查看目录中的文件信息, 发现文件时间被更新了: 37 Jan 25 17:26 --> 37 Jan 25 17:54
23 [root@VM-8-14-centos ~/luffy]# ll
24 total 8
25 drwxr-xr-x 3 root root 4096 Jan 25 17:38 get
26 -rw-r--r-- 2 root root 37 Jan 25 17:54 onepiece.txt
27 -rw-r--r-- 1 root root 0 Jan 25 17:54 robin.txt
```

Fence 107

8.5 whoami：查看当前用户

```
1  [siguyuan@10 ~]$ whoami
2  siguyuan
```

Fence 108

8.6 su 用户名：切换用户

- Linux是一个多用户的操作系统, 可以同时登陆多个用户, 因此很多时候需要在多个用户之间切换, 用户切换需要使用su或者su -。使用su只切换用户, 当前的工作目录不会变化, 但是使用 su -不仅会切换用户也会切换工作目录, 工作目录切换为当前用户的家目录。

从用户A切换到用户B, 如果还想再切换回用户A, 可以直接使用 exit。

```
1  # 只切换用户, 工作目录不变
2  $ su 用户名
```

```

3  # 举例:
4  robin@OS:~/Linux$ su luffy
5  Password:                # 需要输入luffy用户的密码
6  luffy@OS:/home/robin/Linux$ # 工作目录不变
7
8  # 切换用户和工作目录, 会自动跳转到当前用户的家目录中
9  $ su - 用户名
10 # 举例:
11 robin@OS:~/Linux$ su - luffy
12 Password:            # 需要输入luffy用户的密码
13 luffy@OS:~$ pwd
14 /home/luffy          # 工作目录变成了luffy的家目录
15
16 # 回到原来的用户
17 $ exit
18

```

Fence 109

- Ubuntu切换root用户, 需要在普通用户下, 使用 `su - root` 进行切换
 - 设置root用户名时, 出现大于等于8个字符, 且不能为简单数字的提示时, 可以无视 (root密码设置为123456)

8.7 which 命令: 查看命令路径

- which命令可以查看要执行的命令所在的实际路径, 命令解析器工作的时候也会搜索这个目录。需要注意的是该命令只能查看非内建的shell指令所在的实际路径, 有些命令是直接写到内核中的, 无法查看。

我们使用的大部分shell命令都是放在系统的/bin或者/usr/bin目录下:

```

1  # 由于使用的Linux版本不同, 得到的路径也会有不同
2  [root@VM-8-14-centos ~]# which ls
3  alias ls='ls --color=auto'
4      /usr/bin/ls
5
6  [root@VM-8-14-centos ~]# which date
7  /usr/bin/date
8
9  [root@VM-8-14-centos ~]# which cp
10 alias cp='cp -i'
11      /usr/bin/cp
12
13 [root@VM-8-14-centos ~]# which mv
14 alias mv='mv -i'
15      /usr/bin/mv
16
17 [root@VM-8-14-centos ~]# which pwd
18 /usr/bin/pwd

```

Fence 110

8.8 echo 命令：打印到屏幕

```
1 # 打印字符
2 [siguyuan@10 ~]$ echo PATH
3 PATH
4
5 # 打印变量值
6 [siguyuan@10 ~]$ echo $PATH
7 /usr/local/sbin:/sbin:/bin:/usr/sbin:/usr/bin:/root/bin
8
```

Fence 111

8.9 cp 命令：拷贝

- 拷贝文件 ==> 文件不存在得到新文件, 文件存在就覆盖

```
1 # `语法：cp 要拷贝的文件 得到的文件`
2
3 # `场景1：文件A，将A拷贝一份得到文件B（拷贝之前文件B不存在，拷贝后生成文件B）`
4 $ cp 文件A 文件B
5
6 # `场景2：文件A存在的，文件B也是存在的，执行下边的拷贝 ==> 文件A覆盖文件B`
7 $ cp 文件A 文件B
```

Fence 112

- 拷贝目录 ==> 目录不存在得到新目录, 该目录被拷贝到存在的目录中

```
1 # 拷贝目录需要参数 -r （递归）
2 # 场景1：目录A，通过拷贝得到不存在的目录B
3 $ cp 目录A 目录B -r
4
5 # 场景2：目录A存在的，目录B也是存在的，执行下边的拷贝 ==> 目录A会被拷贝并将其放到目录B中
6 $ cp 目录A 目录B -r
7
8 # 场景3：把A目录里的某一个或者多个文件拷贝到B目录中
9 $ cp A/a.txt B # 拷贝 A目录中的 a.txt 到目录B中
10 $ cp A/* B -r # 拷贝 A目录中的所有文件到目录B中，不能确定A目录中是否有子目录，因此需要加 -r
11
```

Fence 113

8.10 mv 命令：移动

- 文件目录的移动


```

1 # 语法: mv 要移动的文件 目录
2 # 有一个文件A, 移动到目录B中
3 # 其中A可以是文件也可以是目录, B必须是目录而且必须是存在的
4 $ mv A B

```

- 文件改名

Fence 114

```

1 # 语法: mv 要改名的文件 新名字(原来是不存在的, 这点很重要)
2 # 其中A可以是文件也可以是目录, 并且是存在的, B原来是不存在的
3 $ mv A B

```

Fence 115

- 文件覆盖

```

1 # 语法: mv 存在文件A 存在的文件B
2 # 其中A是文件(非目录)并且是存在的, B也是一个文件(非目录)并且也存在
3 # A文件中的内容覆盖B文件中的内容, A文件被删除, 只剩下B文件
4 $ mv A B

```

Fence 116

8.11 tree 命令: 目录结构显示

- 该命令的作用是以树状结构显示目录, tree工具默认是没有的, 需要手动安装, 系统版本不同安装方式也不尽相同:

- ubuntu: `sudo apt install tree`

- centos: `sudo yum install tree`

- 如果是基于管理员用户安装软件, 不需要加sudo。该命令有一个常用参数 -L, 即 (layer) 显示目录的层数。

- 下载时, 需要修改yum的镜像源:

- 切换到root用户

- 备份:

```
cp /etc/yum.repos.d/CentOS-Base.repo /etc/yum.repos.d/CentOS-Base.repo.bak
```

```
cp /etc/yum.repos.d/epel.repo /etc/yum.repos.d/epel.repo.bak
```

- 配置镜像源(使用的阿里云镜像)

```
wget -O /etc/yum.repos.d/CentOS-Base.repo http://mirrors.aliyun.com/repo/Centos-7.repo
```

```
wget -O /etc/yum.repos.d/epel.repo http://mirrors.aliyun.com/repo/epel-7.repo
```

- 重启yum

```
yum clean all
```

```
yum makecache
```

```

1 # 语法格式
2 $ tree [-L n]          # 查看当前目录的结构, n为显示的目录层数
3 $ tree 目录名 [-L n]   # 查看指定目录的结构, n为显示的目录层数
4
5 # 只显示1层
6 [root@VM-8-14-centos ~]# tree -L 1

```

```

7  .
8  |-- ace
9  |-- file
10 |-- ipc.tar.gz
11 |-- link.lnk -> /root/luffy/onepiece.txt
12 `-- luffy
13
14 # 显示2层目录
15 [root@VM-8-14-centos ~]# tree -L 2
16 .
17 |-- ace
18 |   `-- brother
19 |-- file
20 |   |-- dir
21 |   |-- haha.tar.gz
22 |   |-- hello
23 |   |-- link -> /root/file/test
24 |   |-- pipe-2
25 |   |-- subdir
26 |   `-- test
27 |-- ipc.tar.gz
28 |-- link.lnk -> /root/luffy/onepiece.txt
29 `-- luffy
30     |-- get
31     `-- onepiece.txt
32

```

Fence 117

8.12 创建删除目录

- 创建目录
 - 目录的创建分为两种, 一种是创建单个目录, 另一种是一次性创建多层目录, 使用的命令是mkdir, 后边参数是要创建的目录的名字, 如果是多层目录需要添加参数-p。
 - 关于创建的目录所在的路径可以是相对路径, 也可以是绝对路径。

```

1  # 单层目录
2  mkdir 新目录的名字
3  # 创建多个单层目录
4  mkdir 目录1 目录2
5
6  # 多层目录, 需要加参数 -p
7  mkdir -p parent/child/baby1/baby2

```

Fence 118

- 删除目录
 - 如果要删除已经存在的路径一共有两种方式, 可以使用rmdir或者rm
 - rmdir: **只能删除空目录**, 有点low, 不好用
 - rm: 可以删除文件也可以删除目录, 如果删除的是目录, 需要加参数 -r, 意思是递归(recursion)

- rm命令还有另外两个经常使用的参数:
 - -i: 删除的时候给提示
 - -f: 强制删除文件, 没有提示直接删除并且不能恢复, 慎用
 - -fi: 效果互斥。一起使用的时候, 谁在后面, 谁的生效。

```
1 # 1. low, 矮穷矬, 只能删除空目录
2 rmdir 目录名
3
4 # 2. 高大上, 可以删除目录也可以删除文件
5 # 删除目录需要加参数 -r, 递归的意思, 删除之后是不能恢复的
6 rm -r 目录名
```

Fence 119

8.13 查看文件内容

- 如果想要查看文件内容方式有很多, 最常用的是vim, 下面介绍一下vim以外的一些的一些方式:

8.13.1 cat

- 该命令可以将文件内容显示到终端, 由于终端是有缓存的, 因此能显示的字节数也是受限制的。如果文件太大数据就不能完全显示出来了, 因此该命令适合查看比较小的文件内容。

```
1 cat 文件名
2
3 #输出行号
4 cat -n 文件名
```

Fence 120

8.13.2 more

- 该命令比cat要高级一点, 我们可以以翻页的方式查看文件中的内容, 使用方式如下:

```
1 $ more 文件名
2 # 快捷键
3 - 回车: 显示下一行
4 - 空格: 向下滚动一屏
5 - b: 返回上一屏
6 - q: 退出more
```

Fence 121

8.13.3 less

- 该命令和more命令差不多, 我们可以以翻页的方式查看文件中的内容, 使用方式如下:

```
1 $ less 文件名
2 # 快捷键
3 - b: 向上翻页
4 - 空格: 向后翻页
5 - 回车: 显示下一行
6 - 上下键: 上下滚动
7 - q: 退出
```

Fence 122

8.13.4 head

- 使用该命令可以查看文件头部的若干行信息, 使用方式如下:

```
1 # 默认显示文件的前10行
2 $ head 文件名
3 # 指定显示头部的前多少行
4 $ head -行数 文件名
```

Fence 123

8.13.5 tail

- 使用该命令可以查看文件尾部的若干行信息, 使用方式如下:

```
1 # 默认显示文件的后10行
2 $ tail 文件名
3 # 指定显示尾部的最后多少行
4 $ tail -行数 文件名
```

Fence 124

8.14 链接创建

- 链接分两种类型: **软连接**和**硬链接**。软连接相当于windows中的快捷方式, 硬链接前边也已经介绍过了文件并不会进行拷贝, 只是多出一个新的文件名并且硬链接计数会加1。
- 软连接

```
1 # 语法: ln -s 源文件路径 软链接文件的名字(可以带路径)
2
3 # 查看目录文件
4 [root@VM-8-14-centos ~/luffy]# ll
5 total 8
6 drwxr-xr-x 3 root root 4096 Jan 25 17:27 get
7 -rw-r--r-- 1 root root 37 Jan 25 17:26 onepiece.txt
8
9 # 给 onepiece.txt 创建软连接, 放到子目录 get 中
10 [root@VM-8-14-centos ~/luffy]# ln -s /root/luffy/onepiece.txt get/link.lnk
11 [root@VM-8-14-centos ~/luffy]# ll get
12 total 4
13 lrwxrwxrwx 1 root root 24 Jan 25 17:27 link.lnk -> /root/luffy/onepiece.txt
14 drwxr-xr-x 2 root root 4096 Jan 24 21:37 onepiece
15
```

```
16 # 在创建软链接的时候，命令中的 源文件路径 建议使用 绝对路径，这样才能保证创建出的软链接文件在任
    # 意目录中移动都可以访问到链接的那个源文件。
```

Fence 125

• 硬链接

```
1 # 语法: ln 源文件 硬链接文件的名字(可以带路径)
2
3 # 创建硬链接文件，放到子目录中
4 [root@VM-8-14-centos ~/luffy]# ln onepiece.txt get/link.txt
5
6 # 查看链接文件和硬链接计数，从 1 --> 2
7 [root@VM-8-14-centos ~/luffy]# ll get
8 total 8
9 lrwxrwxrwx 1 root root 24 Jan 25 17:27 link.lnk -> /root/luffy/onepiece.txt
10 -rw-r--r-- 2 root root 37 Jan 25 17:26 link.txt
11 drwxr-xr-x 2 root root 4096 Jan 24 21:37 onepiece
12
13 # 硬链接和软链接不同，它是通过文件名直接找对应的硬盘地址，而不是基于路径，因此 源文件使用相对路
    # 径即可，无需为其制定绝对路径。
14
15 # 目录是不允许创建硬链接的。
```

Fence 126

8.15 修改文件权限

- 文件权限是针对文件所有者, 文件所属组用户, 其他人这三类人而言的, 对应的操作指令是chmod。设置方式也有两种，分别为文字设定法和数字设定法。
- 文字设定法是通过一些关键字r, w, x, -来描述用户对文件的操作权限。
- 数字设定法是通过一些数字 0, 1, 2, 4, 5, 6, 7 来描述用户对文件的操作权限。

• 文字设定法

```
1 #chmod
2 # 语法格式: chmod who [+|-|=] mod 文件名
3     - who:
4         - u: user -> 文件所有者
5         - g: group -> 文件所属组用户
6         - o: other -> 其他
7         - a: all, 以上是三类人 u+g+o
8     - 对权限的操作:
9         +: 添加权限
10        -: 去除权限
11        =: 权限的覆盖
12    - mod: 权限
13        r: read, 读
```

```

14         w: write, 写
15         x: execute, 执行
16         -: 没有权限
17
18     # 将文件所有者权限设置为读和执行, 也就是权限覆盖
19     robin@OS:~/Linux$ chmod u=rx b.txt
20     robin@OS:~/Linux$ ll b.txt
21     -r-xrw-r-- 2 robin robin 2929 Apr 14 18:53 b.txt*
22
23     # 给其他人添加写和执行权限
24     robin@OS:~/Linux$ chmod o+wx b.txt
25     robin@OS:~/Linux$ ll b.txt
26     -r-xrw-rwx 2 robin robin 2929 Apr 14 18:53 b.txt*
27
28     # 给文件所属组用户和其他用户去掉读和执行权限
29     robin@OS:~/Linux$ chmod go-rx b.txt
30     robin@OS:~/Linux$ ll b.txt
31     -r-x-w--w- 2 robin robin 2929 Apr 14 18:53 b.txt*
32
33     # 将文件所有者,文件所属组用户,其他人权限设置为读+写+执行
34     robin@OS:~/Linux$ chmod a=rwx b.txt
35     robin@OS:~/Linux$ ll b.txt
36     -rwxrwxrwx 2 robin robin 2929 Apr 14 18:53 b.txt*
37

```

• 数字设定法

Fence 127

```

1     # 语法格式: chmod [+|-|=] mod 文件名
2     - 对权限的操作:
3         +: 添加权限
4         -: 去除权限
5         =: 权限的覆盖, 等号可以不写
6     - mod: 权限描述, 所有权限都放开是 7
7         - 4: read, r
8         - 2: write, w
9         - 1: execute, x
10        - 0: 没有权限
11        二进制:111 每一位对应一种权限, 即rwx
12
13    # 分解: chmod 0567 a.txt
14
15        0           5           6           7
16    八进制      文件所有者  文件所属组用户  其他人
17        r + x      r + w      r+w+x
18
19    ##### 举例 #####
20    # 查看文件 c.txt 的权限
21    robin@OS:~/Linux$ ll c.txt
22    -rwxrwxrwx 2 robin robin 2929 Apr 14 18:53 c.txt*
23
24    # 文件所有者去掉执行权限, 所属组用户去掉写权限, 其他人去掉读+写权限
25    robin@OS:~/Linux$ chmod -123 c.txt
26    robin@OS:~/Linux$ ll c.txt

```

```

27 -rw-r-xr-- 2 robin robin 2929 Apr 14 18:53 c.txt*
28
29 # 文件所有者添加执行权限, 所属组用户和其他人权限不变
30 robin@OS:~/Linux$ chmod +100 c.txt
31 robin@OS:~/Linux$ ll c.txt
32 -rwxr-xr-- 2 robin robin 2929 Apr 14 18:53 c.txt*
33
34 # 将文件所有者,文件所属组用户,其他人权限设置为读+写, 没有执行权限
35 robin@OS:~/Linux$ chmod 666 c.txt
36 robin@OS:~/Linux$ ll c.txt
37 -rw-rw-rw- 2 robin robin 2929 Apr 14 18:53 c.txt

```

Fence 128

8.16 修改文件所有者

- 默认情况下, 文件是通过哪个用户创建出来的, 就属于哪个用户, 这个用户属于哪个组, 文件就属于哪个组。如果有特殊需求, 可以修改文件所有者, 对应的操作命令是chown。因为修改文件所有者就跨用户操作, 普通用户没有这个权限, 需要借助管理员权限才能完成该操作。
- 普通用户借助管理员权限执行某些shell命令, 需要在命令前加关键字sudo, 但是普通用户默认是没有使用 sudo的资格的, 需要修改 **/etc/sudoers 文件**。
 - 给普通用户添加sudo权限
 - 先切换到root用户
 - 在root用户下修改这个文件属性, 给其添加写权限
 - 修改文件内容, 把普通用户sanji添加进去, 保存退出
 - 将文件权限修改为原来的 400 (r-----)
 - 切换到用户 sanji, 这时候就可以使用 sudo了, 权限添加成功

```

1  # 1. 切换到root用户
2  $ su root
3  Password:      # 输入root用户的密码
4
5  # 2. 修改文件权限, 暴力一点没有关系, 反正还需要改回去, 直接 777 就行
6  $ chmod 777 sudoers
7
8  # 3. 使用 vim 打开这个文件
9  $ vim sudoers
10
11 # 4. 在文件中找到这一行, 在文件偏尾部的位置
12 root    ALL=(ALL)        ALL
13
14 # 5. 照葫芦画瓢, 在下边新添加一行内容如下:
15 root    ALL=(ALL)        ALL      # 原来的内容
16 sanji    ALL=(ALL)        ALL      # 新添加的行, 将用户名指定为 sanji 即可
17
18 # 6. 保存退出 (先按esc, 然后输入 :wq)
19 # 7. 将文件改回原来的权限
20 $ chmod 400 sudoers

```

```

1  # 语法1-只修改所有者:
2  $ sudo chown 新的所有者 文件名
3
4  # 语法2-同时修改所有者和所属组:
5  $ sudo chown 新的所有者:新的组名 文件名
6
7  # 查看文件所有者: b.txt 属于 robin 用户
8  robin@OS:~/Linux$ ll b.txt
9  -rw-rw-rw- 2 robin robin 2929 Apr 14 18:53 b.txt
10
11 # 将 b.txt 的所有者修改为 luffy
12 robin@OS:~/Linux$ sudo chown luffy b.txt
13 [sudo] password for robin:
14 robin@OS:~/Linux$ ll b.txt
15 -rw-rw-rw- 2 luffy robin 2929 Apr 14 18:53 b.txt
16
17 # 修改文件所有者和文件所属组
18 # 查看文件所有者和所属组
19 robin@OS:~/Linux$ ll b.txt
20 -rw-rw-rw- 2 luffy robin 2929 Apr 14 18:53 b.txt
21
22 # 同时修改文件所有者和文件所属组
23 robin@OS:~/Linux$ sudo chown robin:luffy b.txt
24 robin@OS:~/Linux$ ll b.txt
25 -rw-rw-rw- 2 robin luffy 2929 Apr 14 18:53 b.txt

```

Fence 130

8.17 修改文件所属组

- 普通用户没有修改文件所属组的权限, 如果需要修改需要借助管理员权限才能完成, 需要使用的命令是chgrp。当然了这个属性的修改也可以使用chown命令来完成。

```

1  # 只修改文件所属的组, 普通用户没有这个权限, 借助管理员的权限
2  # 语法: sudo chgrp 新的组 文件名
3
4  # 查看文件所属组信息
5  robin@OS:~/Linux$ ll b.txt
6  -rw-rw-rw- 2 robin luffy 2929 Apr 14 18:53 b.txt
7
8  # 修改文件所属的组
9  robin@OS:~/Linux$ sudo chgrp robin b.txt
10 robin@OS:~/Linux$ ll b.txt
11 -rw-rw-rw- 2 robin robin 2929 Apr 14 18:53 b.txt
12

```

Fence 131

8.18 重定向命令

- 关于重定向使用最多的是就是输出重定向, 顾名思义就是修改输出的数据的位置, 通过重定向操作我们可以非常方便的进行文件的复制, 或者文件内容的追加。输出重定向使用的不是某个关键字而是符号 >或者>>。

>: 将文件内容写入到指定文件中, 如果文件中已有数据, 则会使用新数据覆盖原数据

>>: 将输出的内容追加到指定的文件尾部

```
1  # 输出的重定向举例: printf默认是要将数据打印到终端, 可以修改默认的输出位置 => 重定向到某个文件中
2  # 关键字 >
3  # 执行一个shell指令, 获得一个输出, 这个输出默认显示到终端, 如果要将其保存到文件中, 就可以使用重定向
4  # 如果当前目录下test.txt不存在, 会被创建, 如果存在, 内容被覆盖
5  robin@OS:~/Linux$ date > test.txt
6  # 日期信息被写入到文件 test.txt中
7  robin@OS:~/Linux$ cat test.txt
8  Wed Apr 15 09:37:52 CST 2020
9
10 # 如果不希望文件被覆盖, 而是追加, 需要使用 >>
11 in@OS:~/Linux$ date >> test.txt
12 # 日期信息被追加到 test.txt中
13 robin@OS:~/Linux$ cat test.txt
14 Wed Apr 15 09:37:52 CST 2020
15 Wed Apr 15 09:38:44 CST 2020
16
17 # 继续追加信息
18 robin@OS:~/Linux$ date >> test.txt
19 robin@OS:~/Linux$ cat test.txt
20 Wed Apr 15 09:37:52 CST 2020
21 Wed Apr 15 09:38:44 CST 2020
22 Wed Apr 15 09:39:03 CST 2020
```

Fence 132

8.19 添加/删除用户

- 作为一个普通用户是没有给系统添加新用户这个权限的, 如果想要添加新用户可以先切换到root用户, 或者基于普通用户为其添加管理员权限来完成新用户的添加。添加新用户需要使用adduser/useradd命令来完成。

普通用户没有添加/删除用户的权限 (sudo权限), 需要授权,

8.19.1 添加用户

```
1  # 添加用户
2  # sudo -> 使用管理员权限执行这个命令 (建议使用这个。CentOS7设置时, 也不会设置密码)
3  $ sudo adduser 用户名
4
5  # centos
6  $ sudo useradd 用户名
7
```

```

8 # ubuntu (-m : 如果用户家目录不存在, 自动创建 (也可以使用-d手动指定) -s: 当前用户的命令解释器
  为/bin/bash -p: 设置密码 (不建议使用, 会暴露密码, 所有后面使用修改密码, 设置密码)) 不添加参数, 创建
  的用户不完整
9 $ sudo useradd -m -s /bin/bash 用户名
10
11 # 在使用 adduser 添加新用户的时候, 有的Linux版本执行完命令就结束了, 有的版本会提示设置密码等用户信息
12 robin@OS:~/Linux$ sudo adduser lisi
13 Adding user `lisi' ...
14 Adding new group `lisi' (1004) ...
15 Adding new user `lisi' (1004) with group `lisi' ...
16 Creating home directory `/home/lisi' ...
17 Copying files from `/etc/skel' ...
18 Enter new UNIX password:
19 Retype new UNIX password:
20 passwd: password updated successfully
21 Changing the user information for lisi
22 Enter the new value, or press ENTER for the default # 直接回车
23     Full Name []:
24     Room Number []:
25     Work Phone []:
26     Home Phone []:
27     Other []:
28 Is the information correct? [Y/n] y
29

```

Fence 133

- 当新用户添加完毕之后, 我们可以切换到新添加的用户下, 用来检测是否真的添加成功了, 另外我们也可以使用其他方式来检验, 首先在 /home目录中会出现一个和用户名同名的目录, 这就是新创建的用户的家目录, 另外我们还可以查看 /etc/passwd文件, 里边记录着新添加的用户的更加详细的信息:



```

tuffy:x:1003:1003::/home/tuffy:/bin/bash
robin:x:1004:1004::/home/Robin:/bin/bash
lisi:x:1005:1005::/home/lisi:/bin/bash

```

用户名:加密之后的密码:用户ID:所属组ID::用户家目录:用户默认使用的命令解析器

Figure 18

8.19.2 删除用户

- 删除用户并不是将/home下的用户家目录删除就完事儿了, 我们需要使用userdel命令才能删除用户在系统中的用户ID和所属组ID等相关信息, 但是需要注意的是在某些Linux版本中用户虽然被删除了, 但是它的家目录却没有被删除, 需要我们手动将其删除。

```

1 # 删除用户, 添加参数 -r 就可以一并删除用户的家目录了
2 $ sudo userdel 用户名 -r
3
4 # 删除用户 lisi
5 $ sudo userdel lisi -r
6
7 # 使用deluser不能添加参数-r, 家目录不能被删除, 需要使用 rm 命令删除用户的家目录, 比如:
8 $ sudo rm /home/用户名 -r

```

- 由于Linux的版本很多, 删除用户对应的操作指令不是唯一的, 经测试在 CentOS 版本只支持 `userdel`命令, 在 Ubuntu中既支持 `userdel` 也支持 `deluser`命令。

8.20 添加/删除用户组

- 默认情况下, 只要创建新用户就会得到一个同名的用户组, 并且这个用户属于这个组。一般情况下不需要创建新的用户组, 如果有需求可以使用 `groupadd`添加用户组, 使用 `groupdel`删除用户组。

由于普通用户没有添加删除用户组权限, 因此需要在管理员 (root) 用户下操作, 或者在普通用户下借助管理员权限完成该操作 (sudo权限)。

```
1 # 基于普通用户创建新的用户组
2 $ sudo groupadd 组名
3
4 # 基于普通用户删除已经存在的用户组
5 $ sudo groupdel 组名
```

Fence 135

- 如果验证用户组是否添加成功了, 可以查看 `/etc/group`文件, 里边有用户组相关的信息:



Figure 19

- 在Ubuntu中添加删除用户组可以使用 `addgroup/groupadd` 和 `delgroup/groupdel`
在CentOS中添加和删除用户只能使用 `groupadd` 和 `groupdel`
我们只需要通过 `which` 命令名 查看, 就能知道该Linux版本是不是支持使用该命令了。

8.21 修改密码

- Linux中设置用户密码和修改用户密码的方式是一样的, 修改用户密码又分几种情况: 修改当前用户密码, 普通用户A修改其他普通用户密码, 普通用户A修改root用户密码, root用户修改普通用户密码。修改密码需要使用`passwd`命令。当创建了一个普通用户却没有提示指定密码, 或者忘记了用户密码都可以通过该命令来实现自己重置密码的需求。
 - 当前用户修改自己的密码, 默认是有权限操作的
 - 当前普通用户修改其他用户密码, 默认没有权限, 需要借助管理员权限才能完成操作
 - 当前普通用户修改root用户密码, 默认没有权限, 需要借助管理员权限才能完成操作
 - root用户修改其他普通用户密码, 默认有权限, 可以直接修改

```

1  # passwd
2  # 修改当前用户
3  $ passwd
4
5  # 修改非当前用户密码
6  $ sudo passwd 用户名
7
8  # 修改root
9  $ sudo passwd root
10

```

Fence 136

- 通过以上介绍的相关命令我们可以知道，如果让一个普通用户可以使用管理员权限执行一些指令其实是非常危险的，因此普通用户默认是没有使用 sudo 的权限的，必须授权才能使用，工作场景中授权操作一定要慎重，要三思。

8.22 压缩/解压缩命令

- 常用的压缩包格式包含：tar.gz, .tgz, .tar.bz2, .zip, .rar, .tar.xz

8.22.1 tar

- 在Linux操作系统中默认自带两个原始的压缩工具分别是 gzip和bzip2, 但是它们都有先天的缺陷, **不能打包压缩文件, 每个文件都会生成一个单独的压缩包, 并且压缩之后不会保留原文件, 这是一件权能忍婶也不能忍的事情。**

Linux中自带一个打包工具, 叫做tar, **默认情况下该工具是不能进行压缩操作的**, 在这种情况下tar和gzip, bzip2就联姻了, 各自发挥各自的优势, Linux下最强大的打包压缩工具至此诞生。

8.22.1.1 压缩(.tar.gz .tar.bz2 .tgz)

- 如果使用tar完成文件压缩, 涉及的参数如下, 在使用过程中参数没有先后顺序:
 - c: 创建压缩文件
 - z: 使用gzip的方式进行文件压缩
 - j: 使用bzip2的方式进行文件压缩
 - v: 压缩过程中显示压缩信息, 可以省略不写 (显示log日志)
 - f: 指定压缩包的名字
- 一般认为 .tgz 文件就等同于 .tar.gz 文件, 因此它们的压缩方式是相同的。 (必须写明压缩技术一后缀名, 否则不好解压缩)

```

1  # 语法:
2  $ tar 参数 生成的压缩包的名字 要压缩的文件(文件或者目录)
3
4  # 备注: 关于生成的压缩包的名字, 建议使用标准后缀, 方便识别:
5      - 压缩使用 gzip 方式, 标准后缀格式为: .tar.gz
6      - 压缩使用 bzip2 方式, 标准后缀格式为: .tar.bz2
7
8
9  # 使用gzip压缩
10 # 查看目录内容
11 [root@VM-8-14-centos ~/luffy]# ls

```

```

12  get onepiece.txt robin.txt
13
14  # 压缩目录中所有文件，如果要压缩某几个文件，直接指定文件名即可
15  [root@VM-8-14-centos ~/luffy]# tar zcvf all.tar.gz *
16  get/                                # ..... 压缩信息
17  get/link.lnk                        # ..... 压缩信息
18  get/onepiece/                       # ..... 压缩信息
19  get/onepiece/haha.txt
20  get/link.txt
21  onepiece.txt
22  robin.txt
23
24  # 查看目录文件，多了一个压缩文件 all.tar.gz
25  [root@VM-8-14-centos ~/luffy]# ls
26  all.tar.gz get onepiece.txt robin.txt
27
28
29  # 使用bzip2压缩
30  # 查看目录内容
31  [root@VM-8-14-centos ~/luffy]# ls
32  all.tar.gz get onepiece.txt robin.txt
33
34  # 压缩目录中除 all.tar.gz 的文件和目录
35  [root@VM-8-14-centos ~/luffy]# tar jcvf part.tar.bz2 get onepiece.txt robin.txt
36  get/                                # ..... 压缩信息
37  get/link.lnk                        # ..... 压缩信息
38  get/onepiece/                       # ..... 压缩信息
39  get/onepiece/haha.txt
40  get/link.txt
41  onepiece.txt
42  robin.txt
43
44  # 查看目录信息，多了一个压缩文件 part.tar.bz2
45  [root@VM-8-14-centos ~/luffy]# ls
46  all.tar.gz get onepiece.txt part.tar.bz2 robin.txt
47

```

Fence 137

8.22.1.2 解压缩 (.tar.gz .tar.bz2 .tgz)

- 如果使用tar进行文件的解压缩, 涉及的参数如下, 在使用过程中参数没有先后顺序:
 - x: 释放压缩文件内容
 - z: 使用gzip的方式进行文件压缩, 压缩包后缀为.tar.gz
 - j: 使用bzip2的方式进行文件压缩, 压缩包后缀为.tar.bz2
 - v: 解压缩过程中显示解压缩信息
 - f: 指定压缩包的名字
- 使用以上参数是将压缩包解压到当前目录, 如果需要解压到指定目录, 需要指定参数 -C。一般认为 .tgz 文件就等同于 .tar.gz 文件, 解压缩方式是相同的。解压的语法格式如下:

```
1  # 语法1: 解压到当前目录中
```

```
2 $ tar 参数 压缩包名
3
4 # 语法2: 解压到指定目录中
5 $ tar 参数 压缩包名 -C 解压目录
6
7 # 使用gzip解压缩
8 # 查看目录文件信息
9 [root@VM-8-14-centos ~/luffy]# ls
10 all.tar.gz get onepiece.txt part.tar.bz2 robin.txt temp
11
12 # 将 all.tar.gz 压缩包解压缩到 temp 目录中
13 [root@VM-8-14-centos ~/luffy]# tar zxvf all.tar.gz -C temp
14 get/ # 解压缩文件信息
15 get/link.lnk # 解压缩文件信息
16 get/onepiece/ # 解压缩文件信息
17 get/onepiece/haha.txt # 解压缩文件信息
18 get/link.txt
19 onepiece.txt
20 robin.txt
21
22 # 查看temp目录内容, 都是从压缩包中释放出来的
23 [root@VM-8-14-centos ~/luffy]# ls temp/
24 get onepiece.txt robin.txt
25
26
27 # 使用bzip2解压缩
28 # 删除 temp 目录中的所有文件
29 [root@VM-8-14-centos ~/luffy]# rm temp/* -rf
30
31 # 查看 luffy 目录中的文件信息
32 [root@VM-8-14-centos ~/luffy]# ls
33 all.tar.gz get onepiece.txt part.tar.bz2 robin.txt temp
34
35 # 将 part.tar.bz2 中的文件加压缩到 temp 目录中
36 [root@VM-8-14-centos ~/luffy]# tar jxvf part.tar.bz2 -C temp
37 get/ # 解压缩文件信息
38 get/link.lnk # 解压缩文件信息
39 get/onepiece/ # 解压缩文件信息
40 get/onepiece/haha.txt # 解压缩文件信息
41 get/link.txt
42 onepiece.txt
43 robin.txt
44
45 # 查看 temp 目录中的文件信息
46 [root@VM-8-14-centos ~/luffy]# ls temp/
47 get onepiece.txt robin.txt
48
49
```

8.22.2 zip

- zip格式的压缩包在Linux中也是很常见的,在某些版本中需要安装才能使用(高版本的Ubuntu与Centos以及自带)

- Ubuntu

```
1 | $ sudo apt install zip      # 压缩
2 | $ sudo apt install unzip    # 解压缩
```

Fence 139

- CentOS

```
1 | # 因为 centos 可以使用 root 用户登录, 基于 root 用户安装软件, 不需要加 sudo
2 | $ sudo yum install zip      # 压缩
3 | $ sudo yum install unzip    # 解压缩
```

Fence 140

8.22.2.1 压缩 (.zip)

- 使用zip压缩目录需要注意一点, **必须要添加参数 -r**, 这样才能将子目录中的文件一并压缩, 如果要压缩的文件中没有目录, 该参数就可以不写了。

另外使用zip压缩文件, 会自动生成文件后缀.zip, 因此就不需要额外指定了。

```
1 | # 语法: 后自动添加压缩包后缀 .zip, 如果要压缩目录, 需要添加参数 r
2 | $ zip [-r] 压缩包名 要压缩的文件
3 |
4 |
5 | # 例子
6 | # 查看目录文件信息
7 | [root@VM-8-14-centos ~/luffy]# ls
8 | get onepiece.txt robin.txt temp
9 |
10 | # 压缩目录 get 和文件 onepiece.txt robin.txt 得到压缩包 all.zip(不需要指定后缀, 自动添加)
11 | [root@VM-8-14-centos ~/luffy]# zip all onepiece.txt robin.txt get/ -r
12 |   adding: onepiece.txt (stored 0%)
13 |   adding: robin.txt (stored 0%)
14 |   adding: get/ (stored 0%)
15 |   adding: get/link.lnk (stored 0%)          # 子目录中的文件也被压缩进去了
16 |   adding: get/onepiece/ (stored 0%)         # 子目录中的文件也被压缩进去了
17 |   adding: get/onepiece/haha.txt (stored 0%) # 子目录中的文件也被压缩进去了
18 |   adding: get/link.txt (stored 0%)          # 子目录中的文件也被压缩进去了
19 |
20 | # 查看目录文件信息, 多了一个压缩包文件 all.zip
21 | [root@VM-8-14-centos ~/luffy]# ls
22 | all.zip get onepiece.txt robin.txt temp
23 |
```

Fence 141

8.22.2.2 解压缩 (.zip)

- 对应zip格式的文件解压缩,必须要使用 unzip 命令,和压缩的时候使用的命令是不一样的。如果压缩包中的文件要解压到指定目录需要指定参数-d,默认是解压缩到当前目录中。

```
1  # 语法1: 解压到当前目录中
2  $ unzip 压缩包名
3
4  # 语法: 解压到指定目录, 需要添加参数 -d
5  $ unzip 压缩包名 -d 解压目录
6
7
8  # 例子
9  # 查看目录文件信息
10 [root@VM-8-14-centos ~/luffy]# ls
11 all.zip  get  onepiece.txt  robin.txt  temp
12
13 # 删除 temp 目录中的所有文件
14 [root@VM-8-14-centos ~/luffy]# rm temp/* -rf
15
16 # 将 all.zip 解压缩到 temp 目录中
17 [root@VM-8-14-centos ~/luffy]# unzip all.zip -d temp/
18 Archive:  all.zip
19   extracting: temp/onepiece.txt          # 释放压缩的子目录中的文件
20   extracting: temp/robin.txt             # 释放压缩的子目录中的文件
21     creating: temp/get/
22   extracting: temp/get/link.lnk
23     creating: temp/get/onepiece/
24   extracting: temp/get/onepiece/haha.txt # 释放压缩的子目录中的文件
25   extracting: temp/get/link.txt
26
27 # 查看 temp 目录中的文件信息
28 [root@VM-8-14-centos ~/luffy]# ls temp/
29 get  onepiece.txt  robin.txt
30
```

Fence 142

8.22.3 rar

- rar这种压缩格式在Linux中并不常用,这是Windows常用的压缩格式,如果想要在Linux压缩和解压这种格式的文件需要额外安装对应的工具,不同版本的Linux安装方式也是不同的。
- Ubuntu

```
1  # 执行在线下载命令即可
2  $ sudo apt install rar
```

Fence 143

- CentOS

```
1  # 需要下载安装包, 官方地址: https://www.rarlab.com/download.htm
```



```

2 # 从下载页面找到 Linux 版本的下载链接并复制链接地址，使用 wget 下载到本地
3 $ wget https://www.rarlab.com/rar/rarlinux-x64-6.0.0.tar.gz
4
5 # 将下载得到的 rarlinux-x64-6.0.0.tar.gz 压缩包解压缩，得到解压目录 rar
6 $ tar zxvf rarlinux-x64-6.0.0.tar.gz
7
8 # 将得到的解压目录移动到 /opt 目录中（因为/opt软件安装目录，移动是为了方便管理，不移动也没事儿）
9 # 该操作需要管理员权限，我是使用 root 用户操作的
10 $ mv ./rar /opt
11
12 # 给 /opt/rar 目录中的可执行程序添加软连接，方便命令解析器可以找到该压缩命令
13 $ ln -s /opt/rar/rar /usr/local/bin/rar
14 $ ln -s /opt/rar/unrar /usr/local/bin/unrar
15
16
17 #其他版本的linux，也适用

```

Fence 144

8.22.3.1 压缩 (.rar)

- 使用rar压缩过程中的注意事项和zip是一样的, **如果压缩的是目录, 需要指定参 -r**, 如果只压缩文件就不需要添加了。压缩过程中需要使用参数 **a (archive)**, 压缩归档的意思。

rar工具在生成压缩包的时候也会自动添加后缀, 名字为.rar, 因此我们只需要指定压缩包的名字。

```

1 # 文件压缩，需要使用参数 a，压缩包名会自动添加后缀 .rar
2 # 如果压缩了目录，需要加参数 -r
3 # 语法：
4 $ rar a 压缩包名 要压缩的文件 [-r]
5
6 # 举例
7 # 查看目录文件信息
8 [root@VM-8-14-centos ~/luffy]# ls
9 get onepiece.txt robin.txt temp
10
11 # 压缩文件 onepiece.txt, robin.txt 和目录 get/ 到要是文件 all.rar 中
12 [root@VM-8-14-centos ~/luffy]# rar a all onepiece.txt get/ robin.txt -r
13
14 RAR 6.00 Copyright (c) 1993-2020 Alexander Roshal 1 Dec 2020
15 Trial version Type 'rar -?' for help
16
17 Evaluation copy. Please register.
18
19 Creating archive all.rar
20
21 Adding onepiece.txt OK
22 Adding get/link.lnk OK # 子目录中的文件也被压缩了
23 Adding get/onepiece/haha.txt OK # 子目录中的文件也被压缩了
24 Adding get/link.txt OK # 子目录中的文件也被压缩了
25 Adding robin.txt OK
26 Adding get/onepiece OK
27 Done

```

```

28 [root@VM-8-14-centos ~/luffy]# ls
29 all.rar  get  onepiece.txt  robin.txt  temp
30

```

Fence 145

8.22.3.2 解压缩 (.rar)

- 解压缩.rar格式的文件的的时候, 可以使用 rar也可以使用unrar, 操作方式是一样的, **需要添加参数 x**, 默认是将压缩包内容释放到当前目录中, 如果要释放到指定目录直接指定解压目录名即可, 不需要使用任何参数。

```

1  # 解压缩: 需要参数 x
2  # 语法: 解压缩到当前目录中
3  $ rar/unrar x 压缩包名字
4
5  # 语法: 解压缩到指定目录中
6  rar/unrar x 压缩包名字 解压目录
7
8
9  # 例子
10 # 查看目录文件信息
11 [root@VM-8-14-centos ~/luffy]# ls
12 all.rar  get  onepiece.txt  robin.txt  temp
13
14 # 删除 temp 目录中的所有文件
15 [root@VM-8-14-centos ~/luffy]# rm temp/* -rf
16
17 # 将 all.rar 中的文件解压缩到 temp 目录中
18 [root@VM-8-14-centos ~/luffy]# rar x all.rar temp/
19
20 RAR 6.00   Copyright (c) 1993-2020 Alexander Roshal   1 Dec 2020
21 Trial version           Type 'rar -?' for help
22
23
24 Extracting from all.rar
25
26 Extracting  temp/onepiece.txt                OK
27 Creating    temp/get                          OK
28 Extracting  temp/get/link.lnk                 OK      # 子目录文件也被释放出来了
29 Extracting  temp/get/link.lnk                 OK      # 子目录文件也被释放出来了
30 Extracting  temp/get/link.lnk                 OK      # 子目录文件也被释放出来了
31 Creating    temp/get/onepiece                 OK
32 Extracting  temp/get/link.lnk                 OK      # 子目录文件也被释放出来了
33 Extracting  temp/get/link.lnk                 OK      # 子目录文件也被释放出来了
34 Extracting  temp/get/onepiece/haha.txt        OK
35 Extracting  temp/get/link.txt                 OK
36 Extracting  temp/robin.txt                    OK
37 All OK
38
39 # 查看 temp 目录中文件信息
40 [root@VM-8-14-centos ~/luffy]# ls temp/
41 get  onepiece.txt  robin.txt
42

```

Fence 146

8.22.4 xz

- .xz格式的文件压缩和解压缩都相对比较麻烦, 通过一个命令是完不成对应的操作的, 需要通过两步操作才行。并且操作过程中需要使用tar工具进行打包, 压缩使用的则是 xz工具。

8.22.4.1 压缩 (.tar.xz)

- 创建文件的步骤如下, 首先 将需要压缩的文件打包 `tar cvf xxx.tar files`, 然后再对打包文件进行压缩 `xz -z xxx.tar`, 这样我们就可以得到一个打包之后的压缩文件了。

使用 xz工具压缩文件的时候需要添加参数 `-z`

```
1  # 语法:
2  # 第一步
3  $ tar cvf xxx.tar 要压缩的文件
4  # 第二步, 最终得到一个xxx.tar.xz 格式的压缩文件
5  $ xz -z xxx.tar
6
7
8  # 例子
9  # 查看目录文件信息
10 [root@VM-8-14-centos ~/luffy]# ls
11 get onepiece.txt robin.txt temp
12
13 # 将文件 onepiece.txt, robin.txt 和目录 get 打包到 all.tar 中
14 [root@VM-8-14-centos ~/luffy]# tar cvf all.tar onepiece.txt robin.txt get/
15 onepiece.txt
16 robin.txt
17 get/
18 get/link.lnk
19 get/onepiece/
20 get/onepiece/haha.txt
21 get/link.txt
22
23 # 查看目录文件信息, 多了一个打包文件 all.tar
24 [root@VM-8-14-centos ~/luffy]# ls
25 all.tar get onepiece.txt robin.txt temp
26
27 # 使用 xz 工具压缩打包文件 all.tar
28 [root@VM-8-14-centos ~/luffy]# xz -z all.tar
29
30 # 最终得到了压缩之后的打包文件 all.tar.xz
31 [root@VM-8-14-centos ~/luffy]# ls
32 all.tar.xz get onepiece.txt robin.txt temp
33
34
```

8.22.4.2 解压缩(.tar.xz)

- 解压缩的步骤和压缩的步骤相反, 需要先解压缩, 然后将文件包中的文件释放出来。

使用xz工具解压需要使用参数-d。

```
1  # 语法:
2  # 第一步: 压缩包解压缩, 得到 xxx.tar
3  $ xz -d xxx.tar.xz
4  # 第二步: 将 xxx.tar 中的文件释放到当前目录
5  $ tar xvf xxx.tar
6
7  # 例子
8  # 查看目录中文件信息, 有一个 all.tar.xz
9  [root@VM-8-14-centos ~/luffy]# ls
10 all.tar.xz  get  onepiece.txt  robin.txt  temp
11
12 # 将 all.tar.xz 解压缩, 得到 all.tar
13 [root@VM-8-14-centos ~/luffy]# xz -d all.tar.xz
14
15 # 查看目录文件, 得到了 all.tar
16 [root@VM-8-14-centos ~/luffy]# ls
17 all.tar  get  onepiece.txt  robin.txt  temp
18
19 # 释放 all.tar 到当前目录
20 [root@VM-8-14-centos ~/luffy]# tar xvf all.tar
21 onepiece.txt
22 robin.txt
23 get/
24 get/link.lnk
25 get/onepiece/
26 get/onepiece/haha.txt
27 get/link.txt
28
```

Fence 148

8.23 搜索命令

- Linux为我们提供了很多的用于文件搜索的命令, 如果需求比较简单可以使用 locate, which, whereis 来完成搜索, 如果需求复杂可以使用 find, grep进行搜索。其中 which在前边已经介绍过了, whereis局限性太大, 不常用。

8.23.1 find

- find是Linux中一个搜索功能非常强大的工具, 它的主要功能是根据文件的属性, 查找对应的磁盘文件, 比如说我们常用的一些属性 **文件名, 文件类型, 文件大小, 文件的目录深度** 等, 下面基于这些常用数据来讲解一些具体的使用方法。

如果想用通过属性对文件进行搜索, 只需要指定出属性对应的参数就可以了, 下面将依次进行介绍。

8.23.1.1 文件名 (-name)

- 根据文件名进行搜索有两种方式: **精确查询**和**模糊查询**。关于**模糊查询**必须要使用对应的通配符, 最常用的有两个, 分别为 ***** 和 **?**。其中 ***** 可以匹配零个或者多个字符, **?**用于匹配单个字符。

如果我们进行模糊查询, 建议 (非必须) 将**带有通配符的文件名**写到**引号中** (单引号或者双引号都可以), 这样可以避免搜索命令执行失败 (如果不加引号, 某些情况下会这样)。

如果需要根据文件名进行搜索, 需要使用参数 `-name`。

```
1  # 语法格式: 根据文件名搜索
2  $ find 搜索的路径 -name 要搜索的文件名
3
4
5  # 例子
6  # 模式搜索
7  # 搜索 root 家目录下文件后缀为 txt 的文件
8  [root@VM-8-14-centos ~]# find /root -name "*.txt"
9  /root/luffy/get/onepiece/haha.txt
10 /root/luffy/get/onepiece/onepiece.txt
11 /root/luffy/get/onepiece.txt
12 /root/luffy/get/link.txt
13 /root/luffy/robin.txt
14 /root/luffy/onepiece.txt
15 /root/ace/brother/finally/die.txt
16 /root/onepiece.txt
17
18 #####
19
20 # 精确搜索
21 # 搜索 root 家目录下文件名为 onepiece.txt 的文件
22 [root@VM-8-14-centos ~]# find /root -name "onepiece.txt"
23 /root/luffy/get/onepiece/onepiece.txt
24 /root/luffy/get/onepiece.txt
25 /root/luffy/onepiece.txt
26 /root/onepiece.txt
27
28
29 作者: 苏丙楹
30 链接: https://subingwen.cn/linux/search/#1-1-%E6%96%87%E4%B8%B6%E5%90%8D-name
31 来源: 爱编程的大丙
32 著作权归作者所有。商业转载请联系作者获得授权, 非商业转载请注明出处。
```

8.23.1.2 文件类型 (-type)

- 在前边文章中已经介绍过Linux中有7中文件类型, 如果有去求我们可以通过find对指定类型的文件进行搜索, 该属性对应的参数为 `-type`。其中每种类型都有对应的关键字, 如下表:

文件类型	类型的字符描述
普通文件类型	f
目录类型	d

文件类型	类型的字符描述
软连接类型	l
字符设备类型	c
块设备类型	b
管道类型	p
本地套接字类型	s

Table 22

```
1 # 语法格式：
2 $ find 搜索的路径 -type 文件类型
3
4
5 # 例子
6 # 搜索 root 用户家目录下，软连接类型的文件
7 [root@VM-8-14-centos ~]# find /root -type l
8 /root/link.lnk
9 /root/luffy/get/link.lnk
10 /root/file/link
```

Fence 150

8.23.1.3 文件大小 (-size)

- 如果需要根据文件大小进行搜索, 需要使用参数 -size。关于**文件大小的单位有很多**，可以根据实际需求选择，常用的分别有 **k(小写)**, **M(大写)**, **G(大写)**。

在进行文件大小判断的时候，**需要指定相应的范围**，涉及的符号有两个分别为: **加号(+)** 和 **减号(-)**，下面具体说明其使用方法：

```
1 # 语法格式：
2 $ find 搜索的路径 -size [+|-]文件大小
3 - 文件大小需要加单位：
4 - k (小写)
5 - M (大写)
6 - G (大写)
7
8 # -size 4k 表示的区间为 (4-1k, 4k]，表示一个区间，大于3k,小于等于4k
9 # -size -4k: [0k, 4-1k]，表示一个区间，大于等于0 并且 小于等于3k
10 # -size +4k: (4k, 正无穷)，表示搜索大于4k的文件
11
12
13
14 # 例子
15 # 搜索当前目录下 大于3M的所有文件 (file>3M)
16 $ find ./ -size +3M
17
18 # 搜索当前目录下 大于等于0M并且小于等于2M的文件 (0M <= file <=2M)
19 $ find ./ -size -3M
20
21 # 搜索当前目录下 大于2M并且小于等于3M的文件 (2M < file <=3M)
```

```

22 $ find ./ -size 3M
23
24 # 搜索当前目录下 大于1M 并且 小于等于 3M 的文件
25 $ find ./ -size +1M -size -4M

```

Fence 151

8.23.1.4 目录层级

- 因为Linux的目录是树状结构, 所有目录可能有很多层, 在搜索某些属性的时候可以指定只搜索某几层目录, 相关的参数有两个, 分别为: **-maxdepth**和**-mindepth**。

这两个参数不能单独使用, 必须和其他属性一起使用, 也就是搜索某几层目录中满足条件的文件。

- maxdepth: 最多搜索到第多少层目录,
- mindepth: 至少从第多少层开始搜索

```

1 # 查找文件, 从根目录开始, 最多搜索5层, 这个文件叫做 *.txt (1 <= 层数 <= 5)
2 $ sudo find / -maxdepth 5 -name "*.txt"
3
4 # 查找文件, 从根目录开始, 至少从第5层开始搜索, 这个文件叫做 *.txt (层数>=5层)
5 $ sudo find / -mindepth 5 -name "*.txt"

```

Fence 152

8.23.1.5 同时执行多个操作

- 在搜索文件的时候如果想在find执行多个操作, 通过使用管道(|)的方式是行不通的, 比如下面的操作:

```

1 # 比如: 通过find搜索最多两层目录中后缀为 .txt 的文件, 然后再查看这些满足条件的文件的详细信息
2 # 在find操作中直接通过管道操作多个指令, 最终输出的结果是有问题, 因此不能直接这样使用
3 $ find ./ -maxdepth 2 -name "*.txt" | ls -l
4 total 612
5 drwxr-xr-x 2 root root 4096 Jan 26 18:11 a
6 -rw-r--r-- 1 root root 269 Jan 26 17:44 a.c
7 drwxr-xr-x 3 root root 4096 Jan 26 18:39 ace
8 drwxr-xr-x 4 root root 4096 Jan 25 15:21 file
9 lrwxrwxrwx 1 root root 24 Jan 25 17:27 link.lnk -> /root/luffy/onepiece.txt
10 drwxr-xr-x 4 root root 4096 Jan 26 18:39 luffy
11 -r--r--r-- 1 root root 37 Jan 26 16:50 onepiece.txt
12 -rw-r--r-- 1 root root 598314 Dec 2 02:07 rarlinux-x64-6.0.0.tar.gz
13
14 # 数量多了后, 会出现只执行管道后面的指令

```

Fence 153

- 如果想要实现上面的需求, 需要在find中使用 **exec**, **ok**, **xargs**, 这样就可以在find命令执行完毕之后, 再执行其他的子命令了。

8.23.1.5.1 exec

- exec 是find的参数, 可以在**exec**参数后添加其他需要被执行的shell命令。

find 添加了 exec 参数之后, 命令的尾部需要加一个后缀 **{ \;**, 注意 **}**和****之间需要有一个空格。

在参数-exec后添加的shell命令处理的是find搜索之后的结果, find的结果会作为 新添加的shell命令 的输入, 最后在终端上输出最终的处理结果。

```

1 # 语法:
2 $ find 路径 参数 参数值 -exec shell命令2 {} \;
3
4
5 # 例子
6 # 搜索最多两层目录, 文件名后缀为 .txt的文件
7 $ find ./ -maxdepth 2 -name "*.txt"
8 ./luffy/robin.txt
9 ./luffy/onepiece.txt
10 ./onepiece.txt
11
12 # 搜索到满足条件的文件之后, 再继续查看文件的详细属性信息
13 $ find ./ -maxdepth 2 -name "*.txt" -exec ls -l {} \;
14 -rw-r--r-- 1 root root 0 Jan 25 17:54 ./luffy/robin.txt
15 -r--r--r-- 2 root root 37 Jan 25 17:54 ./luffy/onepiece.txt
16 -r--r--r-- 1 root root 37 Jan 26 16:50 ./onepiece.txt

```

Fence 154

8.23.1.5.2 ok (具有交互)

- -ok 和 -exec 都是find命令的参数, 使用方式类似, 但是这个参数是交互式的, 在处理find的结果的时候, 会向用户发起询问, 比如在删除搜索结果的时候, 为了保险起见, 就需要询问机制了。

语法格式如下:

```

1 # 语法: 其实就是将 -exec 替换为 -ok, 其他都不变
2 $ find 路径 参数 参数值 -ok shell命令2 {} \;
3
4
5 # 例子
6 # 搜索到了2个满足条件的文件
7 $ find ./ -maxdepth 1 -name "*.txt"
8 ./aaaaa.txt
9 ./english.txt
10
11 # 查找并显示文件详细信息
12 $ find ./ -maxdepth 1 -name "*.txt" -ok ls -l {} \;
13 < ls ... ./aaaaa.txt > ? y # 同意显示文件详细信息
14 -rw-rw-r-- 1 robin robin 10 Apr 17 11:34 ./aaaaa.txt
15 < ls ... ./english.txt > ? n # 不同意显示文件详细信息, 会跳过显示该条信息
16
17 # 什么时候需要交互呢? ---> 删除文件的时候
18 $ find ./ -maxdepth 1 -name "*.txt" -ok rm -rf {} \;
19 < rm ... ./aaaaa.txt > ? y # 同意删除
20 < rm ... ./english.txt > ? n # 不同意删除
21
22 # 删除一个文件之后再次进行相同的搜索
23 $ find ./ -maxdepth 1 -name "*.txt"
24 ./english.txt # 只剩下了一个.txt 文件

```

Fence 155

8.23.1.5.3 xargs

- 在使用find的-exec参数的时候,需要在指定的子命令尾部添加几个特殊字符{} \;,一不小心就容易写错,有一种看起来更加直观、书写更加简便的方式,我们可以使用 **xargs** 替换掉-exec参数,而且在处理数据的时候xargs更高效。有了xargs的加持我们就可以在find命令中直接使用管道完成前后命令的数据传递,使用方法如下:

```
1  # 在find 中 使用 xargs 关键字我们就可以使用管道了, 否则使用管道也不会起作用
2  # 将 find 搜索的结果通过管道 (|) 传递给后边的shell命令继续处理
3  $ find 路径 参数 参数值 | xargs shell命令2
4
5
6  # 例子
7  # 查找文件
8  $ find ./ -maxdepth 1 -name "*.cpp"
9  ./occi.cpp
10 ./main.cpp
11 ./test.cpp
12
13 # 查找文件, 并且显示文件的详细信息
14 robin@OS:~$ find ./ -maxdepth 1 -name "*.cpp" | xargs ls -l
15 -rw-r--r-- 1 robin robin 2223 Mar  2  2020 ./main.cpp
16 -rw-r--r-- 1 robin robin 1406 Mar  2  2020 ./occi.cpp
17 -rw-r--r-- 1 robin robin 2015 Mar  1  2020 ./test.cpp
18
19
20 # xargs的效率比使用 -exec 效率高
21     -exec: 将find查询的结果逐条传递给后边的shell命令
22     -xargs: 将find查询的结果一次性传递给后边的shell命令
23
24
```

Fence 156

8.23.2 grep

- 和find不同 grep 命令用于查找文件里符合条件的字符串。grep命令中有几个常用参数,下面介绍一下:
 - r: 如果需要搜索目录中的文件内容,需要进行递归操作,必须指定该参数
 - i: 对应要搜索的关键字,忽略字符大小写的差别
 - n: 在显示符合样式的那一行之前,标示出该行的列数编号
 - 对应要搜索的文件内容,建议放到引号(单或者双引号)中,因为关键字中可能有特殊字符,或者有空格,从而导致解析错误。

```
1  # 语法格式:
2  $ grep "搜索的内容" 搜索的路径/文件 参数
3
4
5  # 例子
6  # 搜索指定文件中是否有字符串 include
7  [root@VM-8-14-centos ~]# grep "include" a.c
8  #include <stdio.h>
```

```

9  #include <unistd.h>
10 #include <fcntl.h>
11
12 # 不区分大小写进行搜索
13 [root@VM-8-14-centos ~]# grep "INCLUDE" a.c
14 [root@VM-8-14-centos ~]# grep "INCLUDE" a.c -i
15 #include <stdio.h>
16 #include <unistd.h>
17 #include <fcntl.h>
18
19 # 搜索指定目录中哪些文件中包含字符串 include 并且显示关键字所在的行号
20 [root@VM-8-14-centos ~]# grep "include" ./ -rn
21 ./a.c:1:#include <stdio.h>
22 ./a.c:2:#include <unistd.h>
23 ./a.c:3:#include <fcntl.h>
24 ./luffy/get/e.c:1:#include <stdio.h>
25 ./luffy/get/e.c:2:#include <unistd.h>
26 ./luffy/get/e.c:3:#include <fcntl.h>
27 ./luffy/c.c:1:#include <stdio.h>
28 ./luffy/c.c:2:#include <unistd.h>
29 ./luffy/c.c:3:#include <fcntl.h>
30 ./ace/b.c:1:#include <stdio.h>
31 ./ace/b.c:2:#include <unistd.h>
32 ./ace/b.c:3:#include <fcntl.h>
33 ./bash_history:1449:grep "include" ./
34 ./bash_history:1451:grep "include" ./ -r
35 ./bash_history:1465:grep "include" a.c
36

```

Fence 157

8.23.3 locate

- 我们可以将locate看作是一个简化版的find, 使用这个命令我们可以**根据文件名搜索本地的磁盘文件**, 但是**locate的效率比find要高很多**。原因在于它不搜索具体目录, 而是搜索一个本地的数据库文件, 这个数据库中含有本地所有文件信息。Linux系统自动创建这个数据库, 并且每天自动更新一次, 所以使用locate命令查不到最新变动过的文件。为了避免这种情况, 可以在**使用locate之前, 先使用updatedb命令, 手动更新数据库**。

```

1  # 使用管理员权限更新本地数据库文件, root用户这样做
2  $ updatedb
3  # 非root用户需要加 sudo
4  $ sudo updatedb
5
6
7  # 常用参数
8  # 搜索所有目录下以某个关键字开头的文件
9  $ locate test      # 搜索所有目录下以 test 开头的文件
10
11 # 搜索指定目录下以某个关键字开头的文件, 指定的目录必须要使用绝对路径
12 $ locate /home/robin/test  # 指定搜索目录为 /home/robin/, 文件以 test 开头
13
14 # 搜索文件的时候, 忽略文件名的大小写, 使用参数 -i
15 $ locate TEST -i      # 文件名以小写的test为前缀的文件也能被搜索到
16

```

```
17 # 列出前N个匹配到的文件名称或路径名称，使用参数-n
18 $ locate test -n 5 # 搜索文件前缀为 test 的文件，并且只显示5条信息
19
20 # 基于正则表达式进行文件名匹配，查找符合条件的文件，使用参数 -r
21 # 使用该参数，需要有正则表达式基础
22 $ locate -r "\.cpp$" # 搜索以 .cpp 结尾的文件
23
24
```

Fence 158

- 正则表达式小科普:

- 在正则表达式中 `.` 可以匹配任意一个非 `\n` 的单字符
- 上边的命令中使用转译字符 `\` 对特殊字符转译, 就得到了普通的字符.
- 在正则表达式中 `$` 放到字符尾部, 表示字符串必须以这个字符结尾, 上边的命令中修饰的是字符 `p`
- 正则表达式中的 字符 `c` 和后边的字符 `p` 需要进行字节匹配, 没有特殊含义
- 通过上面的解释就能明白 `.cpp$` 说的就是以 `.cpp` 结尾的字